

基于逐步“填数”的对称 R/X 回归： Laminar L_1 -MILP 的精确实现、算例与完整源码

terminal_load_only_case33 实现审计稿

2026 年 9 月 4 日

摘要

本文给出一种面向终端量测配电网的结构化灵敏度矩阵回归实现。算法把每条候选公共路径写成对称块原子 $z_k z_k^T$ ，并令 R 、 X 两个通道共享同一组支撑集合、分别估计非负填充值。支撑集合两两只能嵌套或不交，从而形成 laminar family。每次前向迭代利用纯混合整数线性规划 (MILP) 在全部非空端子集中隐式搜索一个新的合法支撑，同时以 L_1 残差重新拟合全部既有原子权重和场景-终端截距。实现采用 `scipy.optimize.milp` 所调用的 HiGHS 求解器；只有当求解器状态、原始-对偶界或相对 gap 共同通过严格证书检查时才接受新原子。本文列出完整数学模型、代码映射、测试设计、复现实验命令和可证明性边界，并在附录中直接嵌入估计器、单元测试、结果表生成器和两套算例驱动脚本的最终源码。

目录

1 任务、实现范围与结论边界	3
2 回归模型与“填数”原子	3
2.1 多场景终端回归	3
2.2 对称块原子与矩阵分解	3
2.3 Laminar 条件及树解释	4
3 单轮精确 L_1 -MILP	4
3.1 决策变量	4
3.2 支撑与子矩阵的精确线性化	4
3.3 统一填充值的精确线性化	5
3.4 Laminar 与禁止重复约束	5
3.5 Fully-corrective 预测与 L_1 目标	6
4 固定支撑 LP 与完整前向算法	6
4.1 固定支撑 LP	6
4.2 算法流程	6
4.3 终止条件与验证选择	7

5 实现映射与可复现命令	7
5.1 代码结构	7
5.2 环境与执行命令	8
6 验证项目与算例结果	9
6.1 单元测试覆盖	9
6.2 small6 结果	9
6.3 IEEE case33 派生集成算例	10
7 RX75–RNJ 末端分块与有限候选域 MILP 联合	10
7.1 为何要区分硬信息与软候选	10
7.2 对角线预处理与候选域精确性	11
7.3 数据配置与四算例结果	11
8 严格可证明性边界与已知限制	11
9 建议的后续扩展	12
A 核心估计器完整源码	13
B 单元测试完整源码	39
C small6 算例驱动完整源码	50
D case33 集成算例驱动完整源码	62
E 实测结果表生成器完整源码	75
F RX75–RNJ 联合算例驱动完整源码	80
G RX75–RNJ 联合回归测试完整源码	92

1 任务、实现范围与结论边界

目标是控制回归所得 R/X 矩阵的对称性与结构复杂度：第 t 步只选择一个终端集合 $S_t \subseteq \{1, \dots, n\}$ ，并分别把同一个数 r_t 、 x_t 加到 R 、 X 的同一主子矩阵 $S_t \times S_t$ 上。本文实现遵循如下约定：

1. 对角线与非对角线执行相同填数操作；单终端独有路径由 singleton 支撑 $S_t = \{i\}$ 表示；
2. R 与 X 共享支撑，但 r_t 与 x_t 独立且非负；允许其中一个通道的权重为零；
3. 已选集合必须构成 laminar family，即任意两集合嵌套或不交；
4. 每轮只新增一个集合，但新增后对全部旧权重进行 fully-corrective 重新拟合；
5. 目标为平均绝对残差（MAE），因此扩展问题是纯 MILP，固定支撑问题是 LP；
6. 路径上的模型用连续时间块验证集的 one-standard-error 规则选择，测试集只用于最终评估。

这里的“精确”有严格限定：HiGHS 必须返回状态 0，并且报告的原始-对偶绝对 gap 不超过 10^{-10} ，或相对 MIP gap 不超过 10^{-10} ；此时当前有界参数域中的最佳单原子扩展才记为已证明全局最优。整个多步前向路径仍然是贪心路径，并不等价于固定 K 个原子的联合全局最优解。

2 回归模型与“填数”原子

2.1 多场景终端回归

设场景为 $s = 1, \dots, m$ ，场景 s 中有 T_s 个时刻，终端数为 n 。代码接收 `P_terminal`、`Q_terminal`、`drop_target` 三个同形状表格。对时刻 τ 及输出终端 i ，模型为

$$Y_{s\tau i} = \sum_{j=1}^n P_{s\tau j} R_{ij} + \sum_{j=1}^n Q_{s\tau j} X_{ij} + c_{si} + e_{s\tau i}. \quad (1)$$

其中 c_{si} 是场景-输出终端截距；其作用是吸收每个场景中不随时间变化的基准项。总观测数为

$$N_{\text{obs}} = n \sum_{s=1}^m T_s. \quad (2)$$

2.2 对称块原子与矩阵分解

对非空集合 S_k 定义二值向量

$$z_{ki} = \mathbb{I}\{i \in S_k\}, \quad \mathbf{z}_k \in \{0, 1\}^n, \quad (3)$$

以及原子

$$\mathbf{A}_k = \mathbf{z}_k \mathbf{z}_k^\top. \quad (4)$$

矩阵由非负权重叠加：

$$\mathbf{R} = \sum_{k=1}^K r_k \mathbf{A}_k, \quad \mathbf{X} = \sum_{k=1}^K x_k \mathbf{A}_k, \quad r_k, x_k \geq 0. \quad (5)$$

若 $S_k = \{3, 6, 8\}$ ，则 $S_k \times S_k$ 的九个位置（包括三个对角位置）同时增加 r_k 或 x_k 。

命题 1 (原子分解的直接保证). 由式 (5) 构造的 R 、 X 自动满足对称、逐元素非负和半正定。

证明. $\mathbf{A}_k = \mathbf{z}_k \mathbf{z}_k^\top$ 对称且逐元素非负; 对任意 $\mathbf{v}$, $\mathbf{v}^\top \mathbf{A}_k \mathbf{v} = (\mathbf{z}_k^\top \mathbf{v})^2 \geq 0$. 非负加权和保持这三项性质. $\square$

对角线无需额外自由变量. singleton 原子满足 $\mathbf{z}_k \mathbf{z}_k^\top = \mathbf{e}_i \mathbf{e}_i^\top$, 只改变 (i, i) 元素, 正好表达终端独有的末端路径。

2.3 Laminar 条件及树解释

定义 1 (Laminar family). 集合族 $\mathcal{F}$ 为 *laminar*, 当且仅当对任意 $A, B \in \mathcal{F}$, 有

$$A \cap B = \emptyset, \text{ 或 } A \subseteq B, \text{ 或 } B \subseteq A. \quad (6)$$

径向树上一条边的下游终端集合天然组成 laminar family。反之, 任意有限 laminar family 都可以嵌入某棵以终端为叶/观测节点的层次树; 未选择的层次可理解为零权重或被压缩的边。因此, 此约束比单纯的对称、非负、PSD 或 $M_{ii} \geq M_{ij}$ 更强。若允许交叉集合, 例如 $\{1, 2\}$ 与 $\{2, 3\}$, 所得 PSD 矩阵一般不再具有树公共路径协方差解释。

需要注意: 这一结构只给出终端 clade 层面的树解释, 并不识别隐藏节点的任意编号; 多个具有相同下游终端集合的串联物理边只能聚合为同一个原子权重。

3 单轮精确 L_1 -MILP

假设已经固定 K 个 laminar 支撑 $C_1, \dots, C_K$ 。下一轮同时搜索新支撑 S 、新填充值 r_+, x_+ , 并重拟合全部旧填充值。

3.1 决策变量

变量	含义	定义域
$r_k, x_k, k \leq K$	既有固定支撑的、在本轮重新拟合的填充值	$[0, \bar{r}]$ 、 $[0, \bar{x}]$
c_{si}	场景-终端截距	自由变量
$h_{s\tau i}$	残差绝对值的 epigraph 变量	$[0, +\infty)$
z_i	新支撑是否包含终端 i	$\{0, 1\}$
$y_{ij}, i \leq j$	$z_i z_j$, 新主子矩阵的位置指示	$[0, 1]$
r_+, x_+	新原子的统一填充值	$[0, \bar{r}]$ 、 $[0, \bar{x}]$
u_{ij}^R, u_{ij}^X	$r_+ y_{ij}$ 、 $x_+ y_{ij}$	相应非负有界变量
a_k, b_k, d_k	新集合相对 C_k 的子集、超集、不交选择	$\{0, 1\}$

实现只为 $i \leq j$ 保存 $y_{ij}, u_{ij}^R, u_{ij}^X$, 预测时以 $(\min(i, j), \max(i, j))$ 查找, 因而严格复用对称变量, 而不是事后对称化。

3.2 支撑与子矩阵的精确线性化

新集合非空:

$$\sum_{i=1}^n z_i \geq 1. \quad (7)$$

对角位置直接约束为

$$y_{ii} = z_i. \quad (8)$$

对 $i < j$ ，二元乘积 $y_{ij} = z_i z_j$ 的凸包线性描述为

$$y_{ij} \leq z_i, \quad y_{ij} \leq z_j, \quad (9)$$

$$y_{ij} \geq z_i + z_j - 1, \quad y_{ij} \geq 0. \quad (10)$$

尽管 y_{ij} 被声明为连续变量，整数可行点上它必然等于 0 或 1。

3.3 统一填充值的精确线性化

对所有 $i \leq j$ ，用有界连续变量与二元变量乘积的凸包约束表达 $u_{ij}^R = r_+ y_{ij}$ ：

$$0 \leq u_{ij}^R \leq \bar{r} y_{ij}, \quad (11)$$

$$u_{ij}^R \leq r_+, \quad (12)$$

$$u_{ij}^R \geq r_+ - \bar{r}(1 - y_{ij}). \quad (13)$$

X 通道同理：

$$0 \leq u_{ij}^X \leq \bar{x} y_{ij}, \quad (14)$$

$$u_{ij}^X \leq x_+, \quad (15)$$

$$u_{ij}^X \geq x_+ - \bar{x}(1 - y_{ij}). \quad (16)$$

当 $y_{ij} = 0$ 时强制 $u_{ij} = 0$ ；当 $y_{ij} = 1$ 时强制 $u_{ij}^R = r_+$ 、 $u_{ij}^X = x_+$ 。因此“选中一个相同行列子矩阵并统一填数”不是松弛近似，而是整数可行解上的精确等式。

3.4 Laminar 与禁止重复约束

对每个旧集合 C_k ，引入三元互斥关系

$$a_k + b_k + d_k = 1, \quad (17)$$

分别表示 $S \subseteq C_k$ 、 $C_k \subseteq S$ 、 $S \cap C_k = \emptyset$ 。对应 implication 直接写成

$$z_i \leq 1 - a_k, \quad i \notin C_k, \quad (18)$$

$$z_i \geq b_k, \quad i \in C_k, \quad (19)$$

$$z_i \leq 1 - d_k, \quad i \in C_k. \quad (20)$$

还需禁止新集合等于旧集合：

$$\sum_{i \in C_k} (1 - z_i) + \sum_{i \notin C_k} z_i \geq 1. \quad (21)$$

式 (17)–(21) 对每个 k 同时成立，因此新增后的整个集合族保持 laminar。

3.5 Fully-corrective 预测与 L_1 目标

对旧支撑 C_k 预计算特征

$$B_{s\tau ik}^P = z_{ki} \sum_{j=1}^n P_{s\tau j} z_{kj}, \quad (22)$$

$$B_{s\tau ik}^Q = z_{ki} \sum_{j=1}^n Q_{s\tau j} z_{kj}. \quad (23)$$

新原子通过上三角共享变量参与预测：令 $\pi(i, j) = (\min(i, j), \max(i, j))$ ，则

$$\hat{Y}_{s\tau i} = c_{si} + \sum_{k=1}^K \left(r_k B_{s\tau ik}^P + x_k B_{s\tau ik}^Q \right) + \sum_{j=1}^n \left(P_{s\tau j} u_{\pi(i,j)}^R + Q_{s\tau j} u_{\pi(i,j)}^X \right). \quad (24)$$

绝对残差的 epigraph 约束为

$$h_{s\tau i} \geq Y_{s\tau i} - \hat{Y}_{s\tau i}, \quad (25)$$

$$h_{s\tau i} \geq - \left(Y_{s\tau i} - \hat{Y}_{s\tau i} \right). \quad (26)$$

求解目标是

$$\min J_{K+1} = \frac{1}{N_{\text{obs}}} \sum_{s=1}^m \sum_{\tau=1}^{T_s} \sum_{i=1}^n h_{s\tau i}. \quad (27)$$

由于所有乘积均已线性化，式 (7)–(27) 构成纯 MILP。

4 固定支撑 LP 与完整前向算法

4.1 固定支撑 LP

给定 $\mathcal{F}_K = (S_1, \dots, S_K)$ 后，不再需要 z, y, u 或关系二元变量。只需优化 $r_k, x_k, c_{si}, h_{s\tau i}$ ，仍使用式 (25)–(27)。该问题是连续 LP，记其最优值为 J_K 。每次接受支撑后再求解此 LP，用于：

- 消除扩展 MILP 与最终固定支撑表达之间的数值差异；
- 重新校正所有旧原子权重，而不是冻结早期填充值；
- 识别 R 、 X 权重均低于阈值的零原子；
- 删除零原子后再次求解，并仅在目标未显著增加时接受删除。

4.2 算法流程

步骤 1. 读取训练场景并核对所有 P, Q, Y 维数、终端标签及有限性；验证集必须使用相同终端标签。

步骤 2. 若未指定 $\bar{r}, \bar{x}$ ，用训练数据的稠密 OLS 系数经对称化、非负截断和安全倍数生成初值。

步骤 3. 从空支撑族 $\mathcal{F}_0 = \emptyset$ 开始，求仅含场景–终端截距的固定支撑 LP，保存路径点 $K = 0$ 。

步骤 4. 在当前 $\mathcal{F}_K$ 上求解第 3 节的一原子扩展 MILP。它隐式考察全部 $2^n - 1$ 个非空集合，但只允许与所有旧集合嵌套或不交且不重复的集合。

步骤 5. 若求解器未证明全局最优，则拒绝 incumbent，并以“扩展未获最优证书”停止；若模型不可行，则以“无可行扩展”停止。

- 步骤 6.** 若任一最优权重达到当前上界的指定比例，则扩大对应上界并重求同一轮；超过最大扩界次数则停止。
- 步骤 7.** 计算精确单原子训练增益 $\Delta_K = J_K - J_{K+1}^*$ 。若其不超过绝对/相对容差，停止。
- 步骤 8.** 接受新支撑，求固定支撑 LP 进行 fully-corrective polishing；谨慎删除双通道均为零且删除后目标不变的原子。
- 步骤 9.** 保存矩阵、支撑、权重、训练 MAE、验证 MAE/标准误及完整求解器诊断，继续下一轮。
- 步骤 10.** 到达原子上限、laminar family 上限或其他停止条件后，在整条路径上执行验证集 one-SE 选择。

4.3 终止条件与验证选择

扩展被证明最优且满足

$$J_K - J_{K+1}^* \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max\{|J_K|, \epsilon_{\text{mach}}\} \quad (28)$$

时，得到“当前支撑族下不存在有价值的单原子合法扩展”的证书。非空 laminar family 的基数至多为 $2n - 1$ ，代码也以此校验 `max_atoms`。

若有验证数据，对每个路径点固定 R/X ，并沿用该路径点由训练 LP 得到的场景-输出终端截距；验证目标不参与截距重估。随后按连续时间块计算 MAE 标准误。令验证 MAE 最小的路径点为 $K_{\min}$ ，阈值为

$$\theta = \text{MAE}_{\text{val}}(K_{\min}) + \text{SE}_{\text{val}}(K_{\min}). \quad (29)$$

最终选择满足 $\text{MAE}_{\text{val}}(K) \leq \theta$ 的第一个（即最小复杂度）路径点。

5 实现映射与可复现命令

5.1 代码结构

文件/接口	角色	关键内容
<code>rnj_wzzt_core/rnj_wzzt/estimation/laminar_l1_milp.py</code>	核心估计器	稀疏 MILP 构造、固定支撑 LP、一原子扩展、完整前向路径、验证选择与诊断
<code>solve_fixed_support_l1</code>	固定支撑求解	给定 laminar 支撑，对全部 r/x 和截距进行 L_1 全量重拟合
<code>solve_best_laminar_extension_l1</code>	单步精确搜索	在全部合法非空子集中搜索全局最优一原子扩展
<code>fit_laminar_l1_sensitivity</code>	主入口	扩界、增益停止、零原子剪枝、完整路径及 one-SE 选择
<code>build_matrices_from_atoms</code>	矩阵重构	在包含对角线的 $S_k \times S_k$ 上统一累加 r_k/x_k
<code>experiments/run_laminar_l1_milp.py</code>	small6 算例驱动	无噪声/污染响应两种制度、时间块划分、固定截距外推和审计文件导出

文件/接口	角色	关键内容
experiments/run_laminar_l1_case33_integration.py	case33 集成驱动	四物理叶精确路径和 32 终端限时规模压力测试；候选搜索不使用真值
experiments/render_laminar_l1_results_tex.py	结果表生成器	直接读取落盘 CSV/JSON，生成本文输入的两个 LaTeX 结果片段
tests/test_laminar_l1_milp.py	单元与穷举测试	证书、矩阵装配、穷举 oracle、fully-corrective、终止、截距和物理重构共 12 项

5.2 环境与执行命令

从 terminal_load_only_case33 目录运行：

```

1 python -m pip install -e .
2 python -m pytest tests/test_laminar_l1_milp.py -q
3 python -m experiments.run_laminar_l1_milp \
4   --output outputs/laminar_l1_milp_small6_final \
5   --seed 20260902 --time-count 72 \
6   --train-count 48 --validation-count 12 \
7   --max-atoms 11 --outlier-fraction 0.05 \
8   --time-limit 120
9 python -m experiments.run_laminar_l1_case33_integration \
10  --output outputs/laminar_l1_milp_case33_bound2_final \
11  --seed 20260903 --aggregate-time-limit 30 \
12  --hybrid-time-limit 60 --coefficient-bound 2
13 python -m experiments.render_laminar_l1_results_tex

```

Listing 1: 安装、测试与算例复现命令

默认算例采用 small6 六终端网络、平方电压 LinDistFlow 真值、连续 72 点 P/Q 轨迹，按时间顺序划分为 48/12/12 个训练/验证/测试点。两种响应制度为：

1. **noiseless**: 严格使用 $Y = PR_{\text{true}}^T + QX_{\text{true}}^T$;
2. **laplace_outliers**: 加入尺度为信号中位绝对偏差 5% 的 Laplace 噪声，并在约 5% 响应元素上加入基准尺度 150% 的带符号 gross outlier。

首要测试指标使用训练集估计的固定截距预测测试段；测试段重新拟合截距所得 MAE 只作为干扰截距诊断，不能替代严格 held-out 指标。

主要输出包括 summary.csv、metrics.json、all_paths.csv、all_extension_attempts.csv、all_selected_atoms.csv，以及每个制度下的 R/X 估计、选中原子、路径、求解器状态和测试残差。本次实测环境为 SciPy 1.18.1/HiGHS；表格生成器以这些落盘文件为唯一数值来源。

6 验证项目与算例结果

6.1 单元测试覆盖

测试	判据	状态
Laminar 判定与矩阵性质	嵌套/不交为真、交叉为假；矩阵对称、非负、PSD，并核对对角与非对角累加	通过
singleton 与输入校验	singleton 只改变一个对角元；拒绝负权、重复索引和越界索引	通过
最优证书真值表	区分 LP、MIP 零 gap、绝对原始-对偶 gap、缺失 gap 与非最优状态	通过
无噪声单原子恢复	恢复 $\{0, 2, 3\}$ 、 $r = 0.7$ 、 $x = 0.3$ ，目标近零且对角线同填	通过
MILP 与穷举 oracle	枚举 $n = 4$ 全部合法集合并逐一求 LP；目标、最优支撑及手算 MAE 一致	通过
交叉集合排除	已有 $\{0, 1\}$ 时排除交叉的 $\{1, 2\}$	通过
无合法扩展	$n = 1$ 且 singleton 已选时得到确定性 infeasible 状态 2，不返回伪支撑	通过
三原子 fully-corrective	确定性正负单位激励下按 root-child-singleton 恢复；旧 root 权重在第二轮被重新校正	通过
一般前向路径	每一路径点与独立固定支撑 LP 一致；MAE 单调，且始终 laminar、对称、非负、PSD	通过
固定训练截距验证	验证目标发生常数漂移时不在验证集重估截距；同时核对 profiled 诊断口径	通过
零信号终止	目标完全由截距解释时不接纳零权伪原子，停止于无显著单原子增益	通过
small6 物理重构	按物理边下游终端集合聚合原子，精确重构平方电压 R/X 真矩阵	通过

定向测试命令返回 12 passed；随后项目完整测试返回 104 passed。其中穷举 oracle 独立枚举离散支撑，手工矩阵预测又独立重算 MAE；这两层检查避免只用同一个辅助函数自治地“验证自己”。

6.2 small6 结果

表 3: small6 最终拟合、固定训练截距验证与测试结果

制度	路径点	选中原子	停止原因	训练 MAE	验证 MAE	测试 MAE	总时间/s
无噪声	9	9	无显著单原子增益	0.000e+00	7.220e-19	6.987e-19	21.27
Laplace+ 异常值	8	8	无显著单原子增益	1.429e-04	1.270e-04	1.239e-04	19.81

表 4: small6 矩阵、clade 与求解证书审计

制度	R 相对 Fro.	X 相对 Fro.	匹配/真值	Precision	Recall	F1	全部扩展有证书
无噪声	4.588e-16	5.045e-15	3/3	1.000	1.000	1.000	是
Laplace+ 异常值	1.797e-01	3.242e-01	3/3	1.000	1.000	1.000	是

最大报告绝对 gap 为 0.000e+00, 最大相对 MIP gap 为 0.000e+00; 两个制度均未触及系数上界。

数据来源: `outputs/laminar_l1_milp_small6_final/summary.csv`。验证和首要测试指标均固定使用训练 LP 返回的截距。

报告结果时至少同时列出: 路径选择位置/原子数、停止原因、训练/验证/测试 MAE、 R/X 矩阵误差、非平凡 clade precision/recall/F1、所有扩展是否均证明最优、最大 MIP gap、运行时间和是否触及系数上界。不能只报告一个 F1 或只报告训练拟合误差。

6.3 IEEE case33 派生集成算例

四物理叶聚合版本用于验证完整 7 原子路径; 32 终端 hybrid 版本仅尝试一个扩展, 用来如实测量全子集 MILP 的规模边界。两者均由平方电压 LinDistFlow 真值生成响应, 但真支撑与真系数不进入候选搜索。

表 5: IEEE case33 派生算例的求解状态与路径结果

算例	n	时限/s	接受原子	选中原子	停止原因	状态全 0	全部有证书	总时间/s
四物理叶聚合	4	30	7	7	达到 laminar 上限	是	是	1.35
32 终端 hybrid	32	60	0	0	扩展未获最优证书	否	否	60.59

表 6: IEEE case33 派生算例的外推、矩阵与 clade 指标

算例	训练 MAE	验证 MAE	测试 MAE	R 相对 Fro.	X 相对 Fro.	F1
四物理叶聚合	0.000e+00	4.091e-17	3.397e-17	9.968e-16	3.067e-15	1.000
32 终端 hybrid	5.972e-03	2.328e-02	1.168e-02	1.000e+00	1.000e+00	0.000

四叶聚合算例的 7 次扩展均取得严格证书, 矩阵达到机器精度且两个非平凡 clade 全部匹配。32 终端算例在上界 2、单轮 60 秒内状态为 1, 最大报告 gap 为 1.000。实现按设计拒绝未获证书的 incumbent, 故空模型指标只表示规模压力测试, 不能解释为已完成的 32 终端回归结果。

数据来源: `outputs/laminar_l1_milp_case33_bound2_final/metrics.json`; 真值仅用于生成响应和事后评分, 未用于候选支撑搜索。

7 RX75-RNJ 末端分块与有限候选域 MILP 联合

7.1 为何要区分硬信息与软候选

最终联合实现只把 bootstrap 置信度不低于 0.75 的至多两个互不相交末端 cherry 当作硬信息。它们被收缩为伪末端, 并以 singleton 形式固定; 展开回原终端后, 该 singleton 正是 $z_C z_C^T$ 。完整 RX75-RNJ 的其余 clade 不直接作为已知线路, 而只构成 MILP 的有限候选域。因而, RNJ 错误候选仍可被 L_1 拟合与独立验证拒绝。

7.2 对角线预处理与候选域精确性

当前四个算例的观测终端均为物理叶节点，所以每个约化终端必有一个 leaf-edge singleton 原子。实现先固定全部 singleton：普通 singleton 只承担对角线叶边项；收缩伪末端的 singleton 对应可信 RNJ 末端块。MILP 只搜索剩余非平凡块，避免用组合搜索重复发现确定存在的对角原子。

若有限候选为 $\mathcal{C} = \{C_1, \dots, C_M\}$ ，实现增加二元选择变量 h_m ，

$$\sum_{m=1}^M h_m = 1, \quad z_i = \sum_{m:i \in C_m} h_m. \quad (30)$$

其余 $y_{ij} = z_i z_j$ 、有界连续乘积、laminar 与 L_1 约束保持不变。因此每次扩展仍是真正的 MILP，并在当前 family、给定系数上界及有限候选域内要求全局最优证书；它不声称覆盖全部 $2^n - 1$ 个支撑，也不把多步贪心路径声称为固定 K 的联合全局最优。

7.3 数据配置与四算例结果

训练与验证分别使用独立的 3×96 含噪 AC 数据；P/Q 相对噪声为 0.5%，电压相对噪声为 0.02%，预处理为 `daily_demean`，矩阵约束为 `ordered`，RNJ 使用 RX75 与 0.16 倍中位根深度容差。做 100 次 circular moving-block bootstrap，块长为 4。真拓扑不参与候选生成。

表 7: 3 场景 $\times$ 96 点含噪 AC 数据上的 RX75-RNJ 与有限候选域 L_1 -MILP

算例	完整 RNJ F1	仅 top-2 F1	约化 n	候选数	联合 F1	MILP/s
<code>paper15</code>	1.0000	0.5000	11	4	1.0000	94.92
<code>soumalas11</code>	1.0000	0.6667	7	2	1.0000	3.57
<code>flynn16</code>	1.0000	0.5000	11	4	1.0000	47.46
<code>pengwah18</code>	0.9333	0.4444	14	6	1.0000	142.82

联合结果的 precision 与 recall 也均为 1；MILP 时间不含 RNJ/bootstrap。

特别地，`pengwah18` 的完整 RNJ 具有额外 clade $\{311, 312\}$ ，F1 为 0.9333；该错误块仍位于 MILP 候选池中，但最终验证路径没有选择它，并恢复全部 7 个真实非平凡 clade，故联合 F1 为 1。这是候选选择带来的改进，不是利用真值人工删块。

8 严格可证明性边界与已知限制

1. 单步全局最优不等于全路径全局最优。通过本文严格证书检查的状态 0，只证明在当前已选支撑不被删除/替换的前提下，新增一个合法原子的有界 MILP 达到数值全局最优。多原子交换、回溯或一次联合选择 K 个集合仍可能改进结果。
2. 必须依赖有限且有效的系数上界。McCormick 线性化对给定 $\bar{r}, \bar{x}$ 是精确的；若真实所需权重超过上界，模型域本身已经截断。实现会检测近触界并扩界重求，但当达到扩界次数上限时不会把结果误称为无约束全局最优。
3. 求解器精确性是数值容差意义的。HiGHS 使用浮点可行性、整数性和最优性容差；`mip_rel_gap=0` 表示请求零相对 gap，不是符号有理运算。实现还检查绝对原始-对偶 gap 或报告的相对 gap；若两者都缺失，即使状态字段为 0 也不输出“精确支撑”。超时 incumbent 同样不会被主算法接受。

4. **Laminar 支撑恢复的是终端 clade, 不是隐藏节点编号。**相同下游终端集合的串联边不可分别识别, 只能估计聚合 r/x ; 缺失激励、共线 P/Q 或零权重也会导致非唯一表示。
5. **原子数不等于矩阵不同数值个数。**一个矩阵元素是所有包含该终端对的原子权重之和; K 个原子可以产生多个不同子集和。本实现直接控制的是 clade/公共路径原子数和 laminar 复杂度, 而不是显式限制 R_{ij} 只能取 K 个数值。
6. **共享支撑不强迫两个通道同时严格为正。**某个集合可有 $r_k = 0, x_k > 0$ 或相反。若要加入物理 R/X 比值区间或共同严格激活, 需要额外约束及可辨识性分析。
7. **验证截距与测试协议必须区分。**主路径选择与严格测试指标均沿用相应训练 LP 的截距, 不读取 held-out 目标来重估截距。`evaluate_ll_matrices(..., fixed_intercepts=None)` 保留了重新拟合 nuisance intercept 的显式诊断模式, 但这种结果必须标为 profiled diagnostic, 不能冒充完全外推性能。
8. **small6 无噪声恢复是实现一致性检查, 不是 AC 真实性证明。**该制度的数据由同一个平方电压线性模型生成, 因此检验的是算法能否恢复模型类内真值。对 AC 潮流、量测误差、反向潮流和大规模 case33 的有效性仍需独立实验。
9. **计算复杂度仍为组合型。**MILP 避免显式枚举 $2^n - 1$ 个集合, 却没有消除最坏情形指数复杂度。实测中, 四叶 case33 的 7 轮均快速取得证书; 32 终端 hybrid 即使把系数上界从 10 收紧到 2, 并给单轮 60 秒, 仍以状态 1、报告 gap 1.0 超时。因此 small6/四叶精确结果不能外推为完整 case33 每轮均可快速证明最优。

9 建议的后续扩展

在不改变本报告主实现的前提下, 可依次研究:

- 为路径加入回溯式“删一加一”或局部交换 MILP, 以减弱纯前向贪心依赖;
- 加入物理 r/x 比值上下界, 并在有/无该约束之间做可辨识性消融;
- 在 case33 上比较精确 MILP、梯度 MILP oracle 和有限候选字典三种搜索代价;
- 对 AC 生成数据分别报告矩阵误差、非平凡 clade 缺失/额外/交叉替代及末端挂接错误;
- 通过 bootstrap 记录每个 clade 的选择频率, 而不把一次贪心路径当作拓扑置信度。

A 核心估计器完整源码

下列清单在编译时直接读取最终 Python 文件，因而数学报告与实际交付实现保持一致。

```

1  """Exact one-atom laminar L1 sensitivity regression with SciPy/HiGHS.
2
3  The estimator represents the reduced sensitivity matrices as
4
5      R = sum_k r_k z_k z_k.T,    X = sum_k x_k z_k z_k.T,
6
7  where every ``z_k`` is a binary terminal-clade indicator. Selected supports
8  are required to be laminar: two supports are nested or disjoint. At every
9  forward step, a MILP searches all admissible nonempty supports and refits all
10 previous atom weights and scenario-by-terminal intercepts with an L1 loss.
11
12 "Exact" means globally optimal for one bounded MILP extension when HiGHS
13 returns status 0 and the reported primal/dual bound or relative gap satisfies
14 this module's strict certificate check. The complete forward path is greedy
15 and is not claimed to be the globally optimal K-atom model.
16 """
17
18 from __future__ import annotations
19
20 from dataclasses import asdict, dataclass
21 from math import sqrt
22 from time import perf_counter
23 from typing import Hashable, Iterable, Sequence
24 import warnings
25
26 import numpy as np
27 from scipy.optimize import Bounds, LinearConstraint, milp
28 from scipy.sparse import coo_array
29
30
31 IndexSupport = tuple[int, ...]
32
33 _EXACT_MIP_REL_GAP_TOLERANCE = 1e-10
34 _EXACT_MIP_ABS_GAP_TOLERANCE = 1e-10
35 _SUPPORT_INTEGRAL_TOLERANCE = 1e-6
36 _OBJECTIVE_REPRODUCTION_REL_TOLERANCE = 1e-7
37
38
39 @dataclass(frozen=True)
40 class SolverDiagnostics:
41     """Auditable information returned by one HiGHS LP/MILP solve."""
42
43     status: int
44     success: bool
45     message: str
46     objective: float | None
47     dual_bound: float | None
48     mip_gap: float | None
49     node_count: int | None
50     runtime_seconds: float
51     variable_count: int
52     binary_variable_count: int
53     constraint_count: int
54 
```

```

55     def to_dict(self) -> dict:
56         return asdict(self)
57
58
59 @dataclass(frozen=True)
60 class FixedSupportSolution:
61     """L1 optimum for a fixed ordered list of supports."""
62
63     supports: tuple[IndexSupport, ...]
64     r_values: np.ndarray
65     x_values: np.ndarray
66     intercepts: np.ndarray
67     objective: float
68     diagnostics: SolverDiagnostics
69
70
71 @dataclass(frozen=True)
72 class ExtensionSolution:
73     """Exact best admissible one-support extension of a fixed family."""
74
75     support: IndexSupport | None
76     r_values: np.ndarray
77     x_values: np.ndarray
78     intercepts: np.ndarray
79     objective: float | None
80     diagnostics: SolverDiagnostics
81
82
83 @dataclass(frozen=True)
84 class LaminarL1PathPoint:
85     """One fully corrected model on the forward path."""
86
87     iteration: int
88     support_indices: tuple[IndexSupport, ...]
89     support_labels: tuple[tuple[Hashable, ...], ...]
90     r_values: np.ndarray
91     x_values: np.ndarray
92     intercepts: np.ndarray
93     r_matrix: np.ndarray
94     x_matrix: np.ndarray
95     train_mae: float
96     validation_mae: float | None
97     validation_se: float | None
98     accepted_gain: float | None
99     solver: SolverDiagnostics
100
101
102 @dataclass(frozen=True)
103 class LaminarL1Result:
104     """Selected model and the complete auditable forward path."""
105
106     terminal_labels: tuple[Hashable, ...]
107     support_indices: tuple[IndexSupport, ...]
108     support_labels: tuple[tuple[Hashable, ...], ...]
109     r_values: np.ndarray
110     x_values: np.ndarray
111     intercepts: np.ndarray
112     r_matrix: np.ndarray

```

```

113     x_matrix: np.ndarray
114     train_mae: float
115     validation_mae: float | None
116     selected_path_index: int
117     path: tuple[LaminarL1PathPoint, ...]
118     attempted_extensions: tuple[ExtensionSolution, ...]
119     stop_reason: str
120     r_upper_bound: float
121     x_upper_bound: float
122     bound_expansions: int
123
124     def summary(self) -> dict:
125         return {
126             "terminal_labels": list(self.terminal_labels),
127             "support_labels": [list(item) for item in self.support_labels],
128             "r_values": self.r_values.tolist(),
129             "x_values": self.x_values.tolist(),
130             "intercepts": self.intercepts.tolist(),
131             "train_mae": self.train_mae,
132             "validation_mae": self.validation_mae,
133             "selected_path_index": self.selected_path_index,
134             "path_length": len(self.path),
135             "stop_reason": self.stop_reason,
136             "r_upper_bound": self.r_upper_bound,
137             "x_upper_bound": self.x_upper_bound,
138             "bound_expansions": self.bound_expansions,
139         }
140
141
142 @dataclass(frozen=True)
143 class _PreparedScenarios:
144     labels: tuple[Hashable, ...]
145     names: tuple[str, ...]
146     p: tuple[np.ndarray, ...]
147     q: tuple[np.ndarray, ...]
148     target: tuple[np.ndarray, ...]
149
150     @property
151     def n(self) -> int:
152         return len(self.labels)
153
154     @property
155     def scenario_count(self) -> int:
156         return len(self.p)
157
158     @property
159     def observation_count(self) -> int:
160         return sum(block.shape[0] * self.n for block in self.p)
161
162
163 def _require_positive_finite(name: str, value: float) -> float:
164     result = float(value)
165     if not np.isfinite(result) or result <= 0.0:
166         raise ValueError(f"{name} must be positive and finite")
167     return result
168
169
170 def _require_nonnegative_finite(name: str, value: float) -> float:

```

```

171     result = float(value)
172     if not np.isfinite(result) or result < 0.0:
173         raise ValueError(f"{name} must be nonnegative and finite")
174     return result
175
176
177 def _validate_solver_parameters(
178     *,
179     r_upper_bound: float,
180     x_upper_bound: float,
181     time_limit: float | None,
182     mip_rel_gap: float | None = None,
183 ) -> tuple[float, float, float | None, float | None]:
184     r_bound = _require_positive_finite("r_upper_bound", r_upper_bound)
185     x_bound = _require_positive_finite("x_upper_bound", x_upper_bound)
186     checked_time_limit = (
187         None
188         if time_limit is None
189         else _require_positive_finite("time_limit", time_limit)
190     )
191     checked_gap = (
192         None
193         if mip_rel_gap is None
194         else _require_nonnegative_finite("mip_rel_gap", mip_rel_gap)
195     )
196     return r_bound, x_bound, checked_time_limit, checked_gap
197
198
199 def solver_diagnostics_prove_optimality(
200     diagnostics: SolverDiagnostics,
201     *,
202     relative_gap_tolerance: float = _EXACT_MIP_REL_GAP_TOLERANCE,
203     absolute_gap_tolerance: float = _EXACT_MIP_ABS_GAP_TOLERANCE,
204 ) -> bool:
205     """Return whether HiGHS certified the requested numerical optimum."""
206
207     if diagnostics.status != 0 or not diagnostics.success:
208         return False
209     if diagnostics.binary_variable_count == 0:
210         return True
211     if diagnostics.objective is not None and diagnostics.dual_bound is not None:
212         if np.isfinite(diagnostics.objective) and np.isfinite(diagnostics.dual_bound):
213             absolute_gap = abs(
214                 float(diagnostics.objective) - float(diagnostics.dual_bound)
215             )
216             if absolute_gap <= absolute_gap_tolerance:
217                 return True
218
219     # A status-zero MIP result without either a finite primal/dual certificate
220     # or a reported relative gap is not enough for this module's deliberately
221     # strict "exact extension" claim. Older SciPy/HiGHS combinations may omit
222     # these attributes; callers then receive the incumbent objective only, not
223     # a support that is labelled exact.
224     if diagnostics.mip_gap is None:
225         return False
226     return bool(
227         np.isfinite(diagnostics.mip_gap)
228         and diagnostics.mip_gap >= 0.0
229         and diagnostics.mip_gap <= relative_gap_tolerance

```

```

229     )
230
231
232 def _dual_bound_certifies_no_gain(
233     current_objective: float,
234     diagnostics: SolverDiagnostics,
235     gain_tolerance: float,
236 ) -> bool:
237     """Use a minimization dual bound to rule out a material extension gain."""
238
239     dual_bound = diagnostics.dual_bound
240     return bool(
241         dual_bound is not None
242         and np.isfinite(dual_bound)
243         and current_objective - float(dual_bound) <= gain_tolerance
244     )
245
246
247 def _objectives_agree(left: float, right: float) -> bool:
248     scale = max(1.0, abs(float(left)), abs(float(right)))
249     return bool(
250         abs(float(left) - float(right))
251         <= _OBJECTIVE_REPRODUCTION_REL_TOLERANCE * scale
252     )
253
254
255 class _VariableBuilder:
256     """Create a flat MILP vector while retaining named slices."""
257
258     def __init__(self) -> None:
259         self.objective: list[float] = []
260         self.lower: list[float] = []
261         self.upper: list[float] = []
262         self.integrality: list[int] = []
263         self.slices: dict[str, slice] = {}
264
265     def add(
266         self,
267         name: str,
268         size: int,
269         *,
270         lower: float | np.ndarray = 0.0,
271         upper: float | np.ndarray = np.inf,
272         objective: float | np.ndarray = 0.0,
273         integral: bool = False,
274     ) -> slice:
275         if name in self.slices:
276             raise ValueError(f"duplicate variable block: {name}")
277         start = len(self.objective)
278         stop = start + int(size)
279         block = slice(start, stop)
280         self.slices[name] = block
281         self.lower.extend(np.broadcast_to(lower, (size,)).astype(float).tolist())
282         self.upper.extend(np.broadcast_to(upper, (size,)).astype(float).tolist())
283         self.objective.extend(np.broadcast_to(objective, (size,)).astype(float).tolist())
284         self.integrality.extend([1 if integral else 0] * size)
285         return block
286

```

```

287     @property
288     def size(self) -> int:
289         return len(self.objective)
290
291
292 class _ConstraintBuilder:
293     """Sparse ranged-row builder for scipy.optimize.LinearConstraint."""
294
295     def __init__(self) -> None:
296         self.row_index: list[int] = []
297         self.col_index: list[int] = []
298         self.data: list[float] = []
299         self.lower: list[float] = []
300         self.upper: list[float] = []
301
302     def add(
303         self,
304         entries: dict[int, float],
305         *,
306         lower: float = -np.inf,
307         upper: float = np.inf,
308     ) -> None:
309         row = len(self.lower)
310         for col, value in entries.items():
311             if value != 0.0:
312                 self.row_index.append(row)
313                 self.col_index.append(int(col))
314                 self.data.append(float(value))
315         self.lower.append(float(lower))
316         self.upper.append(float(upper))
317
318     def build(self, variable_count: int) -> LinearConstraint:
319         matrix = coo_array(
320             (self.data, (self.row_index, self.col_index)),
321             shape=(len(self.lower), variable_count),
322             dtype=float,
323         ).tocsc()
324         return LinearConstraint(
325             matrix,
326             np.asarray(self.lower, dtype=float),
327             np.asarray(self.upper, dtype=float),
328         )
329
330     @property
331     def size(self) -> int:
332         return len(self.lower)
333
334
335 def _matrix_from_scenario(value, labels: tuple[Hashable, ...], key: str) -> np.ndarray:
336     if hasattr(value, "loc") and hasattr(value, "columns"):
337         try:
338             array = value.loc[:, list(labels)].to_numpy(dtype=float)
339         except KeyError as exc:
340             raise ValueError(f"{key} columns do not match the first scenario") from exc
341     else:
342         array = np.asarray(value, dtype=float)
343     if array.ndim != 2:
344         raise ValueError(f"{key} must be a two-dimensional table")

```

```

345     return array
346
347
348 def _prepare_scenarios(scenarios: Sequence[dict]) -> _PreparedScenarios:
349     if not scenarios:
350         raise ValueError("at least one scenario is required")
351     first_p = scenarios[0]["P_terminal"]
352     if hasattr(first_p, "columns"):
353         labels = tuple(first_p.columns.tolist())
354     else:
355         first_array = np.asarray(first_p)
356         if first_array.ndim != 2:
357             raise ValueError("P_terminal must be two-dimensional")
358         labels = tuple(range(first_array.shape[1]))
359     if not labels:
360         raise ValueError("at least one terminal is required")
361
362     p_blocks: list[np.ndarray] = []
363     q_blocks: list[np.ndarray] = []
364     target_blocks: list[np.ndarray] = []
365     names: list[str] = []
366     for scenario_index, scenario in enumerate(scenarios):
367         p = _matrix_from_scenario(scenario["P_terminal"], labels, "P_terminal")
368         q = _matrix_from_scenario(scenario["Q_terminal"], labels, "Q_terminal")
369         target = _matrix_from_scenario(scenario["drop_target"], labels, "drop_target")
370         if p.shape != q.shape or p.shape != target.shape:
371             raise ValueError("P_terminal, Q_terminal, and drop_target must have equal shape")
372         if p.shape[0] == 0:
373             raise ValueError("each scenario must contain at least one time sample")
374         if p.shape[1] != len(labels):
375             raise ValueError("all scenarios must use the same terminal count")
376         if not (np.isfinite(p).all() and np.isfinite(q).all() and np.isfinite(target).all()):
377             raise ValueError("scenario data must contain only finite values")
378         p_blocks.append(p)
379         q_blocks.append(q)
380         target_blocks.append(target)
381         names.append(str(scenario.get("name", f"scenario_{scenario_index}")))
382     return _PreparedScenarios(
383         labels=labels,
384         names=tuple(names),
385         p=tuple(p_blocks),
386         q=tuple(q_blocks),
387         target=tuple(target_blocks),
388     )
389
390
391 def normalize_supports(
392     supports: Iterable[Iterable[int]], n: int
393 ) -> tuple[IndexSupport, ...]:
394     """Validate, sort, and canonicalize positional supports."""
395
396     normalized: list[IndexSupport] = []
397     seen: set[IndexSupport] = set()
398     for support in supports:
399         item = tuple(sorted({int(index) for index in support}))
400         if not item:
401             raise ValueError("supports must be nonempty")
402         if item[0] < 0 or item[-1] >= n:

```

```

403         raise ValueError("support index is outside the terminal range")
404     if item in seen:
405         raise ValueError("duplicate support")
406     seen.add(item)
407     normalized.append(item)
408     if not is_laminar_family(normalized):
409         raise ValueError("supports must form a laminar family")
410     return tuple(normalized)
411
412
413 def normalize_support_pool(
414     supports: Iterable[Iterable[int]], n: int
415 ) -> tuple[IndexSupport, ...]:
416     """Canonicalize a finite candidate pool without requiring mutual laminarity.
417
418     Candidate supports may cross each other because the MILP chooses only one
419     extension at a time. Each chosen support must still be laminar with the
420     already accepted family.
421     """
422
423     normalized: list[IndexSupport] = []
424     seen: set[IndexSupport] = set()
425     for support in supports:
426         item = tuple(sorted({int(index) for index in support}))
427         if not item:
428             raise ValueError("candidate supports must be nonempty")
429         if item[0] < 0 or item[-1] >= n:
430             raise ValueError("candidate support index is outside the terminal range")
431         if item not in seen:
432             seen.add(item)
433             normalized.append(item)
434     return tuple(normalized)
435
436
437 def is_laminar_family(supports: Iterable[Iterable[int]]) -> bool:
438     """Return whether every support pair is nested or disjoint."""
439
440     family = [set(item) for item in supports]
441     for left_index, left in enumerate(family):
442         for right_index, right in enumerate(family[left_index + 1 :]):
443             overlap = left & right
444             if overlap and not (left <= right or right <= left):
445                 return False
446     return True
447
448
449 def is_admissible_extension(
450     support: Iterable[int], existing_supports: Iterable[Iterable[int]], n: int
451 ) -> bool:
452     """Check nonemptiness, uniqueness, range, and laminar compatibility."""
453
454     candidate = tuple(sorted({int(index) for index in support}))
455     if not candidate or candidate[0] < 0 or candidate[-1] >= n:
456         return False
457     family = [tuple(sorted(set(item))) for item in existing_supports]
458     if candidate in family:
459         return False
460     return is_laminar_family([*family, candidate])

```

```

461
462
463 def _round_and_validate_support(
464     z_values: np.ndarray,
465     existing_supports: tuple[IndexSupport, ...],
466     n: int,
467 ) -> IndexSupport:
468     values = np.asarray(z_values, dtype=float)
469     if values.shape != (n,) or not np.isfinite(values).all():
470         raise RuntimeError("MILP returned an invalid support vector")
471     rounded = np.rint(values)
472     if float(np.max(np.abs(values - rounded))) > _SUPPORT_INTEGRAL_TOLERANCE:
473         raise RuntimeError("MILP support vector violates the integrality tolerance")
474     support = tuple(int(index) for index in np.flatnonzero(rounded > 0.5))
475     if not is_admissible_extension(support, existing_supports, n):
476         raise RuntimeError("MILP returned a nonempty/unique/laminar-infeasible support")
477     return support
478
479
480 def build_matrices_from_atoms(
481     n: int,
482     supports: Sequence[Sequence[int]],
483     r_values: Sequence[float],
484     x_values: Sequence[float],
485 ) -> tuple[np.ndarray, np.ndarray]:
486     """Construct symmetric R/X matrices from nonnegative support atoms.
487
488     This low-level constructor intentionally permits crossing supports so it
489     can also build adversarial matrices for tests. The estimators themselves
490     call :func:`normalize_supports` and therefore require a laminar family.
491     """
492
493     if not isinstance(n, (int, np.integer)) or int(n) < 1:
494         raise ValueError("n must be a positive integer")
495     n = int(n)
496     support_values = tuple(tuple(support) for support in supports)
497     r_array = np.asarray(tuple(r_values), dtype=float)
498     x_array = np.asarray(tuple(x_values), dtype=float)
499     if r_array.ndim != 1 or x_array.ndim != 1:
500         raise ValueError("atom coefficients must be one-dimensional")
501     if len(support_values) != len(r_array) or len(support_values) != len(x_array):
502         raise ValueError("support and coefficient counts must agree")
503     if not (np.isfinite(r_array).all() and np.isfinite(x_array).all()):
504         raise ValueError("atom coefficients must be finite")
505     if np.any(r_array < 0.0) or np.any(x_array < 0.0):
506         raise ValueError("atom coefficients must be nonnegative")
507     r_matrix = np.zeros((n, n), dtype=float)
508     x_matrix = np.zeros((n, n), dtype=float)
509     for atom_index, (support, r_value, x_value) in enumerate(
510         zip(support_values, r_array, x_array, strict=True)
511     ):
512         index_tuple = tuple(int(value) for value in support)
513         if not index_tuple:
514             raise ValueError(f"support {atom_index} must be nonempty")
515         if len(set(index_tuple)) != len(index_tuple):
516             raise ValueError(f"support {atom_index} contains duplicate indices")
517         if min(index_tuple) < 0 or max(index_tuple) >= n:
518             raise ValueError(f"support {atom_index} is outside the terminal range")

```

```

519     index = np.asarray(index_tuple, dtype=int)
520     r_matrix[np.ix_(index, index)] += float(r_value)
521     x_matrix[np.ix_(index, index)] += float(x_value)
522     return r_matrix, x_matrix
523
524
525 def _fixed_atom_features(
526     prepared: _PreparedScenarios, supports: tuple[IndexSupport, ...]
527 ) -> tuple[np.ndarray, np.ndarray]:
528     if not supports:
529         empty = np.empty((prepared.observation_count, 0), dtype=float)
530         return empty, empty.copy()
531     p_parts: list[np.ndarray] = []
532     q_parts: list[np.ndarray] = []
533     for p_block, q_block in zip(prepared.p, prepared.q, strict=True):
534         block_p: list[np.ndarray] = []
535         block_q: list[np.ndarray] = []
536         for support in supports:
537             indicator = np.zeros(prepared.n, dtype=float)
538             indicator[list(support)] = 1.0
539             block_p.append(((p_block @ indicator)[: , None] * indicator[None, :]).reshape(-1))
540             block_q.append(((q_block @ indicator)[: , None] * indicator[None, :]).reshape(-1))
541         p_parts.append(np.column_stack(block_p))
542         q_parts.append(np.column_stack(block_q))
543     return np.vstack(p_parts), np.vstack(q_parts)
544
545
546 def _target_and_scenario_output(prepared: _PreparedScenarios) -> tuple[np.ndarray, np.ndarray]:
547     target = np.concatenate([block.reshape(-1) for block in prepared.target])
548     scenario_output: list[np.ndarray] = []
549     for scenario_index, block in enumerate(prepared.target):
550         time_count = block.shape[0]
551         output = np.tile(np.arange(prepared.n), time_count)
552         scenario_output.append(scenario_index * prepared.n + output)
553     return target, np.concatenate(scenario_output)
554
555
556 def _diagnostics(result, elapsed: float, variables: _VariableBuilder, rows: _ConstraintBuilder) ->
557     SolverDiagnostics:
558     objective = getattr(result, "fun", None)
559     return SolverDiagnostics(
560         status=int(result.status),
561         success=bool(result.success),
562         message=str(result.message),
563         objective=None if objective is None else float(objective),
564         dual_bound=(
565             None
566             if getattr(result, "mip_dual_bound", None) is None
567             else float(result.mip_dual_bound)
568         ),
569         mip_gap=(None if getattr(result, "mip_gap", None) is None else float(result.mip_gap)),
570         node_count=(
571             None
572             if getattr(result, "mip_node_count", None) is None
573             else int(result.mip_node_count)
574         ),
575         runtime_seconds=float(elapsed),
576         variable_count=variables.size,

```

```

576         binary_variable_count=int(np.count_nonzero(variables.integrality)),
577         constraint_count=rows.size,
578     )
579
580
581 def _run_milp(
582     variables: _VariableBuilder,
583     rows: _ConstraintBuilder,
584     *,
585     time_limit: float | None,
586     mip_rel_gap: float,
587     presolve: bool,
588     disp: bool,
589 ):
590     options: dict[str, float | bool | int] = {
591         "mip_rel_gap": float(mip_rel_gap),
592         # SciPy forwards this native HiGHS option even though some SciPy
593         # releases do not list it in ``milp``'s public option schema.
594         "mip_abs_gap": _EXACT_MIP_ABS_GAP_TOLERANCE,
595         "presolve": bool(presolve),
596         "disp": bool(disp),
597     }
598     if time_limit is not None:
599         options["time_limit"] = float(time_limit)
600     started = perf_counter()
601     with warnings.catch_warnings():
602         warnings.filterwarnings(
603             "ignore",
604             message=r"Unrecognized options detected: .*mip_abs_gap.*",
605             category=RuntimeWarning,
606         )
607     result = milp(
608         c=np.asarray(variables.objective, dtype=float),
609         integrality=np.asarray(variables.integrality, dtype=np.uint8),
610         bounds=Bounds(
611             np.asarray(variables.lower, dtype=float),
612             np.asarray(variables.upper, dtype=float),
613         ),
614         constraints=rows.build(variables.size),
615         options=options,
616     )
617     elapsed = perf_counter() - started
618     return result, _diagnostics(result, elapsed, variables, rows)
619
620
621 def _add_absolute_residual_constraints(
622     prepared: _PreparedScenarios,
623     variables: _VariableBuilder,
624     rows: _ConstraintBuilder,
625     *,
626     r_block: slice,
627     x_block: slice,
628     intercept_block: slice,
629     absolute_block: slice,
630     supports: tuple[IndexSupport, ...],
631     new_u_r_block: slice | None = None,
632     new_u_x_block: slice | None = None,
633     pair_lookup: dict[tuple[int, int], int] | None = None,

```

```

634 ) -> None:
635     target, scenario_output = _target_and_scenario_output(prepared)
636     p_features, q_features = _fixed_atom_features(prepared, supports)
637     observation = 0
638     for scenario_index, (p_block, q_block) in enumerate(
639         zip(prepared.p, prepared.q, strict=True)
640     ):
641         for time_index in range(p_block.shape[0]):
642             for output_index in range(prepared.n):
643                 prediction: dict[int, float] = {
644                     intercept_block.start + int(scenario_output[observation]): 1.0
645                 }
646                 for atom_index in range(len(supports)):
647                     p_value = p_features[observation, atom_index]
648                     q_value = q_features[observation, atom_index]
649                     if p_value != 0.0:
650                         prediction[r_block.start + atom_index] = float(p_value)
651                     if q_value != 0.0:
652                         prediction[x_block.start + atom_index] = float(q_value)
653                 if new_u_r_block is not None and new_u_x_block is not None:
654                     if pair_lookup is None:
655                         raise AssertionError("pair lookup is required for a new atom")
656                     for input_index in range(prepared.n):
657                         pair = (min(output_index, input_index), max(output_index, input_index))
658                         pair_index = pair_lookup[pair]
659                         p_value = p_block[time_index, input_index]
660                         q_value = q_block[time_index, input_index]
661                         if p_value != 0.0:
662                             prediction[new_u_r_block.start + pair_index] = float(p_value)
663                         if q_value != 0.0:
664                             prediction[new_u_x_block.start + pair_index] = float(q_value)
665
666                 positive = dict(prediction)
667                 positive[absolute_block.start + observation] = 1.0
668                 rows.add(positive, lower=float(target[observation]))
669
670                 negative = dict(prediction)
671                 negative[absolute_block.start + observation] = -1.0
672                 rows.add(negative, upper=float(target[observation]))
673                 observation += 1
674
675
676 def _extract_intercepts(
677     vector: np.ndarray, block: slice, prepared: _PreparedScenarios
678 ) -> np.ndarray:
679     return np.asarray(vector[block], dtype=float).reshape(prepared.scenario_count, prepared.n)
680
681
682 def _solve_fixed_prepared(
683     prepared: _PreparedScenarios,
684     supports: tuple[IndexSupport, ...],
685     *,
686     r_upper_bound: float,
687     x_upper_bound: float,
688     time_limit: float | None,
689     presolve: bool,
690     disp: bool,
691 ) -> FixedSupportSolution:

```

```

692     atom_count = len(supports)
693     variables = _VariableBuilder()
694     r_block = variables.add("r", atom_count, upper=r_upper_bound)
695     x_block = variables.add("x", atom_count, upper=x_upper_bound)
696     intercept_block = variables.add(
697         "intercepts", prepared.scenario_count * prepared.n, lower=-np.inf, upper=np.inf
698     )
699     absolute_block = variables.add(
700         "absolute_residual",
701         prepared.observation_count,
702         objective=1.0 / prepared.observation_count,
703     )
704     rows = _ConstraintBuilder()
705     _add_absolute_residual_constraints(
706         prepared,
707         variables,
708         rows,
709         r_block=r_block,
710         x_block=x_block,
711         intercept_block=intercept_block,
712         absolute_block=absolute_block,
713         supports=supports,
714     )
715     result, diagnostics = _run_milp(
716         variables,
717         rows,
718         time_limit=time_limit,
719         mip_rel_gap=0.0,
720         presolve=presolve,
721         disp=disp,
722     )
723     if (
724         result.x is None
725         or result.fun is None
726         or not solver_diagnostics_prove_optimality(diagnostics)
727     ):
728         raise RuntimeError(
729             "fixed-support L1 LP was not proven optimal: "
730             f"status={diagnostics.status}, gap={diagnostics.mip_gap}, "
731             f"message={diagnostics.message}"
732         )
733     return FixedSupportSolution(
734         supports=supports,
735         r_values=np.asarray(result.x[r_block], dtype=float),
736         x_values=np.asarray(result.x[x_block], dtype=float),
737         intercepts=_extract_intercepts(result.x, intercept_block, prepared),
738         objective=float(result.fun),
739         diagnostics=diagnostics,
740     )
741
742
743 def solve_fixed_support_l1(
744     scenarios: Sequence[dict],
745     supports: Iterable[Iterable[int]],
746     *,
747     r_upper_bound: float,
748     x_upper_bound: float,
749     time_limit: float | None = None,

```

```

750     presolve: bool = True,
751     disp: bool = False,
752 ) -> FixedSupportSolution:
753     """Solve the fully corrective L1 regression for fixed positional supports."""
754
755     r_bound, x_bound, checked_time_limit, _ = _validate_solver_parameters(
756         r_upper_bound=r_upper_bound,
757         x_upper_bound=x_upper_bound,
758         time_limit=time_limit,
759     )
760     prepared = _prepare_scenarios(scenarios)
761     family = normalize_supports(supports, prepared.n)
762     return _solve_fixed_prepared(
763         prepared,
764         family,
765         r_upper_bound=r_bound,
766         x_upper_bound=x_bound,
767         time_limit=checked_time_limit,
768         presolve=presolve,
769         disp=disp,
770     )
771
772
773 def _upper_pairs(n: int) -> tuple[tuple[tuple[int, int], ...], dict[tuple[int, int], int]]:
774     pairs = tuple((i, j) for i in range(n) for j in range(i, n))
775     return pairs, {pair: index for index, pair in enumerate(pairs)}
776
777
778 def _solve_extension_prepared(
779     prepared: _PreparedScenarios,
780     supports: tuple[IndexSupport, ...],
781     *,
782     candidate_supports: tuple[IndexSupport, ...] | None,
783     r_upper_bound: float,
784     x_upper_bound: float,
785     time_limit: float | None,
786     mip_rel_gap: float,
787     presolve: bool,
788     disp: bool,
789 ) -> ExtensionSolution:
790     atom_count = len(supports)
791     admissible_candidates = (
792         None
793         if candidate_supports is None
794         else tuple(
795             support
796             for support in candidate_supports
797             if is_admissible_extension(support, supports, prepared.n)
798         )
799     )
800     pairs, pair_lookup = _upper_pairs(prepared.n)
801     pair_count = len(pairs)
802     variables = _VariableBuilder()
803     old_r = variables.add("old_r", atom_count, upper=r_upper_bound)
804     old_x = variables.add("old_x", atom_count, upper=x_upper_bound)
805     intercepts = variables.add(
806         "intercepts", prepared.scenario_count * prepared.n, lower=-np.inf, upper=np.inf
807     )

```

```

808     absolute = variables.add(
809         "absolute_residual",
810         prepared.observation_count,
811         objective=1.0 / prepared.observation_count,
812     )
813     new_r = variables.add("new_r", 1, upper=r_upper_bound)
814     new_x = variables.add("new_x", 1, upper=x_upper_bound)
815     z_block = variables.add("z", prepared.n, upper=1.0, integral=True)
816     candidate_selector = (
817         None
818         if admissible_candidates is None
819         else variables.add(
820             "candidate_selector",
821             len(admissible_candidates),
822             upper=1.0,
823             integral=True,
824         )
825     )
826     y_block = variables.add("y", pair_count, upper=1.0)
827     u_r_block = variables.add("u_r", pair_count, upper=r_upper_bound)
828     u_x_block = variables.add("u_x", pair_count, upper=x_upper_bound)
829     relation_block = variables.add("relations", 3 * atom_count, upper=1.0, integral=True)
830
831     rows = _ConstraintBuilder()
832     _add_absolute_residual_constraints(
833         prepared,
834         variables,
835         rows,
836         r_block=old_r,
837         x_block=old_x,
838         intercept_block=intercepts,
839         absolute_block=absolute,
840         supports=supports,
841         new_u_r_block=u_r_block,
842         new_u_x_block=u_x_block,
843         pair_lookup=pair_lookup,
844     )
845
846     # Nonempty candidate support.
847     rows.add({z_block.start + i: 1.0 for i in range(prepared.n)}, lower=1.0)
848
849     # Optional finite-domain encoding: choose exactly one admissible proposal
850     # and make z equal its incidence vector. This retains a genuine MILP and
851     # a global optimum certificate relative to the supplied candidate pool.
852     if candidate_selector is not None:
853         rows.add(
854             {
855                 candidate_selector.start + index: 1.0
856                 for index in range(len(admissible_candidates))
857             },
858             lower=1.0,
859             upper=1.0,
860         )
861         for terminal in range(prepared.n):
862             row = {z_block.start + terminal: 1.0}
863             for index, support in enumerate(admissible_candidates):
864                 if terminal in support:
865                     row[candidate_selector.start + index] = -1.0

```

```

866         rows.add(row, lower=0.0, upper=0.0)
867
868     #  $y_{ij} = z_i z_j$ .  $y$  can remain continuous because binary  $z$  fixes it.
869     for pair_index, (i, j) in enumerate(pairs):
870         y_index = y_block.start + pair_index
871         z_i = z_block.start + i
872         z_j = z_block.start + j
873         if i == j:
874             rows.add({y_index: 1.0, z_i: -1.0}, lower=0.0, upper=0.0)
875         else:
876             rows.add({y_index: 1.0, z_i: -1.0}, upper=0.0)
877             rows.add({y_index: 1.0, z_j: -1.0}, upper=0.0)
878             rows.add({y_index: 1.0, z_i: -1.0, z_j: -1.0}, lower=-1.0)
879
880     # Exact bounded products  $u_R = \text{new}_R * y$  and  $u_X = \text{new}_X * y$ .
881     u_r = u_r_block.start + pair_index
882     rows.add({u_r: 1.0, y_index: -r_upper_bound}, upper=0.0)
883     rows.add({u_r: 1.0, new_r.start: -1.0}, upper=0.0)
884     rows.add(
885         {u_r: 1.0, new_r.start: -1.0, y_index: -r_upper_bound},
886         lower=-r_upper_bound,
887     )
888     u_x = u_x_block.start + pair_index
889     rows.add({u_x: 1.0, y_index: -x_upper_bound}, upper=0.0)
890     rows.add({u_x: 1.0, new_x.start: -1.0}, upper=0.0)
891     rows.add(
892         {u_x: 1.0, new_x.start: -1.0, y_index: -x_upper_bound},
893         lower=-x_upper_bound,
894     )
895
896     # New support must be nested with or disjoint from every old support.
897     for atom_index, support in enumerate(supports):
898         support_set = set(support)
899         subset_relation = relation_block.start + 3 * atom_index
900         superset_relation = subset_relation + 1
901         disjoint_relation = subset_relation + 2
902         rows.add(
903             {
904                 subset_relation: 1.0,
905                 superset_relation: 1.0,
906                 disjoint_relation: 1.0,
907             },
908             lower=1.0,
909             upper=1.0,
910         )
911         for terminal in range(prepared.n):
912             z_index = z_block.start + terminal
913             if terminal not in support_set:
914                 rows.add({z_index: 1.0, subset_relation: 1.0}, upper=1.0)
915             else:
916                 rows.add({superset_relation: 1.0, z_index: -1.0}, upper=0.0)
917                 rows.add({z_index: 1.0, disjoint_relation: 1.0}, upper=1.0)
918
919     # Hamming distance from the old support is at least one.
920     duplicate_row: dict[int, float] = {}
921     for terminal in range(prepared.n):
922         duplicate_row[z_block.start + terminal] = (
923             -1.0 if terminal in support_set else 1.0

```

```

924     )
925     rows.add(duplicate_row, lower=1.0 - len(support_set))
926
927     result, diagnostics = _run_milp(
928         variables,
929         rows,
930         time_limit=time_limit,
931         mip_rel_gap=mip_rel_gap,
932         presolve=presolve,
933         disp=disp,
934     )
935     if (
936         result.x is None
937         or result.fun is None
938         or not np.isfinite(result.fun)
939         or not np.isfinite(result.x).all()
940     ):
941         return ExtensionSolution(
942             support=None,
943             r_values=np.empty(0),
944             x_values=np.empty(0),
945             intercepts=np.empty((prepared.scenario_count, prepared.n)),
946             objective=None,
947             diagnostics=diagnostics,
948         )
949     if not solver_diagnostics_prove_optimality(diagnostics):
950         # A time-limited incumbent is deliberately not exposed as an exact
951         # support. This also covers a nominal status-zero result for which the
952         # requested strict gap cannot be reconstructed from diagnostics. Its
953         # dual bound remains available for a certified no-gain decision in the
954         # outer algorithm.
955         return ExtensionSolution(
956             support=None,
957             r_values=np.empty(0),
958             x_values=np.empty(0),
959             intercepts=np.empty((prepared.scenario_count, prepared.n)),
960             objective=float(result.fun),
961             diagnostics=diagnostics,
962         )
963     support = _round_and_validate_support(
964         np.asarray(result.x[z_block], dtype=float), supports, prepared.n
965     )
966     combined_r = np.concatenate(
967         [np.asarray(result.x[old_r], dtype=float), [float(result.x[new_r.start])]]
968     )
969     combined_x = np.concatenate(
970         [np.asarray(result.x[old_x], dtype=float), [float(result.x[new_x.start])]]
971     )
972     return ExtensionSolution(
973         support=support,
974         r_values=combined_r,
975         x_values=combined_x,
976         intercepts=_extract_intercepts(result.x, intercepts, prepared),
977         objective=float(result.fun),
978         diagnostics=diagnostics,
979     )
980
981

```

```

982 def solve_best_laminar_extension_l1(
983     scenarios: Sequence[dict],
984     supports: Iterable[Iterable[int]],
985     *,
986     candidate_supports: Iterable[Iterable[int]] | None = None,
987     r_upper_bound: float,
988     x_upper_bound: float,
989     time_limit: float | None = None,
990     mip_rel_gap: float = 0.0,
991     presolve: bool = True,
992     disp: bool = False,
993 ) -> ExtensionSolution:
994     """Search for the best one-atom L1 extension.
995
996     With no candidate pool the search covers every admissible subset.
997     Otherwise it is exact over the supplied finite candidate pool.
998     """
999
1000     r_bound, x_bound, checked_time_limit, checked_gap = _validate_solver_parameters(
1001         r_upper_bound=r_upper_bound,
1002         x_upper_bound=x_upper_bound,
1003         time_limit=time_limit,
1004         mip_rel_gap=mip_rel_gap,
1005     )
1006     prepared = _prepare_scenarios(scenarios)
1007     family = normalize_supports(supports, prepared.n)
1008     candidate_pool = (
1009         None
1010         if candidate_supports is None
1011         else normalize_support_pool(candidate_supports, prepared.n)
1012     )
1013     return _solve_extension_prepared(
1014         prepared,
1015         family,
1016         candidate_supports=candidate_pool,
1017         r_upper_bound=r_bound,
1018         x_upper_bound=x_bound,
1019         time_limit=checked_time_limit,
1020         mip_rel_gap=float(checked_gap),
1021         presolve=presolve,
1022         disp=disp,
1023     )
1024
1025
1026 def estimate_atom_upper_bounds(
1027     scenarios: Sequence[dict], *, safety_factor: float = 2.0, floor: float = 1e-8
1028 ) -> tuple[float, float]:
1029     """Derive data-only atom bounds from the current dense OLS coefficient scale."""
1030
1031     if not np.isfinite(safety_factor) or safety_factor <= 1.0:
1032         raise ValueError("safety_factor must exceed one")
1033     floor = _require_positive_finite("floor", floor)
1034     prepared = _prepare_scenarios(scenarios)
1035     design_parts: list[np.ndarray] = []
1036     target_parts: list[np.ndarray] = []
1037     for scenario_index, (p_block, q_block, target_block) in enumerate(
1038         zip(prepared.p, prepared.q, prepared.target, strict=True)
1039     ):

```

```

1040     indicator = np.zeros((p_block.shape[0], prepared.scenario_count), dtype=float)
1041     indicator[:, scenario_index] = 1.0
1042     design_parts.append(np.hstack([p_block, q_block, indicator]))
1043     target_parts.append(target_block)
1044     design = np.vstack(design_parts)
1045     target = np.vstack(target_parts)
1046     coefficients, _, _, _ = np.linalg.lstsq(design, target, rcond=None)
1047     n = prepared.n
1048     r_raw = coefficients[n:, :].T
1049     x_raw = coefficients[n : 2 * n, :].T
1050     r_symmetric = np.maximum(0.0, 0.5 * (r_raw + r_raw.T))
1051     x_symmetric = np.maximum(0.0, 0.5 * (x_raw + x_raw.T))
1052     r_scale = max(float(np.max(r_symmetric)), float(floor))
1053     x_scale = max(float(np.max(x_symmetric)), float(floor))
1054     return safety_factor * r_scale, safety_factor * x_scale
1055
1056
1057 def evaluate_l1_matrices(
1058     scenarios: Sequence[dict],
1059     r_matrix: np.ndarray,
1060     x_matrix: np.ndarray,
1061     *,
1062     blocks_per_scenario: int = 4,
1063     fixed_intercepts: np.ndarray | None = None,
1064 ) -> tuple[float, float, np.ndarray]:
1065     """Evaluate MAE with fixed intercepts or profiled scenario/output medians.
1066
1067     Passing ``fixed_intercepts`` is required for genuine held-out prediction.
1068     The ``None`` mode is retained as an explicitly profiled nuisance-intercept
1069     diagnostic for callers that provide a separate calibration interpretation.
1070     """
1071
1072     prepared = _prepare_scenarios(scenarios)
1073     if not isinstance(blocks_per_scenario, (int, np.integer)) or blocks_per_scenario < 1:
1074         raise ValueError("blocks_per_scenario must be a positive integer")
1075     r_value = np.asarray(r_matrix, dtype=float)
1076     x_value = np.asarray(x_matrix, dtype=float)
1077     if r_value.shape != (prepared.n, prepared.n) or x_value.shape != r_value.shape:
1078         raise ValueError("R/X shape does not match validation terminals")
1079     intercept_values: np.ndarray | None = None
1080     if fixed_intercepts is not None:
1081         intercept_values = np.asarray(fixed_intercepts, dtype=float)
1082         expected_shape = (prepared.scenario_count, prepared.n)
1083         if intercept_values.shape != expected_shape:
1084             raise ValueError(
1085                 f"fixed_intercepts must have shape {expected_shape}"
1086             )
1087         if not np.isfinite(intercept_values).all():
1088             raise ValueError("fixed_intercepts must contain only finite values")
1089     absolute_parts: list[np.ndarray] = []
1090     block_means: list[float] = []
1091     for scenario_index, (p_block, q_block, target_block) in enumerate(
1092         zip(prepared.p, prepared.q, prepared.target, strict=True)
1093     ):
1094         raw_residual = target_block - p_block @ r_value.T - q_block @ x_value.T
1095         intercept = (
1096             np.median(raw_residual, axis=0, keepdims=True)
1097             if intercept_values is None

```

```

1098         else intercept_values[scenario_index][None, :]
1099     )
1100     absolute = np.abs(raw_residual - intercept)
1101     absolute_parts.append(absolute.reshape(-1))
1102     for time_indices in np.array_split(np.arange(len(absolute)), blocks_per_scenario):
1103         if time_indices.size:
1104             block_means.append(float(np.mean(absolute[time_indices, :])))
1105     all_absolute = np.concatenate(absolute_parts)
1106     mae = float(np.mean(all_absolute))
1107     block_array = np.asarray(block_means, dtype=float)
1108     standard_error = (
1109         float(np.std(block_array, ddof=1) / sqrt(len(block_array)))
1110         if len(block_array) > 1
1111         else 0.0
1112     )
1113     return mae, standard_error, block_array
1114
1115
1116 def _labels_for_supports(
1117     supports: tuple[IndexSupport, ...], labels: tuple[Hashable, ...]
1118 ) -> tuple[tuple[Hashable, ...], ...]:
1119     return tuple(tuple(labels[index] for index in support) for support in supports)
1120
1121
1122 def _make_path_point(
1123     *,
1124     iteration: int,
1125     prepared: _PreparedScenarios,
1126     solution: FixedSupportSolution,
1127     validation_scenarios: Sequence[dict] | None,
1128     validation_blocks: int,
1129     accepted_gain: float | None,
1130     solver: SolverDiagnostics,
1131 ) -> LaminarL1PathPoint:
1132     r_matrix, x_matrix = build_matrices_from_atoms(
1133         prepared.n,
1134         solution.supports,
1135         solution.r_values,
1136         solution.x_values,
1137     )
1138     validation_mae: float | None = None
1139     validation_se: float | None = None
1140     if validation_scenarios is not None:
1141         validation_mae, validation_se, _ = evaluate_l1_matrices(
1142             validation_scenarios,
1143             r_matrix,
1144             x_matrix,
1145             blocks_per_scenario=validation_blocks,
1146             fixed_intercepts=solution.intercepts,
1147         )
1148     return LaminarL1PathPoint(
1149         iteration=iteration,
1150         support_indices=solution.supports,
1151         support_labels=_labels_for_supports(solution.supports, prepared.labels),
1152         r_values=solution.r_values.copy(),
1153         x_values=solution.x_values.copy(),
1154         intercepts=solution.intercepts.copy(),
1155         r_matrix=r_matrix,

```

```

1156         x_matrix=x_matrix,
1157         train_mae=float(solution.objective),
1158         validation_mae=validation_mae,
1159         validation_se=validation_se,
1160         accepted_gain=accepted_gain,
1161         solver=solver,
1162     )
1163
1164
1165 def _touches_bound(values: np.ndarray, bound: float, fraction: float) -> bool:
1166     return bool(values.size and np.max(values) >= fraction * bound)
1167
1168
1169 def _choose_path_point(path: Sequence[LaminarL1PathPoint]) -> int:
1170     if not path or path[0].validation_mae is None:
1171         return len(path) - 1
1172     means = np.asarray([point.validation_mae for point in path], dtype=float)
1173     best = int(np.argmin(means))
1174     threshold = means[best] + float(path[best].validation_se or 0.0)
1175     eligible = np.flatnonzero(means <= threshold)
1176     return min(
1177         (int(index) for index in eligible),
1178         key=lambda index: (len(path[index].support_indices), index),
1179     )
1180
1181
1182 def fit_laminar_l1_sensitivity(
1183     scenarios: Sequence[dict],
1184     *,
1185     validation_scenarios: Sequence[dict] | None = None,
1186     initial_supports: Iterable[Iterable[int]] | None = None,
1187     candidate_supports: Iterable[Iterable[int]] | None = None,
1188     max_atoms: int | None = None,
1189     r_upper_bound: float | None = None,
1190     x_upper_bound: float | None = None,
1191     bound_safety_factor: float = 2.0,
1192     bound_expansion_factor: float = 2.0,
1193     max_bound_expansions: int = 4,
1194     bound_active_fraction: float = 0.995,
1195     improvement_abs_tol: float = 1e-10,
1196     improvement_rel_tol: float = 1e-8,
1197     zero_tolerance: float = 1e-9,
1198     validation_blocks: int = 4,
1199     time_limit: float | None = None,
1200     mip_rel_gap: float = 0.0,
1201     presolve: bool = True,
1202     disp: bool = False,
1203 ) -> LaminarL1Result:
1204     """Fit a forward path of exact bounded one-atom laminar L1 MILPs.
1205
1206     A non-optimal extension (for example a time limit) is never accepted.
1207     If candidate supports are supplied, each forward extension is globally
1208     optimal over that finite data-derived pool; otherwise it searches all
1209     admissible subsets.
1210     ``initial_supports`` are fixed topology blocks: their weights are jointly
1211     refitted, but the supports themselves are retained and constrain every
1212     later extension. If validation scenarios are supplied, the returned model
1213     is the smallest path point within one standard error of the minimum

```

```

1214     validation MAE, starting from that initial family.
1215     """
1216
1217     prepared = _prepare_scenarios(scenarios)
1218     if validation_scenarios is not None:
1219         validation_prepared = _prepare_scenarios(validation_scenarios)
1220         if validation_prepared.labels != prepared.labels:
1221             raise ValueError("training and validation terminal labels must agree")
1222         if validation_prepared.names != prepared.names:
1223             raise ValueError(
1224                 "training and validation must contain the same scenarios in the same order "
1225                 "when training intercepts are reused"
1226             )
1227     if max_atoms is None:
1228         max_atoms = 2 * prepared.n - 1
1229     if not isinstance(max_atoms, (int, np.integer)):
1230         raise ValueError("max_atoms must be an integer")
1231     max_atoms = int(max_atoms)
1232     if max_atoms < 0 or max_atoms > 2 * prepared.n - 1:
1233         raise ValueError("max_atoms must lie between zero and 2*n-1")
1234     initial_family = normalize_supports(
1235         () if initial_supports is None else initial_supports,
1236         prepared.n,
1237     )
1238     candidate_pool = (
1239         None
1240         if candidate_supports is None
1241         else normalize_support_pool(candidate_supports, prepared.n)
1242     )
1243     if len(initial_family) > max_atoms:
1244         raise ValueError("initial_supports cannot exceed max_atoms")
1245     if not np.isfinite(bound_safety_factor) or bound_safety_factor <= 1.0:
1246         raise ValueError("bound_safety_factor must exceed one")
1247     if not np.isfinite(bound_active_fraction) or not (
1248         0.0 < bound_active_fraction < 1.0
1249     ):
1250         raise ValueError("bound_active_fraction must lie strictly between zero and one")
1251     if not np.isfinite(bound_expansion_factor) or bound_expansion_factor <= 1.0:
1252         raise ValueError("bound_expansion_factor must exceed one")
1253     if not isinstance(max_bound_expansions, (int, np.integer)) or max_bound_expansions < 0:
1254         raise ValueError("max_bound_expansions must be a nonnegative integer")
1255     max_bound_expansions = int(max_bound_expansions)
1256     improvement_abs_tol = _require_nonnegative_finite(
1257         "improvement_abs_tol", improvement_abs_tol
1258     )
1259     improvement_rel_tol = _require_nonnegative_finite(
1260         "improvement_rel_tol", improvement_rel_tol
1261     )
1262     zero_tolerance = _require_nonnegative_finite("zero_tolerance", zero_tolerance)
1263     if not isinstance(validation_blocks, (int, np.integer)) or validation_blocks < 1:
1264         raise ValueError("validation_blocks must be a positive integer")
1265     validation_blocks = int(validation_blocks)
1266
1267     if r_upper_bound is None or x_upper_bound is None:
1268         estimated_r, estimated_x = estimate_atom_upper_bounds(
1269             scenarios, safety_factor=bound_safety_factor
1270         )
1271     if r_upper_bound is None:

```

```

1272         r_upper_bound = estimated_r
1273         if x_upper_bound is None:
1274             x_upper_bound = estimated_x
1275     r_bound, x_bound, time_limit, checked_gap = _validate_solver_parameters(
1276         r_upper_bound=float(r_upper_bound),
1277         x_upper_bound=float(x_upper_bound),
1278         time_limit=time_limit,
1279         mip_rel_gap=mip_rel_gap,
1280     )
1281     mip_rel_gap = float(checked_gap)
1282
1283     attempts: list[ExtensionSolution] = []
1284     stop_reason = "max_atoms"
1285     bound_expansions = 0
1286     supports: tuple[IndexSupport, ...] = initial_family
1287     frozen_support_count = len(initial_family)
1288
1289     def initialize_path() -> tuple[FixedSupportSolution, list[LaminarL1PathPoint]]:
1290         base = _solve_fixed_prepared(
1291             prepared,
1292             initial_family,
1293             r_upper_bound=r_bound,
1294             x_upper_bound=x_bound,
1295             time_limit=time_limit,
1296             presolve=presolve,
1297             disp=disp,
1298         )
1299         return base, [
1300             _make_path_point(
1301                 iteration=frozen_support_count,
1302                 prepared=prepared,
1303                 solution=base,
1304                 validation_scenarios=validation_scenarios,
1305                 validation_blocks=validation_blocks,
1306                 accepted_gain=None,
1307                 solver=base.diagnostics,
1308             )
1309         ]
1310
1311     current, path = initialize_path()
1312     seen_families: set[frozenset[IndexSupport]] = {frozenset(initial_family)}
1313     iteration = frozen_support_count + 1
1314     while iteration <= max_atoms:
1315         gain_tolerance = improvement_abs_tol + improvement_rel_tol * max(
1316             abs(current.objective), np.finfo(float).eps
1317         )
1318
1319         extension = _solve_extension_prepared(
1320             prepared,
1321             supports,
1322             candidate_supports=candidate_pool,
1323             r_upper_bound=r_bound,
1324             x_upper_bound=x_bound,
1325             time_limit=time_limit,
1326             mip_rel_gap=mip_rel_gap,
1327             presolve=presolve,
1328             disp=disp,
1329         )

```

```

1330     attempts.append(extension)
1331     if extension.diagnostics.status == 2:
1332         stop_reason = "no_feasible_extension"
1333         break
1334     if not solver_diagnostics_prove_optimality(extension.diagnostics):
1335         if _dual_bound_certifies_no_gain(
1336             current.objective, extension.diagnostics, gain_tolerance
1337         ):
1338             stop_reason = "no_significant_one_atom_gain_certified_by_dual_bound"
1339         else:
1340             stop_reason = "extension_not_proven_optimal"
1341         break
1342     if extension.support is None or extension.objective is None:
1343         stop_reason = "extension_without_incumbent"
1344         break
1345
1346     candidate_supports = (*supports, extension.support)
1347     polished = _solve_fixed_prepared(
1348         prepared,
1349         candidate_supports,
1350         r_upper_bound=r_bound,
1351         x_upper_bound=x_bound,
1352         time_limit=time_limit,
1353         presolve=presolve,
1354         disp=disp,
1355     )
1356     if not _objectives_agree(polished.objective, extension.objective):
1357         raise RuntimeError(
1358             "fixed-support LP does not reproduce the optimal MILP objective: "
1359             f"milp={extension.objective:.12g}, fixed={polished.objective:.12g}"
1360         )
1361     provisional_gain = current.objective - polished.objective
1362     if provisional_gain <= gain_tolerance:
1363         stop_reason = "no_significant_one_atom_gain"
1364         break
1365
1366     r_active = _touches_bound(polished.r_values, r_bound, bound_active_fraction)
1367     x_active = _touches_bound(polished.x_values, x_bound, bound_active_fraction)
1368     if r_active or x_active:
1369         if bound_expansions >= max_bound_expansions:
1370             stop_reason = "bound_expansion_limit"
1371             break
1372         if r_active:
1373             r_bound *= bound_expansion_factor
1374         if x_active:
1375             x_bound *= bound_expansion_factor
1376         bound_expansions += 1
1377
1378     # A larger coefficient domain can alter every earlier greedy
1379     # support choice. Restarting is required for a coherent exact
1380     # forward path under the final bounds.
1381     supports = initial_family
1382     current, path = initialize_path()
1383     seen_families = {frozenset(initial_family)}
1384     iteration = frozen_support_count + 1
1385     continue
1386
1387     # Remove exactly inactive atoms only when a re-solve preserves the L1 optimum.

```

```

1388     active_indices = list(range(frozen_support_count))
1389     active_indices.extend(
1390         index
1391         for index, (r_value, x_value) in enumerate(
1392             zip(polished.r_values, polished.x_values, strict=True)
1393         )
1394         if index >= frozen_support_count
1395         and (r_value > zero_tolerance or x_value > zero_tolerance)
1396     )
1397     if len(active_indices) < len(candidate_supports):
1398         reduced_supports = tuple(candidate_supports[index] for index in active_indices)
1399         reduced = _solve_fixed_prepared(
1400             prepared,
1401             reduced_supports,
1402             r_upper_bound=r_bound,
1403             x_upper_bound=x_bound,
1404             time_limit=time_limit,
1405             presolve=presolve,
1406             disp=disp,
1407         )
1408         if reduced.objective <= polished.objective + gain_tolerance:
1409             candidate_supports = reduced_supports
1410             polished = reduced
1411
1412     gain = current.objective - polished.objective
1413     if gain <= gain_tolerance:
1414         stop_reason = "no_significant_one_atom_gain_after_polishing"
1415         break
1416
1417     family_signature = frozenset(candidate_supports)
1418     if family_signature in seen_families:
1419         stop_reason = "repeated_family_after_zero_pruning"
1420         break
1421     seen_families.add(family_signature)
1422     supports = candidate_supports
1423     current = polished
1424     path.append(
1425         _make_path_point(
1426             iteration=iteration,
1427             prepared=prepared,
1428             solution=current,
1429             validation_scenarios=validation_scenarios,
1430             validation_blocks=validation_blocks,
1431             accepted_gain=float(gain),
1432             solver=extension.diagnostics,
1433         )
1434     )
1435     if len(supports) >= 2 * prepared.n - 1:
1436         stop_reason = "laminar_family_limit"
1437         break
1438     iteration += 1
1439
1440     selected_index = _choose_path_point(path)
1441     selected = path[selected_index]
1442     return LaminarL1Result(
1443         terminal_labels=prepared.labels,
1444         support_indices=selected.support_indices,
1445         support_labels=selected.support_labels,

```

```
1446     r_values=selected.r_values.copy(),
1447     x_values=selected.x_values.copy(),
1448     intercepts=selected.intercepts.copy(),
1449     r_matrix=selected.r_matrix.copy(),
1450     x_matrix=selected.x_matrix.copy(),
1451     train_mae=selected.train_mae,
1452     validation_mae=selected.validation_mae,
1453     selected_path_index=selected_index,
1454     path=tuple(path),
1455     attempted_extensions=tuple(attempts),
1456     stop_reason=stop_reason,
1457     r_upper_bound=r_bound,
1458     x_upper_bound=x_bound,
1459     bound_expansions=bound_expansions,
1460 )
1461
1462
1463 __all__ = [
1464     "ExtensionSolution",
1465     "FixedSupportSolution",
1466     "LaminarL1PathPoint",
1467     "LaminarL1Result",
1468     "SolverDiagnostics",
1469     "build_matrices_from_atoms",
1470     "estimate_atom_upper_bounds",
1471     "evaluate_l1_matrices",
1472     "fit_laminar_l1_sensitivity",
1473     "is_admissible_extension",
1474     "is_laminar_family",
1475     "normalize_support_pool",
1476     "normalize_supports",
1477     "solver_diagnostics_prove_optimality",
1478     "solve_best_laminar_extension_l1",
1479     "solve_fixed_support_l1",
1480 ]
```

Listing 2: terminal_case33/estimation/laminar_l1_milp.py

B 单元测试完整源码

```

1  """Tests for exact laminar one-atom L1 MILP regression."""
2
3  from itertools import combinations
4
5  import networkx as nx
6  import numpy as np
7  import pytest
8
9  scipy_optimize = pytest.importorskip(
10     "scipy.optimize", reason="requires scipy.optimize.milp/HiGHS"
11 )
12 if not hasattr(scipy_optimize, "milp"):
13     pytest.skip("requires scipy.optimize.milp/HiGHS", allow_module_level=True)
14
15 from terminal_case33.data.small_terminal_lv import build_small_terminal_lv_case
16 from terminal_case33.estimation.laminar_l1_milp import (
17     SolverDiagnostics,
18     build_matrices_from_atoms,
19     evaluate_l1_matrices,
20     fit_laminar_l1_sensitivity,
21     is_admissible_extension,
22     is_laminar_family,
23     normalize_support_pool,
24     solver_diagnostics_prove_optimality,
25     solve_best_laminar_extension_l1,
26     solve_fixed_support_l1,
27 )
28 from terminal_case33.models.lin_distflow import build_reduced_sensitivity_matrices
29
30
31 def _scenario_from_atoms(
32     *,
33     n: int,
34     supports: tuple[tuple[int, ...], ...],
35     r_values: tuple[float, ...],
36     x_values: tuple[float, ...],
37     samples: int,
38     seed: int,
39     noise_scale: float = 0.0,
40 ) -> dict:
41     rng = np.random.default_rng(seed)
42     p = rng.normal(size=(samples, n))
43     q = rng.normal(size=(samples, n))
44     r_matrix, x_matrix = build_matrices_from_atoms(
45         n, supports, r_values, x_values
46     )
47     intercept = np.linspace(-0.2, 0.2, n)
48     target = p @ r_matrix.T + q @ x_matrix.T + intercept
49     if noise_scale:
50         target += rng.laplace(0.0, noise_scale, size=target.shape)
51     return {
52         "name": "synthetic",
53         "P_terminal": p,
54         "Q_terminal": q,
55         "drop_target": target,

```

```

56     }
57
58
59 def _all_nonempty_supports(n: int):
60     for size in range(1, n + 1):
61         yield from combinations(range(n), size)
62
63
64 def test_laminar_predicate_and_atom_matrix_properties() -> None:
65     family = ((0, 1, 2), (0, 1), (0,), (1,), (3,))
66     assert is_laminar_family(family)
67     assert not is_laminar_family(((0, 1), (1, 2)))
68
69     r_matrix, x_matrix = build_matrices_from_atoms(
70         4,
71         family,
72         (0.4, 0.3, 0.2, 0.1, 0.5),
73         (0.2, 0.15, 0.1, 0.05, 0.25),
74     )
75     for matrix in (r_matrix, x_matrix):
76         assert np.allclose(matrix, matrix.T)
77         assert np.min(matrix) >= 0.0
78         assert np.min(np.linalg.eigvalsh(matrix)) >= -1e-12
79     assert np.isclose(r_matrix[0, 0], 0.9)
80     assert np.isclose(r_matrix[0, 1], 0.7)
81     assert np.isclose(r_matrix[0, 2], 0.4)
82
83
84 def test_singleton_changes_only_diagonal_and_atom_inputs_are_validated() -> None:
85     r_matrix, x_matrix = build_matrices_from_atoms(4, ((2,),), (0.7,), (0.3,))
86     expected_r = np.zeros((4, 4))
87     expected_x = np.zeros((4, 4))
88     expected_r[2, 2] = 0.7
89     expected_x[2, 2] = 0.3
90     assert np.array_equal(r_matrix, expected_r)
91     assert np.array_equal(x_matrix, expected_x)
92
93     with pytest.raises(ValueError, match="nonnegative"):
94         build_matrices_from_atoms(4, ((0, 1),), (-0.1,), (0.2,))
95     with pytest.raises(ValueError, match="duplicate"):
96         build_matrices_from_atoms(4, ((0, 0),), (0.1,), (0.2,))
97     with pytest.raises(ValueError, match="terminal range"):
98         build_matrices_from_atoms(4, ((0, 4),), (0.1,), (0.2,))
99
100
101 def test_solver_certificate_requires_reported_gap_or_primal_dual_bound() -> None:
102     common = {
103         "status": 0,
104         "success": True,
105         "message": "optimal",
106         "objective": 1.0,
107         "dual_bound": None,
108         "mip_gap": None,
109         "node_count": 0,
110         "runtime_seconds": 0.0,
111         "variable_count": 3,
112         "binary_variable_count": 1,
113         "constraint_count": 2,

```

```

114 }
115 assert not solver_diagnostics_prove_optimality(SolverDiagnostics(**common))
116 assert solver_diagnostics_prove_optimality(
117     SolverDiagnostics(**{**common, "mip_gap": 0.0})
118 )
119 assert solver_diagnostics_prove_optimality(
120     SolverDiagnostics(
121         **{
122             **common,
123             "dual_bound": 1.0 - 5e-11,
124             "mip_gap": 0.5,
125         }
126     )
127 )
128 assert not solver_diagnostics_prove_optimality(
129     SolverDiagnostics(**{**common, "status": 1, "mip_gap": 0.0})
130 )
131 assert not solver_diagnostics_prove_optimality(
132     SolverDiagnostics(
133         **{
134             **common,
135             "dual_bound": 2.0,
136             "mip_gap": 0.5,
137         }
138     )
139 )
140 assert not solver_diagnostics_prove_optimality(
141     SolverDiagnostics(**{**common, "mip_gap": -1e-12})
142 )
143 assert solver_diagnostics_prove_optimality(
144     SolverDiagnostics(
145         **{
146             **common,
147             "binary_variable_count": 0,
148             "dual_bound": None,
149             "mip_gap": None,
150         }
151     )
152 )
153
154
155 def test_exact_single_atom_recovers_support_diagonal_and_values() -> None:
156     scenario = _scenario_from_atoms(
157         n=4,
158         supports=((0, 2, 3),),
159         r_values=(0.7,),
160         x_values=(0.3,),
161         samples=64,
162         seed=11,
163     )
164     solution = solve_best_laminar_extension_l1(
165         [scenario],
166         (),
167         r_upper_bound=2.0,
168         x_upper_bound=2.0,
169         mip_rel_gap=0.0,
170     )
171     assert solution.diagnostics.status == 0

```

```

172     assert solver_diagnostics_prove_optimality(solution.diagnostics)
173     assert solution.support == (0, 2, 3)
174     assert np.isclose(solution.r_values[-1], 0.7, atol=1e-8)
175     assert np.isclose(solution.x_values[-1], 0.3, atol=1e-8)
176     assert solution.objective is not None and solution.objective < 1e-9
177
178     r_matrix, x_matrix = build_matrices_from_atoms(
179         4, (solution.support,), solution.r_values[-1:], solution.x_values[-1:]
180     )
181     expected_r, expected_x = build_matrices_from_atoms(
182         4, ((0, 2, 3),), (0.7,), (0.3,)
183     )
184     assert np.allclose(r_matrix, expected_r, atol=1e-8)
185     assert np.allclose(x_matrix, expected_x, atol=1e-8)
186     assert r_matrix[0, 0] == r_matrix[0, 2] == r_matrix[2, 3]
187
188
189 def test_finite_candidate_pool_is_canonical_and_exactly_enforced() -> None:
190     assert normalize_support_pool(((2, 0), (0, 2), (1, 2)), 3) == (
191         (0, 2),
192         (1, 2),
193     )
194     scenario = _scenario_from_atoms(
195         n=4,
196         supports=((0, 2, 3),),
197         r_values=(0.7,),
198         x_values=(0.3,),
199         samples=48,
200         seed=13,
201     )
202     pool = ((0, 1), (0, 2, 3), (1, 3))
203
204     solution = solve_best_laminar_extension_l1(
205         [scenario],
206         (),
207         candidate_supports=pool,
208         r_upper_bound=2.0,
209         x_upper_bound=2.0,
210         mip_rel_gap=0.0,
211     )
212
213     assert solver_diagnostics_prove_optimality(solution.diagnostics)
214     assert solution.support == (0, 2, 3)
215     assert solution.support in pool
216
217
218 def test_finite_candidate_pool_reports_exhaustion_after_laminar_filter() -> None:
219     scenario = _scenario_from_atoms(
220         n=3,
221         supports=((0, 1),),
222         r_values=(0.5,),
223         x_values=(0.2,),
224         samples=24,
225         seed=15,
226     )
227     solution = solve_best_laminar_extension_l1(
228         [scenario],
229         ((0, 1),),

```

```

230     candidate_supports=((0, 1), (1, 2)),
231     r_upper_bound=2.0,
232     x_upper_bound=2.0,
233 )
234
235 assert solution.diagnostics.status == 2
236 assert solution.support is None
237
238
239 def test_milp_objective_matches_exhaustive_subset_oracle() -> None:
240     existing = ((0, 1, 2),)
241     scenario = _scenario_from_atoms(
242         n=4,
243         supports=((0, 1, 2), (0, 1)),
244         r_values=(0.25, 0.8),
245         x_values=(0.15, 0.35),
246         samples=40,
247         seed=17,
248         noise_scale=0.002,
249     )
250     milp_solution = solve_best_laminar_extension_l1(
251         [scenario],
252         existing,
253         r_upper_bound=2.0,
254         x_upper_bound=2.0,
255         mip_rel_gap=0.0,
256     )
257     assert milp_solution.diagnostics.status == 0
258     assert milp_solution.objective is not None
259
260     candidates: list[tuple[tuple[int, ...], float]] = []
261     for candidate in _all_nonempty_supports(4):
262         if not is_admissible_extension(candidate, existing, 4):
263             continue
264         fit = solve_fixed_support_l1(
265             [scenario],
266             (*existing, candidate),
267             r_upper_bound=2.0,
268             x_upper_bound=2.0,
269         )
270         candidates.append((candidate, fit.objective))
271     best_objective = min(value for _, value in candidates)
272     best_supports = {
273         support for support, value in candidates if value <= best_objective + 1e-9
274     }
275     assert np.isclose(milp_solution.objective, best_objective, atol=1e-9)
276     assert milp_solution.support in best_supports
277     assert solver_diagnostics_prove_optimality(milp_solution.diagnostics)
278
279     r_matrix, x_matrix = build_matrices_from_atoms(
280         4,
281         (*existing, milp_solution.support),
282         milp_solution.r_values,
283         milp_solution.x_values,
284     )
285     p = np.asarray(scenario["P_terminal"], dtype=float)
286     q = np.asarray(scenario["Q_terminal"], dtype=float)
287     target = np.asarray(scenario["drop_target"], dtype=float)

```

```

288     prediction = (
289         p @ r_matrix.T
290         + q @ x_matrix.T
291         + milp_solution.intercepts[0][None, :])
292     )
293     assert np.isclose(
294         np.mean(np.abs(target - prediction)),
295         milp_solution.objective,
296         atol=1e-9,
297     )
298
299
300 def test_crossing_extension_is_excluded() -> None:
301     existing = ((0, 1),)
302     scenario = _scenario_from_atoms(
303         n=3,
304         supports=((0, 1), (1, 2)),
305         r_values=(0.1, 1.0),
306         x_values=(0.05, 0.4),
307         samples=48,
308         seed=23,
309     )
310     solution = solve_best_laminar_extension_l1(
311         [scenario],
312         existing,
313         r_upper_bound=2.0,
314         x_upper_bound=2.0,
315     )
316     assert solution.diagnostics.status == 0
317     assert solution.support != (1, 2)
318     assert solution.support is not None
319     assert is_admissible_extension(solution.support, existing, 3)
320
321
322 def test_no_admissible_extension_reports_infeasible_without_support() -> None:
323     scenario = _scenario_from_atoms(
324         n=1,
325         supports=((0,)),),
326         r_values=(0.4,),
327         x_values=(0.2,),
328         samples=8,
329         seed=27,
330     )
331     solution = solve_best_laminar_extension_l1(
332         [scenario],
333         ((0,)),),
334         r_upper_bound=1.0,
335         x_upper_bound=1.0,
336     )
337     assert solution.diagnostics.status == 2
338     assert not solution.diagnostics.success
339     assert solution.support is None
340     assert solution.objective is None
341
342
343 def test_deterministic_three_atom_path_is_genuinely_fully_corrective() -> None:
344     n = 4
345     identity = np.eye(n)

```

```

346 zero = np.zeros_like(identity)
347 p = np.vstack([identity, -identity, zero, zero])
348 q = np.vstack([zero, zero, identity, -identity])
349 supports = ((0, 1, 2, 3), (0, 1, 2), (0,))
350 r_values = (10.0, 2.0, 0.5)
351 x_values = (8.0, 1.5, 0.4)
352 r_true, x_true = build_matrices_from_atoms(
353     n, supports, r_values, x_values
354 )
355 scenario = {
356     "name": "deterministic_hierarchy",
357     "P_terminal": p,
358     "Q_terminal": q,
359     "drop_target": p @ r_true.T + q @ x_true.T,
360 }
361 result = fit_laminar_l1_sensitivity(
362     [scenario],
363     max_atoms=3,
364     r_upper_bound=20.0,
365     x_upper_bound=20.0,
366     max_bound_expansions=0,
367     improvement_abs_tol=1e-12,
368     improvement_rel_tol=1e-12,
369 )
370 assert result.path[1].support_indices == (supports[0],)
371 assert result.path[2].support_indices == supports[:2]
372 assert result.path[3].support_indices == supports
373 assert np.allclose(result.path[1].r_values, (12.0,), atol=1e-8)
374 assert np.allclose(result.path[1].x_values, (9.5,), atol=1e-8)
375 assert np.allclose(result.path[2].r_values, (10.0, 2.0), atol=1e-8)
376 assert np.allclose(result.path[2].x_values, (8.0, 1.5), atol=1e-8)
377 assert np.allclose(result.r_values, r_values, atol=1e-8)
378 assert np.allclose(result.x_values, x_values, atol=1e-8)
379 assert np.allclose(result.r_matrix, r_true, atol=1e-8)
380 assert np.allclose(result.x_matrix, x_true, atol=1e-8)
381 assert result.train_mae < 1e-10
382
383
384 def test_forward_path_is_fully_corrective_monotone_and_laminar() -> None:
385     scenario = _scenario_from_atoms(
386         n=4,
387         supports=((0, 1, 2, 3), (0, 1), (0,), (1,), (2,), (3,)),
388         r_values=(0.15, 0.55, 0.20, 0.25, 0.30, 0.35),
389         x_values=(0.08, 0.25, 0.10, 0.12, 0.14, 0.16),
390         samples=72,
391         seed=29,
392         noise_scale=0.001,
393     )
394     result = fit_laminar_l1_sensitivity(
395         [scenario],
396         max_atoms=6,
397         r_upper_bound=2.0,
398         x_upper_bound=2.0,
399         improvement_abs_tol=1e-9,
400     )
401     objectives = np.asarray([point.train_mae for point in result.path])
402     assert np.all(np.diff(objectives) <= 1e-9)
403     assert all(is_laminar_family(point.support_indices) for point in result.path)

```

```

404     for point in result.path:
405         fixed = solve_fixed_support_l1(
406             [scenario],
407             point.support_indices,
408             r_upper_bound=2.0,
409             x_upper_bound=2.0,
410         )
411         assert np.isclose(fixed.objective, point.train_mae, atol=1e-9)
412         assert np.allclose(fixed.r_values, point.r_values, atol=1e-8)
413         assert np.allclose(fixed.x_values, point.x_values, atol=1e-8)
414         for matrix in (point.r_matrix, point.x_matrix):
415             assert np.allclose(matrix, matrix.T)
416             assert np.min(matrix) >= -1e-10
417             assert np.min(np.linalg.eigvalsh(matrix)) >= -1e-8
418         assert all(
419             solver_diagnostics_prove_optimality(attempt.diagnostics)
420             for attempt in result.attempted_extensions
421         )
422
423
424 def test_validation_path_reuses_training_intercept() -> None:
425     n = 2
426     p_train = np.zeros((6, n))
427     q_train = np.zeros((6, n))
428     train_level = np.array([1.0, -2.0])
429     validation_shift = np.array([7.0, -5.0])
430     training = {
431         "name": "same_scenario",
432         "P_terminal": p_train,
433         "Q_terminal": q_train,
434         "drop_target": np.tile(train_level, (6, 1)),
435     }
436     validation = {
437         "name": "same_scenario",
438         "P_terminal": np.zeros((4, n)),
439         "Q_terminal": np.zeros((4, n)),
440         "drop_target": np.tile(train_level + validation_shift, (4, 1)),
441     }
442     result = fit_laminar_l1_sensitivity(
443         [training],
444         validation_scenarios=[validation],
445         max_atoms=0,
446         r_upper_bound=1.0,
447         x_upper_bound=1.0,
448     )
449     fixed_mae, _, _ = evaluate_l1_matrices(
450         [validation],
451         result.r_matrix,
452         result.x_matrix,
453         fixed_intercepts=result.intercepts,
454     )
455     profiled_mae, _, _ = evaluate_l1_matrices(
456         [validation], result.r_matrix, result.x_matrix
457     )
458     assert np.allclose(result.intercepts[0], train_level, atol=1e-9)
459     assert np.isclose(fixed_mae, np.mean(np.abs(validation_shift)))
460     assert np.isclose(result.validation_mae, fixed_mae)
461     assert profiled_mae < 1e-12

```

```

462
463
464 def test_zero_signal_stops_without_accepting_an_atom() -> None:
465     rng = np.random.default_rng(41)
466     scenario = {
467         "name": "zero_signal",
468         "P_terminal": rng.normal(size=(16, 3)),
469         "Q_terminal": rng.normal(size=(16, 3)),
470         "drop_target": np.tile(np.array([0.4, -0.2, 0.1]), (16, 1)),
471     }
472     result = fit_laminar_l1_sensitivity(
473         [scenario],
474         max_atoms=3,
475         r_upper_bound=2.0,
476         x_upper_bound=2.0,
477         improvement_abs_tol=1e-10,
478         improvement_rel_tol=1e-10,
479     )
480     assert result.support_indices == ()
481     assert len(result.path) == 1
482     assert result.stop_reason == "no_significant_one_atom_gain"
483     assert len(result.attempted_extensions) == 1
484     assert solver_diagnostics_prove_optimality(
485         result.attempted_extensions[0].diagnostics
486     )
487
488
489 def test_initial_support_is_fitted_and_cannot_be_pruned() -> None:
490     scenario = _scenario_from_atoms(
491         n=3,
492         supports=((0, 1),),
493         r_values=(0.7,),
494         x_values=(0.3,),
495         samples=48,
496         seed=143,
497         noise_scale=0.0,
498     )
499     result = fit_laminar_l1_sensitivity(
500         [scenario],
501         initial_supports=((0, 1),),
502         max_atoms=1,
503         r_upper_bound=2.0,
504         x_upper_bound=2.0,
505     )
506     assert result.support_indices == ((0, 1),)
507     assert result.path[0].iteration == 1
508     assert result.attempted_extensions == ()
509     assert np.allclose(result.r_values, (0.7,), atol=1e-8)
510     assert np.allclose(result.x_values, (0.3,), atol=1e-8)
511     assert result.train_mae < 1e-10
512
513
514 def test_initial_supports_must_be_laminar_and_fit_atom_budget() -> None:
515     scenario = _scenario_from_atoms(
516         n=3,
517         supports=((0, 1),),
518         r_values=(0.2,),
519         x_values=(0.1,),

```

```

520     samples=16,
521     seed=149,
522     noise_scale=0.0,
523 )
524 with pytest.raises(ValueError, match="laminar"):
525     fit_laminar_l1_sensitivity(
526         [scenario],
527         initial_supports=((0, 1), (1, 2)),
528         max_atoms=2,
529         r_upper_bound=1.0,
530         x_upper_bound=1.0,
531     )
532 with pytest.raises(ValueError, match="cannot exceed"):
533     fit_laminar_l1_sensitivity(
534         [scenario],
535         initial_supports=((0,), (1,)),
536         max_atoms=1,
537         r_upper_bound=1.0,
538         x_upper_bound=1.0,
539     )
540
541
542 def test_small6_physical_edge_atoms_reconstruct_exact_sensitivity() -> None:
543     net = build_small_terminal_lv_case()
544     terminals = (
545         net.buses.loc[
546             net.buses["bus_type"].eq("observed_terminal"), "bus_id"
547         ]
548         .astype(int)
549         .tolist()
550     )
551     terminal_set = set(terminals)
552     terminal_position = {terminal: index for index, terminal in enumerate(terminals)}
553     graph = net.to_networkx_graph()
554     oriented = nx.bfs_tree(graph, net.root_bus)
555     impedance_base = net.base_kv**2 / net.base_mva
556     grouped: dict[tuple[int, ...], list[float]] = {}
557     for parent, child in oriented.edges():
558         descendants = {
559             int(child),
560             *(int(node) for node in nx.descendants(oriented, child)),
561         }
562         support_labels = tuple(sorted(descendants & terminal_set))
563         if not support_labels:
564             continue
565         support = tuple(terminal_position[label] for label in support_labels)
566         values = grouped.setdefault(support, [0.0, 0.0])
567         values[0] += 2.0 * float(graph.edges[parent, child]["r_ohm"]) / impedance_base
568         values[1] += 2.0 * float(graph.edges[parent, child]["x_ohm"]) / impedance_base
569
570     supports = tuple(grouped)
571     reconstructed_r, reconstructed_x = build_matrices_from_atoms(
572         len(terminals),
573         supports,
574         tuple(grouped[support][0] for support in supports),
575         tuple(grouped[support][1] for support in supports),
576     )
577     true_r, true_x = build_reduced_sensitivity_matrices(

```

```
578         net, terminals, voltage_model="squared-voltage"
579     )
580     assert is_laminar_family(supports)
581     assert np.allclose(reconstructed_r, true_r, atol=1e-12)
582     assert np.allclose(reconstructed_x, true_x, atol=1e-12)
```

Listing 3: tests/test_laminar_l1_milp.py

C small6 算例驱动完整源码

```

1  """Run the exact laminar L1-MILP estimator on the six-terminal LV case.
2
3  The experiment deliberately separates two questions:
4
5  * a noiseless squared-voltage LinDistFlow case checks whether the implemented
6    regression can recover a matrix that is exactly in the atom model; and
7  * a robust case adds Laplace response noise and sparse gross outliers before a
8    contiguous train/validation/test split.
9
10 Both regimes use the same continuous P/Q realization and the same physical
11 truth. No truth information is supplied to the estimator.
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import json
18 from math import ceil
19 from pathlib import Path
20 from time import perf_counter
21
22 import networkx as nx
23 import numpy as np
24 import pandas as pd
25
26 from terminal_case33.data.small_terminal_lv import build_small_terminal_lv_case
27 from terminal_case33.estimation.laminar_l1_milp import (
28     evaluate_l1_matrices,
29     fit_laminar_l1_sensitivity,
30     solver_diagnostics_prove_optimality,
31 )
32 from terminal_case33.graph.rooted_hierarchy import rooted_clades
33 from terminal_case33.models.lin_distflow import build_reduced_sensitivity_matrices
34 from terminal_case33.utils.io import ensure_dir, write_json
35
36
37 DEFAULT_OUTPUT = Path("outputs/laminar_l1_milp_small6")
38
39
40 def _terminal_labels(net) -> list[int]:
41     return (
42         net.buses.loc[net.buses["bus_type"].eq("observed_terminal"), "bus_id"]
43         .astype(int)
44         .tolist()
45     )
46
47
48 def _ar1_standardized(
49     rng: np.random.Generator, time_count: int, terminal_count: int, rho: float
50 ) -> np.ndarray:
51     """Generate smooth, terminal-specific unit-scale excitation."""
52
53     innovations = rng.normal(size=(time_count, terminal_count))
54     values = np.zeros_like(innovations)
55     values[0] = innovations[0]

```

```

56 innovation_scale = np.sqrt(1.0 - rho**2)
57 for time_index in range(1, time_count):
58     values[time_index] = (
59         rho * values[time_index - 1]
60         + innovation_scale * innovations[time_index]
61     )
62 values -= values.mean(axis=0, keepdims=True)
63 values /= np.maximum(values.std(axis=0, keepdims=True), 1e-12)
64 return values
65
66
67 def _continuous_pq(net, time_count: int, seed: int) -> tuple[pd.DataFrame, pd.DataFrame]:
68     """Create one physically scaled continuous P/Q trajectory.
69
70     P and Q have separate smooth innovations so that R and X are identifiable;
71     a common slow mode keeps the trajectory representative of a feeder profile.
72     """
73
74     terminals = _terminal_labels(net)
75     terminal_frame = net.buses.set_index("bus_id").loc[terminals]
76     base = net.base_mva * 1000.0
77     p_nominal = terminal_frame["pd_kw"].to_numpy(dtype=float) / base
78     q_nominal = terminal_frame["qd_kvar"].to_numpy(dtype=float) / base
79     rng = np.random.default_rng(seed)
80     p_local = _ari_standardized(rng, time_count, len(terminals), rho=0.78)
81     q_local = _ari_standardized(rng, time_count, len(terminals), rho=0.72)
82     time = np.arange(time_count, dtype=float)
83     phases = np.linspace(0.0, 2.0 * np.pi, len(terminals), endpoint=False)
84     common_p = np.sin(2.0 * np.pi * time / time_count)
85     common_q = np.cos(2.0 * np.pi * time / time_count + 0.35)
86     terminal_wave_p = np.sin(6.0 * np.pi * time[:, None] / time_count + phases)
87     terminal_wave_q = np.cos(8.0 * np.pi * time[:, None] / time_count + 0.7 * phases)
88     p_multiplier = (
89         1.0
90         + 0.22 * common_p[:, None]
91         + 0.24 * p_local
92         + 0.10 * terminal_wave_p
93     )
94     q_multiplier = (
95         1.0
96         + 0.18 * common_q[:, None]
97         + 0.28 * q_local
98         + 0.12 * terminal_wave_q
99     )
100     p = np.maximum(0.15 * p_nominal[None, :], p_nominal[None, :] * p_multiplier)
101     q = np.maximum(0.15 * q_nominal[None, :], q_nominal[None, :] * q_multiplier)
102     index = pd.RangeIndex(time_count, name="time_index")
103     return (
104         pd.DataFrame(p, index=index, columns=terminals),
105         pd.DataFrame(q, index=index, columns=terminals),
106     )
107
108
109 def _response_regimes(
110     exact_target: pd.DataFrame,
111     *,
112     seed: int,
113     outlier_fraction: float,

```

```

114 ) -> tuple[dict[str, pd.DataFrame], dict, np.ndarray]:
115     """Return noiseless and Laplace-plus-outlier targets on the same timeline."""
116
117     exact = exact_target.to_numpy(dtype=float)
118     centered = exact - np.median(exact, axis=0, keepdims=True)
119     signal_scale = max(float(np.median(np.abs(centered))), 1e-8)
120     laplace_scale = 0.05 * signal_scale
121     gross_outlier_scale = 1.5 * signal_scale
122     rng = np.random.default_rng(seed)
123     laplace = rng.laplace(0.0, laplace_scale, size=exact.shape)
124     outlier_count = max(1, int(ceil(outlier_fraction * exact.size)))
125     flat_locations = rng.choice(exact.size, size=outlier_count, replace=False)
126     outlier_mask = np.zeros(exact.size, dtype=bool)
127     outlier_mask[flat_locations] = True
128     outlier_mask = outlier_mask.reshape(exact.shape)
129     gross = np.zeros_like(exact)
130     gross[outlier_mask] = (
131         rng.choice(np.array([-1.0, 1.0]), size=outlier_count)
132         * gross_outlier_scale
133         * (1.0 + rng.exponential(scale=0.5, size=outlier_count))
134     )
135     contaminated = exact + laplace + gross
136     regimes = {
137         "noiseless": exact_target.copy(),
138         "laplace_outliers": pd.DataFrame(
139             contaminated,
140             index=exact_target.index,
141             columns=exact_target.columns,
142         ),
143     }
144     metadata = {
145         "response_noise_only": True,
146         "laplace_scale_pu2": laplace_scale,
147         "gross_outlier_base_scale_pu2": gross_outlier_scale,
148         "outlier_fraction_requested": outlier_fraction,
149         "outlier_count": outlier_count,
150         "outlier_fraction_realized": outlier_count / exact.size,
151         "signal_median_absolute_deviation_pu2": signal_scale,
152     }
153     return regimes, metadata, outlier_mask
154
155
156 def _scenario_slice(
157     p: pd.DataFrame,
158     q: pd.DataFrame,
159     target: pd.DataFrame,
160     selected: slice,
161 ) -> list[dict]:
162     return [
163         {
164             "name": "small6_continuous_scenario",
165             "P_terminal": p.iloc[selected].copy(),
166             "Q_terminal": q.iloc[selected].copy(),
167             "drop_target": target.iloc[selected].copy(),
168         }
169     ]
170
171

```

```

172 def _score_clades(
173     predicted: set[frozenset[int]], truth: set[frozenset[int]]
174 ) -> dict:
175     matched = predicted & truth
176     precision = len(matched) / len(predicted) if predicted else float(not truth)
177     recall = len(matched) / len(truth) if truth else float(not predicted)
178     f1 = (
179         2.0 * precision * recall / (precision + recall)
180         if precision + recall
181         else 0.0
182     )
183     serialize = lambda family: [
184         list(item) for item in sorted(family, key=lambda value: (len(value), tuple(value)))
185     ]
186     return {
187         "predicted_nontrivial_clade_count": len(predicted),
188         "true_nontrivial_clade_count": len(truth),
189         "matched_nontrivial_clade_count": len(matched),
190         "nontrivial_clade_precision": precision,
191         "nontrivial_clade_recall": recall,
192         "nontrivial_clade_f1": f1,
193         "nontrivial_clade_exact": predicted == truth,
194         "matched_nontrivial_clades": serialize(matched),
195         "missing_nontrivial_clades": serialize(truth - predicted),
196         "extra_nontrivial_clades": serialize(predicted - truth),
197     }
198
199
200 def _matrix_errors(estimate: np.ndarray, truth: np.ndarray, prefix: str) -> dict:
201     difference = np.asarray(estimate) - np.asarray(truth)
202     return {
203         f"{prefix}_matrix_mae": float(np.mean(np.abs(difference))),
204         f"{prefix}_matrix_max_abs_error": float(np.max(np.abs(difference))),
205         f"{prefix}_matrix_relative_frobenius_error": float(
206             np.linalg.norm(difference, ord="fro")
207             / max(np.linalg.norm(truth, ord="fro"), 1e-15)
208         ),
209     }
210
211
212 def _fixed_training_intercept(
213     train: list[dict], r_matrix: np.ndarray, x_matrix: np.ndarray
214 ) -> np.ndarray:
215     scenario = train[0]
216     p = scenario["P_terminal"].to_numpy(dtype=float)
217     q = scenario["Q_terminal"].to_numpy(dtype=float)
218     target = scenario["drop_target"].to_numpy(dtype=float)
219     return np.median(target - p @ r_matrix.T - q @ x_matrix.T, axis=0)
220
221
222 def _heldout_residual_frame(
223     heldout: list[dict],
224     r_matrix: np.ndarray,
225     x_matrix: np.ndarray,
226     intercept: np.ndarray,
227     outlier_mask: np.ndarray,
228 ) -> tuple[pd.DataFrame, float]:
229     scenario = heldout[0]

```

```

230 p_frame = scenario["P_terminal"]
231 q_frame = scenario["Q_terminal"]
232 target_frame = scenario["drop_target"]
233 prediction = (
234     p_frame.to_numpy(dtype=float) @ r_matrix.T
235     + q_frame.to_numpy(dtype=float) @ x_matrix.T
236     + intercept[None, :])
237 )
238 target = target_frame.to_numpy(dtype=float)
239 residual = target - prediction
240 records = []
241 for time_offset, time_label in enumerate(target_frame.index):
242     for terminal_offset, terminal in enumerate(target_frame.columns):
243         records.append(
244             {
245                 "time_index": int(time_label),
246                 "terminal": int(terminal),
247                 "target_pu2": float(target[time_offset, terminal_offset]),
248                 "prediction_pu2": float(prediction[time_offset, terminal_offset]),
249                 "residual_pu2": float(residual[time_offset, terminal_offset]),
250                 "absolute_residual_pu2": float(abs(residual[time_offset, terminal_offset])),
251                 "is_injected_outlier": bool(outlier_mask[time_offset, terminal_offset]),
252             }
253         )
254 return pd.DataFrame(records), float(np.mean(np.abs(residual)))
255
256
257 def _truth_atom_frame(net, terminals: list[int]) -> pd.DataFrame:
258     """Aggregate physical edge coefficients by downstream terminal support."""
259
260     graph = net.to_networkx_graph()
261     oriented = nx.bfs_tree(graph, net.root_bus)
262     terminal_set = set(terminals)
263     position = {terminal: index for index, terminal in enumerate(terminals)}
264     z_base = net.base_kv**2 / net.base_mva
265     grouped: dict[tuple[int, ...], dict] = {}
266     for parent, child in oriented.edges():
267         descendants = {int(child), *(int(node) for node in nx.descendants(oriented, child))}
268         support_labels = tuple(sorted(descendants & terminal_set))
269         if not support_labels:
270             continue
271         edge = graph.edges[parent, child]
272         record = grouped.setdefault(
273             support_labels,
274             {
275                 "support_labels": ",".join(map(str, support_labels)),
276                 "support_indices": ",".join(str(position[node]) for node in support_labels),
277                 "support_size": len(support_labels),
278                 "r_value": 0.0,
279                 "x_value": 0.0,
280                 "physical_edges": [],
281             },
282         )
283         record["r_value"] += 2.0 * float(edge["r_ohm"]) / z_base
284         record["x_value"] += 2.0 * float(edge["x_ohm"]) / z_base
285         record["physical_edges"].append(f"{int(parent)}--{int(child)}")
286     rows = []
287     for record in grouped.values():

```

```

288     copied = dict(record)
289     copied["physical_edges"] = ";".join(copied["physical_edges"])
290     rows.append(copied)
291     return pd.DataFrame(rows).sort_values(
292         ["support_size", "support_labels"], ignore_index=True
293     )
294
295
296 def _path_frame(result, regime: str) -> pd.DataFrame:
297     rows = []
298     for path_index, point in enumerate(result.path):
299         diagnostics = point.solver
300         rows.append(
301             {
302                 "regime": regime,
303                 "path_index": path_index,
304                 "algorithm_iteration": point.iteration,
305                 "selected_model": path_index == result.selected_path_index,
306                 "atom_count": len(point.support_labels),
307                 "supports": ";".join(
308                     ", ".join(map(str, support)) for support in point.support_labels
309                 ),
310                 "train_mae": point.train_mae,
311                 "validation_mae": point.validation_mae,
312                 "validation_se": point.validation_se,
313                 "accepted_gain": point.accepted_gain,
314                 "solver_status": diagnostics.status,
315                 "solver_success": diagnostics.success,
316                 "solver_message": diagnostics.message,
317                 "solver_objective": diagnostics.objective,
318                 "solver_dual_bound": diagnostics.dual_bound,
319                 "solver_mip_gap": diagnostics.mip_gap,
320                 "solver_node_count": diagnostics.node_count,
321                 "solver_runtime_seconds": diagnostics.runtime_seconds,
322                 "solver_variable_count": diagnostics.variable_count,
323                 "solver_binary_variable_count": diagnostics.binary_variable_count,
324                 "solver_constraint_count": diagnostics.constraint_count,
325                 "solver_certified_optimal": solver_diagnostics.prove_optimality(
326                     diagnostics
327                 ),
328             }
329         )
330     return pd.DataFrame(rows)
331
332
333 def _attempt_frame(result, regime: str) -> pd.DataFrame:
334     labels = result.terminal_labels
335     rows = []
336     for attempt_index, attempt in enumerate(result.attempted_extensions, start=1):
337         support_labels = (
338             tuple(labels[index] for index in attempt.support)
339             if attempt.support is not None
340             else ()
341         )
342         diagnostics = attempt.diagnostics
343         rows.append(
344             {
345                 "regime": regime,

```

```

346         "attempt_index": attempt_index,
347         "candidate_support": ", ".join(map(str, support_labels)),
348         "candidate_support_size": len(support_labels),
349         "candidate_r_value": (
350             float(attempt.r_values[-1]) if attempt.r_values.size else np.nan
351         ),
352         "candidate_x_value": (
353             float(attempt.x_values[-1]) if attempt.x_values.size else np.nan
354         ),
355         "objective": attempt.objective,
356         "status": diagnostics.status,
357         "success": diagnostics.success,
358         "message": diagnostics.message,
359         "dual_bound": diagnostics.dual_bound,
360         "mip_gap": diagnostics.mip_gap,
361         "node_count": diagnostics.node_count,
362         "runtime_seconds": diagnostics.runtime_seconds,
363         "variable_count": diagnostics.variable_count,
364         "binary_variable_count": diagnostics.binary_variable_count,
365         "constraint_count": diagnostics.constraint_count,
366         "solver_certified_optimal": solver_diagnostics_prove_optimality(
367             diagnostics
368         ),
369     }
370 )
371 return pd.DataFrame(rows)
372
373
374 def _atom_frame(result, regime: str) -> pd.DataFrame:
375     return pd.DataFrame(
376         [
377             {
378                 "regime": regime,
379                 "atom_index": atom_index,
380                 "support_labels": ", ".join(map(str, labels)),
381                 "support_indices": ", ".join(map(str, indices)),
382                 "support_size": len(indices),
383                 "r_value": float(r_value),
384                 "x_value": float(x_value),
385                 "is_nontrivial_clade": 1 < len(indices) < len(result.terminal_labels),
386                 "is_singleton": len(indices) == 1,
387                 "is_full_support": len(indices) == len(result.terminal_labels),
388             }
389             for atom_index, (indices, labels, r_value, x_value) in enumerate(
390                 zip(
391                     result.support_indices,
392                     result.support_labels,
393                     result.r_values,
394                     result.x_values,
395                     strict=True,
396                 ),
397                 start=1,
398             )
399         ]
400     )
401
402
403 def run(

```

```

404     output_dir: str | Path = DEFAULT_OUTPUT,
405     *,
406     seed: int = 20260902,
407     time_count: int = 72,
408     train_count: int = 48,
409     validation_count: int = 12,
410     max_atoms: int = 11,
411     outlier_fraction: float = 0.05,
412     time_limit: float | None = 120.0,
413 ) -> dict:
414     """Execute both small6 regimes and persist all auditable results."""
415
416     if time_count <= 0 or train_count <= 0 or validation_count <= 0:
417         raise ValueError("time and split counts must be positive")
418     test_count = time_count - train_count - validation_count
419     if test_count <= 0:
420         raise ValueError("the split must leave a nonempty test suffix")
421     if not 0.0 < outlier_fraction < 1.0:
422         raise ValueError("outlier_fraction must lie strictly between zero and one")
423
424     out_dir = ensure_dir(output_dir)
425     net = build_small_terminal_lv_case()
426     terminals = _terminal_labels(net)
427     r_true, x_true = build_reduced_sensitivity_matrices(
428         net,
429         terminals,
430         voltage_model="squared-voltage",
431     )
432     p, q = _continuous_pq(net, time_count, seed)
433     exact_target = pd.DataFrame(
434         p.to_numpy(dtype=float) @ r_true.T + q.to_numpy(dtype=float) @ x_true.T,
435         index=p.index,
436         columns=terminals,
437     )
438     targets, noise_metadata, full_outlier_mask = _response_regimes(
439         exact_target,
440         seed=seed + 1,
441         outlier_fraction=outlier_fraction,
442     )
443
444     train_slice = slice(0, train_count)
445     validation_slice = slice(train_count, train_count + validation_count)
446     test_slice = slice(train_count + validation_count, time_count)
447     split_metadata = {
448         "same_continuous_scenario": True,
449         "train_time_indices": [0, train_count - 1],
450         "validation_time_indices": [train_count, train_count + validation_count - 1],
451         "test_time_indices": [train_count + validation_count, time_count - 1],
452         "train_count": train_count,
453         "validation_count": validation_count,
454         "test_count": test_count,
455     }
456     true_clades = set(rooted_clades(net.closed_edges(), net.root_bus, terminals))
457
458     labels = pd.Index(terminals, name="terminal")
459     pd.DataFrame(r_true, index=labels, columns=labels).to_csv(out_dir / "R_true.csv")
460     pd.DataFrame(x_true, index=labels, columns=labels).to_csv(out_dir / "X_true.csv")
461     p.to_csv(out_dir / "P_continuous_pu.csv")

```

```

462 q.to_csv(out_dir / "Q_continuous_pu.csv")
463 exact_target.to_csv(out_dir / "squared_voltage_drop_exact_pu2.csv")
464 _truth_atom_frame(net, terminals).to_csv(out_dir / "truth_atoms.csv", index=False)
465
466 regime_metrics: list[dict] = []
467 path_frames: list[pd.DataFrame] = []
468 attempt_frames: list[pd.DataFrame] = []
469 atom_frames: list[pd.DataFrame] = []
470 regime_summaries: dict[str, dict] = {}
471
472 for regime, target in targets.items():
473     regime_dir = ensure_dir(out_dir / regime)
474     target.to_csv(regime_dir / "squared_voltage_drop_observed_pu2.csv")
475     train = _scenario_slice(p, q, target, train_slice)
476     validation = _scenario_slice(p, q, target, validation_slice)
477     test = _scenario_slice(p, q, target, test_slice)
478
479     started = perf_counter()
480     result = fit_laminar_l1_sensitivity(
481         train,
482         validation_scenarios=validation,
483         max_atoms=max_atoms,
484         r_upper_bound=1.0,
485         x_upper_bound=1.0,
486         max_bound_expansions=0,
487         bound_active_fraction=0.999,
488         improvement_abs_tol=1e-12,
489         improvement_rel_tol=1e-9,
490         zero_tolerance=1e-10,
491         validation_blocks=4,
492         time_limit=time_limit,
493         mip_rel_gap=0.0,
494         presolve=True,
495         disp=False,
496     )
497     fit_wall_seconds = perf_counter() - started
498
499     estimated_r = result.r_matrix
500     estimated_x = result.x_matrix
501     # Preserve the exact intercept returned by the selected training LP.
502     # Recomputing a median is mathematically equivalent in generic cases,
503     # but can choose a different point when the L1 intercept is non-unique.
504     training_intercept = result.intercepts[0].copy()
505     test_mask = (
506         full_outlier_mask[test_slice]
507         if regime == "laplace_outliers"
508         else np.zeros((test_count, len(terminals)), dtype=bool)
509     )
510     test_residuals, test_mae = _heldout_residual_frame(
511         test,
512         estimated_r,
513         estimated_x,
514         training_intercept,
515         test_mask,
516     )
517     test_refit_mae, test_refit_se, test_block_mae = evaluate_l1_matrices(
518         test,
519         estimated_r,

```

```

520         estimated_x,
521         blocks_per_scenario=4,
522     )
523     predicted_clades = {
524         frozenset(int(node) for node in support)
525         for support in result.support_labels
526         if 1 < len(support) < len(terminals)
527     }
528     metrics = {
529         "regime": regime,
530         "selected_path_index": result.selected_path_index,
531         "selected_atom_count": len(result.support_labels),
532         "path_length": len(result.path),
533         "attempted_extension_count": len(result.attempted_extensions),
534         "stop_reason": result.stop_reason,
535         "train_mae": result.train_mae,
536         "validation_mae_fixed_training_intercept": result.validation_mae,
537         "test_mae_fixed_training_intercept": test_mae,
538         "test_mae_refit_intercept": test_refit_mae,
539         "test_mae_refit_intercept_se": test_refit_se,
540         "fit_wall_seconds": fit_wall_seconds,
541         "all_extension_attempts_certified_optimal": all(
542             solver_diagnostics_prove_optimality(attempt.diagnostics)
543             for attempt in result.attempted_extensions
544         ),
545         "maximum_extension_absolute_gap": max(
546             (
547                 max(
548                     0.0,
549                     float(attempt.diagnostics.objective)
550                     - float(attempt.diagnostics.dual_bound),
551                 )
552                 for attempt in result.attempted_extensions
553                 if attempt.diagnostics.objective is not None
554                 and attempt.diagnostics.dual_bound is not None
555             ),
556             default=0.0,
557         ),
558         "maximum_extension_mip_gap": max(
559             (
560                 float(attempt.diagnostics.mip_gap)
561                 for attempt in result.attempted_extensions
562                 if attempt.diagnostics.mip_gap is not None
563             ),
564             default=0.0,
565         ),
566         "r_upper_bound_final": result.r_upper_bound,
567         "x_upper_bound_final": result.x_upper_bound,
568         "bound_expansions": result.bound_expansions,
569         **_matrix_errors(estimated_r, r_true, "R"),
570         **_matrix_errors(estimated_x, x_true, "X"),
571         **_score_clades(predicted_clades, true_clades),
572     }
573     regime_metrics.append(metrics)
574
575     pd.DataFrame(estimated_r, index=labels, columns=labels).to_csv(
576         regime_dir / "R_estimated.csv"
577     )

```

```

578     pd.DataFrame(estimated_x, index=labels, columns=labels).to_csv(
579         regime_dir / "X_estimated.csv"
580     )
581     pd.Series(training_intercept, index=labels, name="intercept_pu2").to_csv(
582         regime_dir / "training_intercept.csv"
583     )
584     test_residuals.to_csv(regime_dir / "test_residuals.csv", index=False)
585     pd.DataFrame(
586         {
587             "block_index": np.arange(len(test_block_mae)),
588             "test_mae_refit_intercept": test_block_mae,
589         }
590     ).to_csv(regime_dir / "test_block_mae.csv", index=False)
591     regime_path = _path_frame(result, regime)
592     regime_attempts = _attempt_frame(result, regime)
593     regime_atoms = _atom_frame(result, regime)
594     regime_path.to_csv(regime_dir / "path.csv", index=False)
595     regime_attempts.to_csv(regime_dir / "extension_attempts.csv", index=False)
596     regime_atoms.to_csv(regime_dir / "selected_atoms.csv", index=False)
597     write_json(regime_dir / "metrics.json", metrics)
598     write_json(regime_dir / "fit_summary.json", result.summary())
599     path_frames.append(regime_path)
600     attempt_frames.append(regime_attempts)
601     atom_frames.append(regime_atoms)
602     regime_summaries[regime] = {
603         "fit": result.summary(),
604         "metrics": metrics,
605     }
606
607     summary_frame = pd.DataFrame(regime_metrics)
608     summary_frame.to_csv(out_dir / "summary.csv", index=False)
609     pd.concat(path_frames, ignore_index=True).to_csv(out_dir / "all_paths.csv", index=False)
610     pd.concat(attempt_frames, ignore_index=True).to_csv(
611         out_dir / "all_extension_attempts.csv", index=False
612     )
613     pd.concat(atom_frames, ignore_index=True).to_csv(
614         out_dir / "all_selected_atoms.csv", index=False
615     )
616     config = {
617         "case": "small_terminal_lv",
618         "terminal_labels": terminals,
619         "terminal_count": len(terminals),
620         "hidden_internal_count": len(net.hidden_buses()),
621         "squared_voltage_truth": True,
622         "seed": seed,
623         "time_count": time_count,
624         "split": split_metadata,
625         "max_atoms": max_atoms,
626         "coefficient_bounds": [1.0, 1.0],
627         "solver": "scipy.optimize.milp/HiGHS",
628         "mip_rel_gap": 0.0,
629         "time_limit_seconds_per_solve": time_limit,
630         "noise": noise_metadata,
631     }
632     output = {
633         "config": config,
634         "truth_nontrivial_clades": [
635             list(item)

```

```

636         for item in sorted(true_clades, key=lambda value: (len(value), tuple(value)))
637     ],
638     "regimes": regime_summaries,
639 }
640 write_json(out_dir / "metrics.json", output)
641 report_lines = [
642     "# Exact laminar L1-MILP small6 experiment",
643     "",
644     "One continuous 72-point P/Q scenario is split without shuffling into "
645     f"train/validation/test blocks of {train_count}/{validation_count}/{test_count} points.",
646     "The truth uses squared-voltage R/X matrices. The robust regime adds "
647     "Laplace response noise and sparse gross outliers before splitting.",
648     "Coefficient bounds are fixed at (1, 1), and every accepted extension "
649     "must pass the strict primal/dual-bound or relative-gap certificate "
650     "with zero requested relative MIP gap.",
651     "",
652     "``text",
653     summary_frame.to_string(index=False),
654     "``",
655     "",
656     "`test_mae_fixed_training_intercept` is the primary held-out metric. "
657     "The refitted-intercept value is exported only as a nuisance-intercept diagnostic.",
658 ]
659 (out_dir / "report.md").write_text("\n".join(report_lines) + "\n", encoding="utf-8")
660 return output
661
662
663 def main() -> None:
664     parser = argparse.ArgumentParser()
665     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
666     parser.add_argument("--seed", type=int, default=20260902)
667     parser.add_argument("--time-count", type=int, default=72)
668     parser.add_argument("--train-count", type=int, default=48)
669     parser.add_argument("--validation-count", type=int, default=12)
670     parser.add_argument("--max-atoms", type=int, default=11)
671     parser.add_argument("--outlier-fraction", type=float, default=0.05)
672     parser.add_argument("--time-limit", type=float, default=120.0)
673     args = parser.parse_args()
674     run(
675         args.output,
676         seed=args.seed,
677         time_count=args.time_count,
678         train_count=args.train_count,
679         validation_count=args.validation_count,
680         max_atoms=args.max_atoms,
681         outlier_fraction=args.outlier_fraction,
682         time_limit=args.time_limit,
683     )
684
685
686 if __name__ == "__main__":
687     main()

```

Listing 4: experiments/run_laminar_l1_milp.py

D case33 集成算例驱动完整源码

```

1  """Run auditable laminar L1-MILP integration cases derived from IEEE case33.
2
3  Two terminal-load-only variants are exercised:
4
5  * ``aggregate_leaves`` keeps the four physical leaves as terminals. Its small
6    MILPs are solved along a multi-atom forward path and serve as an end-to-end
7    exact-status integration case.
8  * ``hybrid_full`` uses the standard hybrid terminalization with all 32 loads
9    observed at leaves. By default only one extension is attempted under a
10   finite time limit, making the combinatorial scaling limit explicit.
11
12 The data-generating network is used only to synthesize the response and to
13 compute post-fit diagnostics. It is never used to construct candidate
14 supports: every extension MILP searches all nonempty terminal subsets that are
15 laminar with the already selected supports.
16
17 Validation and test metrics keep the scenario/output intercept estimated on
18 the training prefix fixed. The core helper that refits a validation intercept
19 is deliberately not used for model selection in this experiment.
20 """
21
22 from __future__ import annotations
23
24 import argparse
25 import json
26 from pathlib import Path
27 from time import perf_counter
28
29 import networkx as nx
30 import numpy as np
31 import pandas as pd
32 import scipy
33
34 from terminal_case33.data.case33bw_raw import load_raw_case33bw
35 from terminal_case33.estimation.laminar_l1_milp import (
36     LaminarL1PathPoint,
37     fit_laminar_l1_sensitivity,
38     is_laminar_family,
39     solver_diagnostics_prove_optimality,
40 )
41 from terminal_case33.models.lin_distflow import build_reduced_sensitivity_matrices
42 from terminal_case33.scenario.terminalize import terminalize_case33
43 from terminal_case33.utils.io import ensure_dir
44
45
46 DEFAULT_OUTPUT = Path("outputs/laminar_l1_milp_case33_integration")
47
48
49 def _write_json(path: Path, payload: dict) -> None:
50     path.write_text(
51         json.dumps(payload, indent=2, ensure_ascii=False, allow_nan=False),
52         encoding="utf-8",
53     )
54
55

```

```

56 def _terminal_labels(net) -> list[int]:
57     return (
58         net.buses.loc[net.buses["bus_type"].eq("observed_terminal"), "bus_id"]
59         .astype(int)
60         .tolist()
61     )
62
63
64 def _ar1_standardized(
65     rng: np.random.Generator,
66     sample_count: int,
67     terminal_count: int,
68     rho: float,
69 ) -> np.ndarray:
70     innovations = rng.normal(size=(sample_count, terminal_count))
71     values = np.zeros_like(innovations)
72     values[0] = innovations[0]
73     innovation_scale = np.sqrt(1.0 - rho**2)
74     for sample_index in range(1, sample_count):
75         values[sample_index] = (
76             rho * values[sample_index - 1]
77             + innovation_scale * innovations[sample_index]
78         )
79     values -= values.mean(axis=0, keepdims=True)
80     values /= np.maximum(values.std(axis=0, keepdims=True), 1e-12)
81     return values
82
83
84 def _continuous_pq(net, sample_count: int, seed: int) -> tuple[pd.DataFrame, pd.DataFrame]:
85     """Generate identifiable smooth P/Q excitation without consulting topology."""
86
87     terminals = _terminal_labels(net)
88     terminal_frame = net.buses.set_index("bus_id").loc[terminals]
89     base_power_kw = float(net.base_mva) * 1000.0
90     p_nominal = terminal_frame["pd_kw"].to_numpy(dtype=float) / base_power_kw
91     q_nominal = terminal_frame["qd_kvar"].to_numpy(dtype=float) / base_power_kw
92     rng = np.random.default_rng(seed)
93     p_local = _ar1_standardized(rng, sample_count, len(terminals), rho=0.71)
94     q_local = _ar1_standardized(rng, sample_count, len(terminals), rho=0.63)
95     time = np.arange(sample_count, dtype=float)
96     phases = np.linspace(0.0, 2.0 * np.pi, len(terminals), endpoint=False)
97     common_p = np.sin(2.0 * np.pi * time / max(sample_count, 2))
98     common_q = np.cos(2.0 * np.pi * time / max(sample_count, 2) + 0.31)
99     p_wave = np.sin(6.0 * np.pi * time[:, None] / max(sample_count, 2) + phases)
100    q_wave = np.cos(8.0 * np.pi * time[:, None] / max(sample_count, 2) + 0.73 * phases)
101    p_multiplier = 1.0 + 0.20 * common_p[:, None] + 0.27 * p_local + 0.08 * p_wave
102    q_multiplier = 1.0 + 0.18 * common_q[:, None] + 0.31 * q_local + 0.10 * q_wave
103    p = np.maximum(0.10 * p_nominal[None, :], p_nominal[None, :] * p_multiplier)
104    q = np.maximum(0.10 * q_nominal[None, :], q_nominal[None, :] * q_multiplier)
105    index = pd.RangeIndex(sample_count, name="time_index")
106    return (
107        pd.DataFrame(p, index=index, columns=terminals),
108        pd.DataFrame(q, index=index, columns=terminals),
109    )
110
111
112 def _scenario(
113     p: pd.DataFrame,

```

```

114     q: pd.DataFrame,
115     target: pd.DataFrame,
116     selected: slice,
117     name: str,
118 ) -> list[dict]:
119     return [
120         {
121             "name": name,
122             "P_terminal": p.iloc[selected].copy(),
123             "Q_terminal": q.iloc[selected].copy(),
124             "drop_target": target.iloc[selected].copy(),
125         }
126     ]
127
128
129 def _training_intercepts(
130     training: list[dict], r_matrix: np.ndarray, x_matrix: np.ndarray
131 ) -> tuple[np.ndarray, ...]:
132     """Return one L1-optimal output intercept vector per training scenario."""
133
134     fitted: list[np.ndarray] = []
135     for scenario in training:
136         p = scenario["P_terminal"].to_numpy(dtype=float)
137         q = scenario["Q_terminal"].to_numpy(dtype=float)
138         target = scenario["drop_target"].to_numpy(dtype=float)
139         fitted.append(np.median(target - p @ r_matrix.T - q @ x_matrix.T, axis=0))
140     return tuple(fitted)
141
142
143 def _fixed_intercept_score(
144     scenarios: list[dict],
145     r_matrix: np.ndarray,
146     x_matrix: np.ndarray,
147     intercepts: tuple[np.ndarray, ...],
148     *,
149     blocks_per_scenario: int,
150 ) -> tuple[float, float, np.ndarray]:
151     """Score held-out observations without using their targets to refit intercepts."""
152
153     if len(scenarios) != len(intercepts):
154         raise ValueError("held-out scenarios and training intercepts must align")
155     absolute_parts: list[np.ndarray] = []
156     block_means: list[float] = []
157     for scenario, intercept in zip(scenarios, intercepts, strict=True):
158         p = scenario["P_terminal"].to_numpy(dtype=float)
159         q = scenario["Q_terminal"].to_numpy(dtype=float)
160         target = scenario["drop_target"].to_numpy(dtype=float)
161         prediction = p @ r_matrix.T + q @ x_matrix.T + intercept[None, :]
162         absolute = np.abs(target - prediction)
163         absolute_parts.append(absolute.reshape(-1))
164         for indices in np.array_split(np.arange(len(absolute)), blocks_per_scenario):
165             if indices.size:
166                 block_means.append(float(np.mean(absolute[indices, :])))
167     block_array = np.asarray(block_means, dtype=float)
168     standard_error = (
169         float(np.std(block_array, ddof=1) / np.sqrt(len(block_array)))
170         if len(block_array) > 1
171         else 0.0

```

```

172 )
173 return float(np.mean(np.concatenate(absolute_parts))), standard_error, block_array
174
175
176 def _fixed_intercept_path(
177     path: tuple[LaminarL1PathPoint, ...],
178     training: list[dict],
179     validation: list[dict],
180     *,
181     validation_blocks: int,
182 ) -> tuple[list[dict], int, tuple[np.ndarray, ...]]:
183     rows: list[dict] = []
184     intercepts_by_path: list[tuple[np.ndarray, ...]] = []
185     for path_index, point in enumerate(path):
186         intercepts = _training_intercepts(training, point.r_matrix, point.x_matrix)
187         validation_mae, validation_se, _ = _fixed_intercept_score(
188             validation,
189             point.r_matrix,
190             point.x_matrix,
191             intercepts,
192             blocks_per_scenario=validation_blocks,
193         )
194         intercepts_by_path.append(intercepts)
195         rows.append(
196             {
197                 "path_index": path_index,
198                 "algorithm_iteration": point.iteration,
199                 "atom_count": len(point.support_indices),
200                 "supports": ";".join(
201                     ",".join(map(str, support)) for support in point.support_labels
202                 ),
203                 "train_mae": float(point.train_mae),
204                 "validation_mae_fixed_training_intercept": validation_mae,
205                 "validation_se_fixed_training_intercept": validation_se,
206                 "accepted_gain": (
207                     None if point.accepted_gain is None else float(point.accepted_gain)
208                 ),
209                 "solver_status": int(point.solver.status),
210                 "solver_success": bool(point.solver.success),
211                 "solver_objective": point.solver.objective,
212                 "solver_dual_bound": point.solver.dual_bound,
213                 "solver_mip_gap": point.solver.mip_gap,
214                 "solver_node_count": point.solver.node_count,
215                 "solver_runtime_seconds": float(point.solver.runtime_seconds),
216             }
217         )
218     validation_means = np.asarray(
219         [row["validation_mae_fixed_training_intercept"] for row in rows], dtype=float
220     )
221     best = int(np.argmin(validation_means))
222     threshold = validation_means[best] + float(
223         rows[best]["validation_se_fixed_training_intercept"]
224     )
225     selected = int(np.flatnonzero(validation_means <= threshold)[0])
226     for index, row in enumerate(rows):
227         row["selected_by_fixed_intercept_one_se"] = index == selected
228     return rows, selected, intercepts_by_path[selected]
229

```

```

230
231 def _truth_atoms(net, terminals: list[int]) -> pd.DataFrame:
232     """Aggregate physical edges sharing one downstream terminal support."""
233
234     graph = net.to_networkx_graph()
235     oriented = nx.bfs_tree(graph, net.root_bus)
236     terminal_set = set(terminals)
237     terminal_position = {terminal: index for index, terminal in enumerate(terminals)}
238     z_base = float(net.base_kv) ** 2 / float(net.base_mva)
239     grouped: dict[tuple[int, ...], dict] = {}
240     for parent, child in oriented.edges():
241         descendants = {int(child), *(int(node) for node in nx.descendants(oriented, child))}
242         support = tuple(sorted(descendants & terminal_set))
243         if not support:
244             continue
245         edge = graph.edges[parent, child]
246         record = grouped.setdefault(
247             support,
248             {
249                 "support_labels": ", ".join(map(str, support)),
250                 "support_indices": ", ".join(str(terminal_position[node]) for node in support),
251                 "support_size": len(support),
252                 "r_value": 0.0,
253                 "x_value": 0.0,
254                 "physical_edges": [],
255             },
256         )
257         record["r_value"] += 2.0 * float(edge["r_ohm"]) / z_base
258         record["x_value"] += 2.0 * float(edge["x_ohm"]) / z_base
259         record["physical_edges"].append(f"{int(parent)}--{int(child)}")
260     rows: list[dict] = []
261     for record in grouped.values():
262         copied = dict(record)
263         copied["physical_edges"] = ";".join(copied["physical_edges"])
264         rows.append(copied)
265     return pd.DataFrame(rows).sort_values(
266         ["support_size", "support_labels"], ignore_index=True
267     )
268
269
270 def _family_score(predicted: set[frozenset[int]], truth: set[frozenset[int]]) -> dict:
271     matched = predicted & truth
272     precision = len(matched) / len(predicted) if predicted else float(not truth)
273     recall = len(matched) / len(truth) if truth else float(not predicted)
274     f1 = 2.0 * precision * recall / (precision + recall) if precision + recall else 0.0
275
276     def serialize(family: set[frozenset[int]]) -> list[list[int]]:
277         return [
278             sorted(item)
279             for item in sorted(family, key=lambda value: (len(value), tuple(sorted(value))))
280         ]
281
282     return {
283         "predicted_nontrivial_support_count": len(predicted),
284         "true_nontrivial_support_count": len(truth),
285         "matched_nontrivial_support_count": len(matched),
286         "nontrivial_support_precision": precision,
287         "nontrivial_support_recall": recall,

```

```

288     "nontrivial_support_f1": f1,
289     "matched_nontrivial_supports": serialize(matched),
290     "missing_nontrivial_supports": serialize(truth - predicted),
291     "extra_nontrivial_supports": serialize(predicted - truth),
292 }
293
294
295 def _matrix_error(estimate: np.ndarray, truth: np.ndarray, prefix: str) -> dict:
296     difference = np.asarray(estimate, dtype=float) - np.asarray(truth, dtype=float)
297     return {
298         f"{prefix}_matrix_mae": float(np.mean(np.abs(difference))),
299         f"{prefix}_matrix_max_abs_error": float(np.max(np.abs(difference))),
300         f"{prefix}_matrix_relative_frobenius_error": float(
301             np.linalg.norm(difference, ord="fro")
302             / max(np.linalg.norm(truth, ord="fro"), 1e-15)
303         ),
304     }
305
306
307 def _attempt_rows(result) -> list[dict]:
308     rows: list[dict] = []
309     labels = result.terminal_labels
310     for attempt_index, attempt in enumerate(result.attempted_extensions, start=1):
311         support_labels = (
312             tuple(labels[index] for index in attempt.support)
313             if attempt.support is not None
314             else ()
315         )
316         diagnostics = attempt.diagnostics
317         rows.append(
318             {
319                 "attempt_index": attempt_index,
320                 "incumbent_support": ",".join(map(str, support_labels)),
321                 "incumbent_support_size": len(support_labels),
322                 "incumbent_was_accepted": any(
323                     tuple(support_labels) == tuple(path_support)
324                     for point in result.path[1:]
325                     for path_support in point.support_labels
326                 ),
327                 "objective": attempt.objective,
328                 "status": int(diagnostics.status),
329                 "success": bool(diagnostics.success),
330                 "message": diagnostics.message,
331                 "dual_bound": diagnostics.dual_bound,
332                 "mip_gap": diagnostics.mip_gap,
333                 "node_count": diagnostics.node_count,
334                 "runtime_seconds": float(diagnostics.runtime_seconds),
335                 "variable_count": int(diagnostics.variable_count),
336                 "binary_variable_count": int(diagnostics.binary_variable_count),
337                 "constraint_count": int(diagnostics.constraint_count),
338                 "solver_certified_optimal": solver_diagnostics_prove_optimality(
339                     diagnostics
340                 ),
341             }
342         )
343     return rows
344
345

```

```

346 def _run_one_case(
347     output_dir: Path,
348     *,
349     case_name: str,
350     terminalization_mode: str,
351     seed: int,
352     sample_count: int,
353     train_count: int,
354     validation_count: int,
355     max_atoms: int,
356     coefficient_bound: float,
357     time_limit: float,
358 ) -> dict:
359     test_count = sample_count - train_count - validation_count
360     if min(train_count, validation_count, test_count) <= 0:
361         raise ValueError("train/validation/test blocks must all be nonempty")
362
363     case_dir = ensure_dir(output_dir / case_name)
364     raw = load_raw_case33bw(include_tie_lines=False)
365     net = terminalize_case33(
366         raw,
367         mode=terminalization_mode,
368         service_impedance_mode="scaled_original",
369         seed=seed,
370     )
371     terminals = _terminal_labels(net)
372     r_true, x_true = build_reduced_sensitivity_matrices(
373         net,
374         terminals,
375         voltage_model="squared-voltage",
376     )
377     p, q = _continuous_pq(net, sample_count, seed)
378     target = pd.DataFrame(
379         p.to_numpy(dtype=float) @ r_true.T + q.to_numpy(dtype=float) @ x_true.T,
380         index=p.index,
381         columns=terminals,
382     )
383
384     train_slice = slice(0, train_count)
385     validation_slice = slice(train_count, train_count + validation_count)
386     test_slice = slice(train_count + validation_count, sample_count)
387     training = _scenario(p, q, target, train_slice, f"{case_name}_train")
388     validation = _scenario(p, q, target, validation_slice, f"{case_name}_validation")
389     test = _scenario(p, q, target, test_slice, f"{case_name}_test")
390
391     # No true support or true coefficient is passed below. Binary z variables
392     # search the complete nonempty subset space, with only learned laminar
393     # compatibility constraints added after an atom has been accepted.
394     started = perf_counter()
395     result = fit_laminar_l1_sensitivity(
396         training,
397         validation_scenarios=None,
398         max_atoms=max_atoms,
399         r_upper_bound=coefficient_bound,
400         x_upper_bound=coefficient_bound,
401         max_bound_expansions=0,
402         bound_active_fraction=0.999,
403         improvement_abs_tol=1e-12,

```

```

404         improvement_rel_tol=1e-9,
405         zero_tolerance=1e-10,
406         time_limit=time_limit,
407         mip_rel_gap=0.0,
408         presolve=True,
409         disp=False,
410     )
411     fit_wall_seconds = perf_counter() - started
412
413     path_rows, selected_path_index, selected_intercepts = _fixed_intercept_path(
414         result.path,
415         training,
416         validation,
417         validation_blocks=min(4, validation_count),
418     )
419     selected = result.path[selected_path_index]
420     test_mae, test_se, test_blocks = _fixed_intercept_score(
421         test,
422         selected.r_matrix,
423         selected.x_matrix,
424         selected_intercepts,
425         blocks_per_scenario=min(4, test_count),
426     )
427     attempt_rows = _attempt_rows(result)
428     truth_atoms = _truth_atoms(net, terminals)
429     true_nontrivial = {
430         frozenset(map(int, str(row.support_labels).split(",")))
431         for row in truth_atoms.itertuples(index=False)
432         if 1 < int(row.support_size) < len(terminals)
433     }
434     predicted_nontrivial = {
435         frozenset(map(int, support))
436         for support in selected.support_labels
437         if 1 < len(support) < len(terminals)
438     }
439     reported_gaps = [
440         float(row["mip_gap"]) for row in attempt_rows if row["mip_gap"] is not None
441     ]
442     attempted_statuses = [int(row["status"]) for row in attempt_rows]
443     selected_r = np.asarray(selected.r_values, dtype=float)
444     selected_x = np.asarray(selected.x_values, dtype=float)
445     coefficient_bound_active = bool(
446         (selected_r.size and np.max(selected_r) >= 0.999 * coefficient_bound)
447         or (selected_x.size and np.max(selected_x) >= 0.999 * coefficient_bound)
448     )
449     metrics = {
450         "case": case_name,
451         "terminalization_mode": terminalization_mode,
452         "terminal_count": len(terminals),
453         "terminal_labels": terminals,
454         "sample_count": sample_count,
455         "train_count": train_count,
456         "validation_count": validation_count,
457         "test_count": test_count,
458         "max_atoms_requested": max_atoms,
459         "accepted_path_length_excluding_zero": len(result.path) - 1,
460         "selected_path_index_fixed_intercept_one_se": selected_path_index,
461         "selected_atom_count": len(selected.support_indices),

```

```

462     "selected_supports": [list(item) for item in selected.support_labels],
463     "train_mae": float(selected.train_mae),
464     "validation_mae_fixed_training_intercept": float(
465         path_rows[selected_path_index]["validation_mae_fixed_training_intercept"]
466     ),
467     "validation_se_fixed_training_intercept": float(
468         path_rows[selected_path_index]["validation_se_fixed_training_intercept"]
469     ),
470     "test_mae_fixed_training_intercept": test_mae,
471     "test_se_fixed_training_intercept": test_se,
472     "stop_reason": result.stop_reason,
473     "attempt_count": len(attempt_rows),
474     "attempt_statuses": attempted_statuses,
475     "all_attempts_status_zero": bool(attempt_rows)
476     and all(status == 0 for status in attempted_statuses),
477     "all_attempts_certified_optimal": bool(attempt_rows)
478     and all(bool(row["solver_certified_optimal"]) for row in attempt_rows),
479     "maximum_reported_mip_gap": max(reported_gaps, default=None),
480     "fit_wall_seconds": fit_wall_seconds,
481     "maximum_extension_runtime_seconds": max(
482         (float(row["runtime_seconds"]) for row in attempt_rows), default=0.0
483     ),
484     "coefficient_bound": coefficient_bound,
485     "coefficient_bound_active_in_selected_model": coefficient_bound_active,
486     "truth_used_for_candidate_generation": False,
487     "candidate_generation": (
488         "implicit exhaustive search over all nonempty terminal subsets; "
489         "laminar compatibility only with already selected supports"
490     ),
491     "validation_intercept_policy": "fixed training L1-median intercept",
492     "exactness_scope": (
493         "each extension with HiGHS/SciPy status 0 is globally optimal for the "
494         "bounded one-atom MILP up to solver numerical tolerances; the forward "
495         "multi-atom path remains greedy"
496     ),
497     "global_k_atom_optimum_claimed": False,
498     "selected_family_is_laminar": is_laminar_family(selected.support_indices),
499     "selected_r_min_eigenvalue": float(np.min(np.linalg.eigvalsh(selected.r_matrix))),
500     "selected_x_min_eigenvalue": float(np.min(np.linalg.eigvalsh(selected.x_matrix))),
501     **_matrix_error(selected.r_matrix, r_true, "R"),
502     **_matrix_error(selected.x_matrix, x_true, "X"),
503     **_family_score(predicted_nontrivial, true_nontrivial),
504 }
505
506 labels = pd.Index(terminals, name="terminal")
507 pd.DataFrame(r_true, index=labels, columns=labels).to_csv(case_dir / "R_true.csv")
508 pd.DataFrame(x_true, index=labels, columns=labels).to_csv(case_dir / "X_true.csv")
509 pd.DataFrame(selected.r_matrix, index=labels, columns=labels).to_csv(
510     case_dir / "R_selected.csv"
511 )
512 pd.DataFrame(selected.x_matrix, index=labels, columns=labels).to_csv(
513     case_dir / "X_selected.csv"
514 )
515 p.to_csv(case_dir / "P_pu.csv")
516 q.to_csv(case_dir / "Q_pu.csv")
517 target.to_csv(case_dir / "squared_voltage_drop_pu2.csv")
518 split_frame = pd.DataFrame(
519     {

```

```

520         "time_index": np.arange(sample_count),
521         "split": (
522             ["train"] * train_count
523             + ["validation"] * validation_count
524             + ["test"] * test_count
525         ),
526     }
527 )
528 split_frame.to_csv(case_dir / "split.csv", index=False)
529 truth_atoms.to_csv(case_dir / "truth_atoms_postfit_only.csv", index=False)
530 pd.DataFrame(path_rows).to_csv(case_dir / "path.csv", index=False)
531 pd.DataFrame(attempt_rows).to_csv(case_dir / "extension_attempts.csv", index=False)
532 pd.DataFrame(
533     {
534         "block_index": np.arange(len(test_blocks)),
535         "test_mae_fixed_training_intercept": test_blocks,
536     }
537 ).to_csv(case_dir / "test_block_mae.csv", index=False)
538 pd.DataFrame(
539     [
540         {
541             "atom_index": atom_index,
542             "support_labels": ", ".join(map(str, labels_value)),
543             "support_indices": ", ".join(map(str, indices)),
544             "r_value": float(r_value),
545             "x_value": float(x_value),
546         }
547         for atom_index, (indices, labels_value, r_value, x_value) in enumerate(
548             zip(
549                 selected.support_indices,
550                 selected.support_labels,
551                 selected.r_values,
552                 selected.x_values,
553                 strict=True,
554             ),
555             start=1,
556         )
557     ]
558 ).to_csv(case_dir / "selected_atoms.csv", index=False)
559 for scenario_index, intercept in enumerate(selected_intercepts):
560     pd.Series(intercept, index=labels, name="training_intercept_pu2").to_csv(
561         case_dir / f"training_intercept_scenario_{scenario_index}.csv"
562     )
563 _write_json(case_dir / "metrics.json", metrics)
564 return metrics
565
566
567 def run(
568     output_dir: str | Path = DEFAULT_OUTPUT,
569     *,
570     seed: int = 20260903,
571     aggregate_time_limit: float = 30.0,
572     hybrid_time_limit: float = 20.0,
573     coefficient_bound: float = 10.0,
574     skip_hybrid: bool = False,
575 ) -> dict:
576     output = ensure_dir(output_dir)
577     cases = [

```

```

578     _run_one_case(
579         output,
580         case_name="aggregate_leaves",
581         terminalization_mode="aggregate_to_original_leaves",
582         seed=seed,
583         sample_count=72,
584         train_count=48,
585         validation_count=12,
586         max_atoms=7,
587         coefficient_bound=coefficient_bound,
588         time_limit=aggregate_time_limit,
589     )
590 ]
591 if not skip_hybrid:
592     cases.append(
593         _run_one_case(
594             output,
595             case_name="hybrid_full",
596             terminalization_mode="hybrid_leaf",
597             seed=seed + 1,
598             sample_count=12,
599             train_count=8,
600             validation_count=2,
601             max_atoms=1,
602             coefficient_bound=coefficient_bound,
603             time_limit=hybrid_time_limit,
604         )
605     )
606
607 summary = pd.DataFrame(
608     [
609         {
610             "case": metrics["case"],
611             "terminal_count": metrics["terminal_count"],
612             "max_atoms_requested": metrics["max_atoms_requested"],
613             "accepted_atom_count": metrics["accepted_path_length_excluding_zero"],
614             "selected_atom_count": metrics["selected_atom_count"],
615             "stop_reason": metrics["stop_reason"],
616             "attempt_statuses": ", ".join(map(str, metrics["attempt_statuses"])),
617             "all_attempts_status_zero": metrics["all_attempts_status_zero"],
618             "all_attempts_certified_optimal": metrics[
619                 "all_attempts_certified_optimal"
620             ],
621             "maximum_reported_mip_gap": metrics["maximum_reported_mip_gap"],
622             "fit_wall_seconds": metrics["fit_wall_seconds"],
623             "train_mae": metrics["train_mae"],
624             "validation_mae_fixed_training_intercept": metrics[
625                 "validation_mae_fixed_training_intercept"
626             ],
627             "test_mae_fixed_training_intercept": metrics[
628                 "test_mae_fixed_training_intercept"
629             ],
630             "R_matrix_relative_frobenius_error": metrics[
631                 "R_matrix_relative_frobenius_error"
632             ],
633             "X_matrix_relative_frobenius_error": metrics[
634                 "X_matrix_relative_frobenius_error"
635             ],

```

```

636         "nontrivial_support_f1": metrics["nontrivial_support_f1"],
637     }
638     for metrics in cases
639 ]
640 )
641 summary.to_csv(output / "summary.csv", index=False)
642 config = {
643     "seed": seed,
644     "solver": "scipy.optimize.milp/HiGHS",
645     "scipy_version": scipy.__version__,
646     "requested_mip_relative_gap": 0.0,
647     "aggregate_time_limit_seconds_per_solve": aggregate_time_limit,
648     "hybrid_time_limit_seconds_per_solve": hybrid_time_limit,
649     "coefficient_bound_for_both_R_and_X": coefficient_bound,
650     "truth_used_for_candidate_generation": False,
651     "fixed_training_intercept_for_validation_and_test": True,
652     "hybrid_case_skipped": skip_hybrid,
653 }
654 payload = {"config": config, "cases": cases}
655 _write_json(output / "metrics.json", payload)
656 report = [
657     "# Laminar L1-MILP IEEE case33 integration",
658     "",
659     "Both cases use noiseless squared-voltage LinDistFlow responses. The physical "
660     "case is used to generate data and post-fit truth metrics only; the solver "
661     "searches every nonempty subset implicitly and receives no true clade list.",
662     "",
663     "Validation and test MAE use the intercept estimated from the training prefix "
664     "without held-out-target refitting. Status 0 denotes HiGHS/SciPy optimality "
665     "within numerical tolerances for that bounded one-atom MILP. It does not turn "
666     "the greedy multi-atom path into a globally optimal K-atom solution.",
667     "",
668     "```text",
669     summary.to_string(index=False),
670     "```",
671 ]
672 (output / "report.md").write_text("\n".join(report) + "\n", encoding="utf-8")
673 return payload
674
675
676 def main() -> None:
677     parser = argparse.ArgumentParser()
678     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
679     parser.add_argument("--seed", type=int, default=20260903)
680     parser.add_argument("--aggregate-time-limit", type=float, default=30.0)
681     parser.add_argument("--hybrid-time-limit", type=float, default=20.0)
682     parser.add_argument("--coefficient-bound", type=float, default=10.0)
683     parser.add_argument("--skip-hybrid", action="store_true")
684     args = parser.parse_args()
685     run(
686         args.output,
687         seed=args.seed,
688         aggregate_time_limit=args.aggregate_time_limit,
689         hybrid_time_limit=args.hybrid_time_limit,
690         coefficient_bound=args.coefficient_bound,
691         skip_hybrid=args.skip_hybrid,
692     )
693 
```

```
694  
695 if __name__ == "__main__":  
696     main()
```

Listing 5: experiments/run_laminar_l1_case33_integration.py

E 实测结果表生成器完整源码

```

1  """Render audited laminar L1-MILP CSV/JSON results as LaTeX fragments."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  import json
8  from pathlib import Path
9
10
11  DEFAULT_SMALL6 = Path("outputs/laminar_l1_milp_small6_final/summary.csv")
12  DEFAULT_CASE33 = Path("outputs/laminar_l1_milp_case33_bound2_final/metrics.json")
13  DEFAULT_SMALL6_TEX = Path("docs/laminar_l1_milp_results_table.tex")
14  DEFAULT_CASE33_TEX = Path("docs/laminar_l1_case33_results_table.tex")
15
16
17  def _escape(value: object) -> str:
18      text = str(value)
19      replacements = {
20          "\\": r"\textbackslash{}",
21          "&": r"\&",
22          "%": r"\%",
23          "$": r"\$",
24          "#": r"\#",
25          "_": r"\_",
26          "{": r"\{",
27          "}": r"\}",
28      }
29      return "".join(replacements.get(character, character) for character in text)
30
31
32  def _sci(value: object) -> str:
33      return f"{float(value):.3e}"
34
35
36  def _fixed(value: object, digits: int = 3) -> str:
37      return f"{float(value):.{digits}f}"
38
39
40  def _yes_no(value: object) -> str:
41      if isinstance(value, str):
42          checked = value.strip().lower() in {"1", "true", "yes"}
43      else:
44          checked = bool(value)
45      return "是" if checked else "否"
46
47
48  def _stop_reason(value: object) -> str:
49      names = {
50          "no_significant_one_atom_gain": "无显著单原子增益",
51          "laminar_family_limit": "达到 laminar 上限",
52          "extension_not_proven_optimal": "扩展未获最优证书",
53          "max_atoms": "达到原子数上限",
54      }
55      text = str(value)

```

```

56     return names.get(text, _escape(text))
57
58
59 def _read_csv(path: Path) -> list[dict[str, str]]:
60     with path.open("r", encoding="utf-8-sig", newline="") as handle:
61         return list(csv.DictReader(handle))
62
63
64 def render_small6(summary_path: Path, output_path: Path) -> None:
65     rows = _read_csv(summary_path)
66     if not rows:
67         raise ValueError(f"small6 summary is empty: {summary_path}")
68     lines = [
69         r"\begin{table}[htbp]",
70         r"\centering",
71         r"\caption{small6 最终拟合、固定训练截距验证与测试结果}",
72         r"\label{tab:small6-fit}",
73         r"\scriptsize",
74         r"\begin{tabularx}{\textwidth}{lcc>{\raggedright\arraybackslash}Xrrr r}",
75         r"\toprule",
76         r"制度 & 路径点 & 选中原子 & 停止原因 & 训练 MAE & 验证 MAE & 测试 MAE & 总时间/s \\",
77         r"\midrule",
78     ]
79     display_names = {
80         "noiseless": "无噪声",
81         "laplace_outliers": "Laplace+异常值",
82     }
83     for row in rows:
84         lines.append(
85             (r"{ } & { } & { } & { } & { } & { } & { } & { } \\\").format(
86                 display_names.get(row["regime"], _escape(row["regime"])),
87                 int(row["selected_path_index"]),
88                 int(row["selected_atom_count"]),
89                 _stop_reason(row["stop_reason"]),
90                 _sci(row["train_mae"]),
91                 _sci(row["validation_mae_fixed_training_intercept"]),
92                 _sci(row["test_mae_fixed_training_intercept"]),
93                 _fixed(row["fit_wall_seconds"], 2),
94             )
95         )
96     lines.extend(
97         [
98             r"\bottomrule",
99             r"\end{tabularx}",
100             r"\end{table}",
101             "",
102             r"\begin{table}[htbp]",
103             r"\centering",
104             r"\caption{small6 矩阵、clade 与求解证书审计}",
105             r"\label{tab:small6-structure}",
106             r"\scriptsize",
107             r"\begin{tabular}{lrrrrrrr}",
108             r"\toprule",
109             r"制度 & $$ 相对 Fro. & $$ 相对 Fro. & 匹配/真值 & Precision & Recall & F1 & 全部扩展有证书 \\",
110             r"\midrule",
111         ]
112     )
113     for row in rows:

```

```

114         lines.append(
115             (r"{} & {} & {} & {}/{ } & {} & {} & {} & {} \\").format(
116                 display_names.get(row["regime"], _escape(row["regime"])),
117                 _sci(row["R_matrix_relative_frobenius_error"]),
118                 _sci(row["X_matrix_relative_frobenius_error"]),
119                 int(row["matched_nontrivial_clade_count"]),
120                 int(row["true_nontrivial_clade_count"]),
121                 _fixed(row["nontrivial_clade_precision"]),
122                 _fixed(row["nontrivial_clade_recall"]),
123                 _fixed(row["nontrivial_clade_f1"]),
124                 _yes_no(row["all_extension_attempts_certified_optimal"]),
125             )
126         )
127     maximum_absolute_gap = max(
128         float(row["maximum_extension_absolute_gap"]) for row in rows
129     )
130     maximum_relative_gap = max(
131         float(row["maximum_extension_mip_gap"]) for row in rows
132     )
133     lines.extend(
134         [
135             r"\bottomrule",
136             r"\end{tabular}",
137             (
138                 r"\par\smallskip 最大报告绝对 gap 为 "
139                 + _sci(maximum_absolute_gap)
140                 + r", 最大相对 MIP gap 为 "
141                 + _sci(maximum_relative_gap)
142                 + r"; 两个制度均未触及系数上界。"
143             ),
144             r"\end{table}",
145             "",
146             r"\FloatBarrier",
147             (
148                 r"\noindent\textit{数据来源: }\path{"
149                 + summary_path.as_posix()
150                 + r"}。验证和首要测试指标均固定使用训练 LP 返回的截距。"
151             ),
152             "",
153         ]
154     )
155     output_path.write_text("\n".join(lines), encoding="utf-8")
156
157
158 def render_case33(metrics_path: Path, output_path: Path) -> None:
159     payload = json.loads(metrics_path.read_text(encoding="utf-8"))
160     rows = payload["cases"]
161     config = payload["config"]
162     if not rows:
163         raise ValueError(f"case33 metrics contain no cases: {metrics_path}")
164     limits = {
165         "aggregate_leaves": config["aggregate_time_limit_seconds_per_solve"],
166         "hybrid_full": config["hybrid_time_limit_seconds_per_solve"],
167     }
168     display_names = {
169         "aggregate_leaves": "四物理叶聚合",
170         "hybrid_full": "32 终端 hybrid",
171     }

```

```

172 lines = [
173     r"\begin{table}[htbp]",
174     r"\centering",
175     r"\caption{IEEE case33 派生算例的求解状态与路径结果}",
176     r"\label{tab:case33-status}",
177     r"\scriptsize",
178     r"\begin{tabularx}{\textwidth}{\lrrrr>\raggedright\arraybackslash}Xccr",
179     r"\toprule",
180     r"算例 & \$n\$ & 时限/s & 接受原子 & 选中原子 & 停止原因 & 状态全 0 & 全部有证书 & 总时间/s \\",
181     r"\midrule",
182 ]
183 for row in rows:
184     lines.append(
185         (r"{ } & { } & { } & { } & { } & { } & { } & { } & { } \\\").format(
186             display_names.get(row["case"], _escape(row["case"])),
187             int(row["terminal_count"]),
188             _fixed(limits[row["case"]], 0),
189             int(row["accepted_path_length_excluding_zero"]),
190             int(row["selected_atom_count"]),
191             _stop_reason(row["stop_reason"]),
192             _yes_no(row["all_attempts_status_zero"]),
193             _yes_no(row["all_attempts_certified_optimal"]),
194             _fixed(row["fit_wall_seconds"], 2),
195         )
196     )
197 lines.extend(
198     [
199         r"\bottomrule",
200         r"\end{tabularx}",
201         r"\end{table}",
202         "",
203         r"\begin{table}[htbp]",
204         r"\centering",
205         r"\caption{IEEE case33 派生算例的外推、矩阵与 clade 指标}",
206         r"\label{tab:case33-quality}",
207         r"\scriptsize",
208         r"\begin{tabular}{\lrrrrrr}",
209         r"\toprule",
210         r"算例 & 训练 MAE & 验证 MAE & 测试 MAE & \$R\$ 相对 Fro. & \$X\$ 相对 Fro. & F1 \\",
211         r"\midrule",
212     ]
213 )
214 for row in rows:
215     lines.append(
216         (r"{ } & { } & { } & { } & { } & { } & { } \\\").format(
217             display_names.get(row["case"], _escape(row["case"])),
218             _sci(row["train_mae"]),
219             _sci(row["validation_mae_fixed_training_intercept"]),
220             _sci(row["test_mae_fixed_training_intercept"]),
221             _sci(row["R_matrix_relative_frobenius_error"]),
222             _sci(row["X_matrix_relative_frobenius_error"]),
223             _fixed(row["nontrivial_support_f1"]),
224         )
225     )
226 hybrid = next((row for row in rows if row["case"] == "hybrid_full"), None)
227 lines.extend(
228     [
229         r"\bottomrule",

```

```

230         r"\end{tabular}",
231         r"\end{table}",
232         "",
233         r"\FloatBarrier",
234         (
235             r"\noindent 四叶聚合算例的 7 次扩展均取得严格证书，矩阵达到机器精度且两个"
236             r"非平凡 clade 全部匹配。"
237         ),
238     ]
239 )
240 if hybrid is not None:
241     lines.append(
242         (
243             r"32 终端算例在上界 "
244             + _fixed(hybrid["coefficient_bound"], 0)
245             + r"、单轮 "
246             + _fixed(limits["hybrid_full"], 0)
247             + r" 秒内状态为 "
248             + _escape(",".join(map(str, hybrid["attempt_statuses"])))
249             + r", 最大报告 gap 为 "
250             + _fixed(hybrid["maximum_reported_mip_gap"])
251             + r"。实现按设计拒绝未获证书的 incumbent，故空模型指标只表示规模压力测试，"
252             r"不能解释为已完成的 32 终端回归结果。"
253         )
254     )
255     lines.extend(
256         [
257             (
258                 r"\par\smallskip\textit{数据来源: }\path{"
259                 + metrics_path.as_posix()
260                 + r"}; 真值仅用于生成响应和事后评分，未用于候选支撑搜索。"
261             ),
262             "",
263         ]
264     )
265     output_path.write_text("\n".join(lines), encoding="utf-8")
266
267
268 def main() -> None:
269     parser = argparse.ArgumentParser()
270     parser.add_argument("--small6-summary", type=Path, default=DEFAULT_SMALL6)
271     parser.add_argument("--case33-metrics", type=Path, default=DEFAULT_CASE33)
272     parser.add_argument("--small6-tex", type=Path, default=DEFAULT_SMALL6_TEX)
273     parser.add_argument("--case33-tex", type=Path, default=DEFAULT_CASE33_TEX)
274     args = parser.parse_args()
275     render_small6(args.small6_summary, args.small6_tex)
276     render_case33(args.case33_metrics, args.case33_tex)
277
278
279 if __name__ == "__main__":
280     main()

```

Listing 6: experiments/render_laminar_l1_results_tex.py

F RX75-RNJ 联合算例驱动完整源码

```

1  """RX75-RNJ peripheral blocks followed by exact laminar L1-MILP completion.
2
3  The default data configuration intentionally matches the earlier high-F1 RNJ
4  benchmark: three noisy AC scenarios, 96 samples per scenario, daily demeaning,
5  ordered R/X projection, RX75 score, and RNJ tolerance factor 0.16.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 from collections import Counter
13 from pathlib import Path
14 from time import perf_counter
15
16 import numpy as np
17 import pandas as pd
18
19 from rnj_wzst.helpers import (
20     RECIPE,
21     _boundary_cherries,
22     _family_score,
23     _moving_block_bootstrap_copy,
24     _select_disjoint,
25     _simulate_pool,
26     _terminal_buses,
27     _truth_nontrivial_clades,
28 )
29 from rnj_wzst.data.paper_style_case_bank import CASE_BUILDERS
30 from rnj_wzst.termination.laminar_l1_milp import (
31     fit_laminar_l1_sensitivity,
32     solver_diagnostics_prove_optimality,
33 )
34 from rnj_wzst.termination.multiscenario import (
35     fit_projected_sensitivity,
36     preprocess_scenarios,
37 )
38 from rnj_wzst.graph.rooted_hierarchy import (
39     PseudoCluster,
40     aggregate_rooted_scenarios,
41     rooted_clades,
42 )
43 from rnj_wzst.graph.rooted_neighbor_joining import rooted_neighbor_joining
44 from rnj_wzst.graph.sensitivity_geometry import sensitivity_geometry
45 from rnj_wzst.utils.io import ensure_dir
46
47
48 DEFAULT_OUTPUT = Path("outputs/mainline")
49 DEFAULT_CASES = ("paper15", "soumalas11", "flynn16", "pengwah18")
50
51
52 def _serialize(clade: frozenset[int]) -> str:
53     return ",".join(map(str, sorted(clade)))
54
55

```

```

56 def _serialize_family(family: set[frozenset[int]]) -> str:
57     return ";".join(
58         _serialize(clade)
59         for clade in sorted(family, key=lambda item: (len(item), tuple(sorted(item))))
60     )
61
62
63 def _rx75_rnj_clades(
64     scenarios: list[dict],
65     terminals: list[int],
66     root: int,
67     *,
68     tolerance_factor: float,
69 ) -> tuple[set[frozenset[int]], float, float]:
70     r_matrix, x_matrix, r2_score, condition = fit_projected_sensitivity(
71         scenarios,
72         constraint_mode="ordered",
73     )
74     geometry = sensitivity_geometry(r_matrix, x_matrix, "RX_75R_25X")
75     scale = max(float(np.median(geometry.root_depths)), 1e-12)
76     tree = rooted_neighbor_joining(
77         geometry.shared_paths,
78         geometry.root_depths,
79         terminals,
80         root,
81         group_tolerance=tolerance_factor * scale,
82     )
83     return rooted_clades(tree.edges, root, terminals), float(r2_score), float(condition)
84
85
86 def _select_boundary_blocks(
87     scenarios: list[dict],
88     terminals: list[int],
89     root: int,
90     *,
91     bootstrap_replicates: int,
92     block_length: int,
93     confidence_threshold: float,
94     maximum_candidate_count: int,
95     tolerance_factor: float,
96     seed: int,
97 ) -> tuple[set[frozenset[int]], dict[frozenset[int], float], list[frozenset[int]], float, float]:
98     full_clades, r2_score, condition = _rx75_rnj_clades(
99         scenarios,
100         terminals,
101         root,
102         tolerance_factor=tolerance_factor,
103     )
104     full_cherries = _boundary_cherries(full_clades)
105     counter: Counter = Counter()
106     rng = np.random.default_rng(seed)
107     for _ in range(bootstrap_replicates):
108         sampled = _moving_block_bootstrap_copy(scenarios, rng, block_length)
109         sampled_clades, _, _ = _rx75_rnj_clades(
110             sampled,
111             terminals,
112             root,
113             tolerance_factor=tolerance_factor,

```

```

114     )
115     counter.update(_boundary_cherries(sampled_clades))
116     confidence = {
117         clade: counter[clade] / bootstrap_replicates
118         for clade in set(counter) | full_cherries
119     }
120     stable = {
121         clade
122         for clade in full_cherries
123         if confidence[clade] >= confidence_threshold
124     }
125     selected = _select_disjoint(stable, confidence, maximum_candidate_count)
126     return full_clades, confidence, selected, r2_score, condition
127
128
129 def _align_scenario_names(reference: list[dict], candidate: list[dict]) -> list[dict]:
130     if len(reference) != len(candidate):
131         raise ValueError("scenario lists must have the same length")
132     aligned: list[dict] = []
133     for reference_item, candidate_item in zip(reference, candidate, strict=True):
134         copied = dict(candidate_item)
135         copied["name"] = str(reference_item["name"])
136         aligned.append(copied)
137     return aligned
138
139
140 def _nontrivial_result_clades(result, terminal_count: int) -> set[frozenset[int]]:
141     return {
142         frozenset(map(int, labels))
143         for labels in result.support_labels
144         if 1 < len(labels) < terminal_count
145     }
146
147
148 def _contracted_milp_inputs(
149     training_raw: list[dict],
150     validation_raw: list[dict],
151     training_preprocessed: list[dict],
152     terminals: list[int],
153     selected: list[frozenset[int]],
154     confidence: dict[frozenset[int], float],
155     *,
156     deembedding_weight: float,
157 ) -> tuple[list[dict], list[dict], list[tuple[int, ...]], dict[int, frozenset[int]]]:
158     """Contract RNJ cherries and freeze every reduced leaf-edge singleton.
159
160     Every observed terminal in the reduced problem is a physical leaf. Its
161     singleton atom is therefore structural, rather than a topology candidate:
162     ordinary pseudo terminals represent original terminal leaf edges, while a
163     contracted pseudo terminal represents the trusted RNJ boundary block.
164     """
165
166     r_matrix, x_matrix, _, _ = fit_projected_sensitivity(
167         training_preprocessed,
168         constraint_mode="ordered",
169     )
170     clusters = [
171         PseudoCluster(

```

```

172         pseudo_id=900000 + index,
173         members=clade,
174         confidence=float(confidence[clade]),
175         frozen_clades=tuple(),
176         frozen_sibling_pairs=tuple(),
177     )
178     for index, clade in enumerate(selected)
179 ]
180 contracted_training_raw, pseudo_members = aggregate_rooted_scenarios(
181     training_raw,
182     terminals,
183     r_matrix,
184     x_matrix,
185     clusters,
186     voltage_mode="deembedded_vsqs",
187     deembedding_weight=deembedding_weight,
188 )
189 contracted_validation_raw, validation_members = aggregate_rooted_scenarios(
190     validation_raw,
191     terminals,
192     r_matrix,
193     x_matrix,
194     clusters,
195     voltage_mode="deembedded_vsqs",
196     deembedding_weight=deembedding_weight,
197 )
198 if validation_members != pseudo_members:
199     raise RuntimeError("training and validation pseudo mappings differ")
200 contracted_training = preprocess_scenarios(contracted_training_raw, RECIPE)
201 contracted_validation = _align_scenario_names(
202     contracted_training,
203     preprocess_scenarios(contracted_validation_raw, RECIPE),
204 )
205 pseudo_labels = list(contracted_training[0]["P_terminal"].columns)
206 pseudo_position = {int(label): index for index, label in enumerate(pseudo_labels)}
207 frozen_singletons = [(pseudo_position[int(label)],) for label in pseudo_labels]
208 return contracted_training, contracted_validation, frozen_singletons, pseudo_members
209
210
211 def _leaf_singletons(terminal_count: int) -> list[tuple[int, ...]]:
212     """Return the structurally known leaf-edge atoms for a terminal-only tree."""
213
214     if terminal_count < 1:
215         raise ValueError("terminal_count must be positive")
216     return [(index,) for index in range(terminal_count)]
217
218
219 def _map_clade_to_reduced_support(
220     clade: frozenset[int],
221     reduced_labels: list[int],
222     pseudo_members: dict[int, frozenset[int]],
223 ) -> tuple[int, ...] | None:
224     """Map an original-terminal clade to reduced positional indices.
225
226     A proposal that cuts through a contracted trusted block is deliberately
227     rejected because it would contradict the frozen RNJ boundary decision.
228     """
229

```

```

230     support: list[int] = []
231     covered: set[int] = set()
232     for index, label in enumerate(reduced_labels):
233         members = pseudo_members[int(label)]
234         overlap = members & clade
235         if overlap and overlap != members:
236             return None
237         if overlap:
238             support.append(index)
239             covered.update(members)
240     if covered != set(clade):
241         return None
242     return tuple(support)
243
244
245 def _rnj_reduced_candidate_pool(
246     full_rnj_clades: set[frozenset[int]],
247     reduced_labels: list[int],
248     pseudo_members: dict[int, frozenset[int]],
249     *,
250     include_one_edit: bool,
251 ) -> tuple[tuple[int, ...], ...]:
252     """Build a finite, truth-free MILP proposal pool from the RX75 hierarchy."""
253
254     terminal_count = len(reduced_labels)
255     base: set[tuple[int, ...]] = set()
256     for clade in full_rnj_clades:
257         support = _map_clade_to_reduced_support(clade, reduced_labels, pseudo_members)
258         if support is not None and 1 < len(support) <= terminal_count:
259             base.add(support)
260     candidates = set(base)
261     if include_one_edit:
262         universe = set(range(terminal_count))
263         for support in base:
264             current = set(support)
265             for added in universe - current:
266                 enlarged = tuple(sorted((*current, added)))
267                 if 1 < len(enlarged) <= terminal_count:
268                     candidates.add(enlarged)
269             if len(current) > 2:
270                 for removed in current:
271                     reduced = tuple(sorted(current - {removed}))
272                     if len(reduced) > 1:
273                         candidates.add(reduced)
274     return tuple(sorted(candidates, key=lambda item: (len(item), item)))
275
276
277 def _expand_candidate_pool(
278     supports: tuple[tuple[int, ...], ...],
279     reduced_labels: list[int],
280     pseudo_members: dict[int, frozenset[int]],
281     terminal_count: int,
282 ) -> set[frozenset[int]]:
283     expanded: set[frozenset[int]] = set()
284     for support in supports:
285         members = frozenset().union(
286             *(pseudo_members[int(reduced_labels[index])] for index in support)
287         )

```

```

288         if 1 < len(members) < terminal_count:
289             expanded.add(members)
290         return expanded
291
292
293 def _expand_pseudo_result_clades(
294     result,
295     pseudo_members: dict[int, frozenset[int]],
296     terminal_count: int,
297 ) -> set[frozenset[int]]:
298     expanded: set[frozenset[int]] = set()
299     for support in result.support_labels:
300         members = frozenset().union(*(pseudo_members[int(label)] for label in support))
301         if 1 < len(members) < terminal_count:
302             expanded.add(members)
303     return expanded
304
305
306 def run(
307     output_dir: str | Path = DEFAULT_OUTPUT,
308     *,
309     cases: tuple[str, ...] = DEFAULT_CASES,
310     scenario_count: int = 3,
311     samples_per_scenario: int = 96,
312     training_replicate: int = 0,
313     validation_replicate: int = 1,
314     pq_noise_rel: float = 0.005,
315     voltage_noise_rel: float = 0.0002,
316     tolerance_factor: float = 0.16,
317     bootstrap_replicates: int = 100,
318     block_length: int = 4,
319     confidence_threshold: float = 0.75,
320     maximum_candidate_count: int = 2,
321     selection_only: bool = True,
322     run_baseline: bool = True,
323     contract_blocks: bool = False,
324     deembedding_weight: float = 0.5,
325     candidate_pool_mode: str = "rnj",
326     time_limit: float = 1800.0,
327     coefficient_bound: float = 2.0,
328 ) -> dict:
329     if samples_per_scenario < 96:
330         raise ValueError("at least 96 samples per scenario are required")
331     if scenario_count < 1:
332         raise ValueError("scenario_count must be positive")
333     if candidate_pool_mode not in {"unrestricted", "rnj", "rnj_one_edit"}:
334         raise ValueError("unknown candidate_pool_mode")
335     output = ensure_dir(output_dir)
336     candidate_rows: list[dict] = []
337     summary_rows: list[dict] = []
338     milp_rows: list[dict] = []
339
340     for case_name in cases:
341         net, training_raw = _simulate_pool(
342             case_name,
343             samples_per_scenario,
344             training_replicate,
345             scenario_count,

```

```

346         pq_noise_rel,
347         voltage_noise_rel,
348     )
349     _, validation_raw = _simulate_pool(
350         case_name,
351         samples_per_scenario,
352         validation_replicate,
353         scenario_count,
354         pq_noise_rel,
355         voltage_noise_rel,
356     )
357     training = preprocess_scenarios(training_raw, RECIPE)
358     validation = _align_scenario_names(
359         training,
360         preprocess_scenarios(validation_raw, RECIPE),
361     )
362     terminals = _terminal_buses(net)
363     root = int(net.root_bus)
364     truth_clades = _truth_nontrivial_clades(net, terminals)
365     truth_cherries = _boundary_cherries(truth_clades)
366     started = perf_counter()
367     full_clades, confidence, selected, r2_score, condition = _select_boundary_blocks(
368         training,
369         terminals,
370         root,
371         bootstrap_replicates=bootstrap_replicates,
372         block_length=block_length,
373         confidence_threshold=confidence_threshold,
374         maximum_candidate_count=maximum_candidate_count,
375         tolerance_factor=tolerance_factor,
376         seed=20_260_903 + training_replicate,
377     )
378     selection_seconds = perf_counter() - started
379     full_cherries = _boundary_cherries(full_clades)
380     full_score = _family_score(full_clades, truth_clades)
381     cherry_score = _family_score(full_cherries, truth_cherries)
382     selected_score = _family_score(set(selected), truth_cherries)
383     summary_rows.append(
384         {
385             "case": case_name,
386             "scenario_count": scenario_count,
387             "samples_per_scenario": samples_per_scenario,
388             "total_training_rows": scenario_count * samples_per_scenario,
389             "fit_r2": r2_score,
390             "condition_number": condition,
391             "selection_seconds": selection_seconds,
392             "full_rnj_clade_precision": full_score["nontrivial_support_precision"],
393             "full_rnj_clade_recall": full_score["nontrivial_support_recall"],
394             "full_rnj_clade_f1": full_score["nontrivial_support_f1"],
395             "full_rnj_cherry_precision": cherry_score["nontrivial_support_precision"],
396             "full_rnj_cherry_recall": cherry_score["nontrivial_support_recall"],
397             "full_rnj_cherry_f1": cherry_score["nontrivial_support_f1"],
398             "selected_count": len(selected),
399             "selected_supports": ";".join(_serialize(clade) for clade in selected),
400             "selected_confidences": ";".join(f"{confidence[clade]:.6g}" for clade in selected),
401             "selected_precision_against_truth": selected_score["nontrivial_support_precision"],
402             "selected_recall_against_truth_cherries": selected_score["nontrivial_support_recall"],
403             "true_cherries": _serialize_family(truth_cherries),

```

```

404     }
405 )
406 for clade in sorted(
407     set(confidence) | full_cherries,
408     key=lambda item: (len(item), tuple(sorted(item))),
409 ):
410     candidate_rows.append(
411         {
412             "case": case_name,
413             "support_labels": _serialize(clade),
414             "support_size": len(clade),
415             "present_in_full_tree": clade in full_cherries,
416             "bootstrap_confidence": confidence.get(clade, 0.0),
417             "passes_threshold": (
418                 clade in full_cherries
419                 and confidence.get(clade, 0.0) >= confidence_threshold
420             ),
421             "selected_top_k": clade in selected,
422             "is_true_boundary_cherry": clade in truth_cherries,
423         }
424     )
425 pd.DataFrame(summary_rows).to_csv(output / "rnj_summary.csv", index=False)
426 pd.DataFrame(candidate_rows).to_csv(output / "rnj_boundary_candidates.csv", index=False)
427
428 if selection_only:
429     continue
430 position = {label: index for index, label in enumerate(terminals)}
431 selected_supports = [
432     tuple(position[label] for label in sorted(clade)) for clade in selected
433 ]
434 leaf_singletons = _leaf_singletons(len(terminals))
435 initial_supports = [*leaf_singletons, *selected_supports]
436 fit_variants = [("rx75_rnj_plus_milp", initial_supports)]
437 if run_baseline:
438     fit_variants.insert(0, ("milp_only", leaf_singletons))
439 for initializer, supports in fit_variants:
440     fit_training = training
441     fit_validation = validation
442     pseudo_members = None
443     if initializer == "rx75_rnj_plus_milp" and contract_blocks:
444         (
445             fit_training,
446             fit_validation,
447             supports,
448             pseudo_members,
449         ) = _contracted_milp_inputs(
450             training_raw,
451             validation_raw,
452             training,
453             terminals,
454             selected,
455             confidence,
456             deembedding_weight=deembedding_weight,
457         )
458     initializer = "rx75_rnj_contracted_plus_milp"
459     reduced_labels = [
460         int(label) for label in fit_training[0]["P_terminal"].columns
461     ]

```

```

462     if pseudo_members is None:
463         pseudo_members = {
464             label: frozenset({label}) for label in reduced_labels
465         }
466     candidate_supports = None
467     candidate_clades: set[frozenset[int]] = set()
468     if initializer != "milp_only" and candidate_pool_mode != "unrestricted":
469         candidate_supports = _rnj_reduced_candidate_pool(
470             full_clades,
471             reduced_labels,
472             pseudo_members,
473             include_one_edit=candidate_pool_mode == "rnj_one_edit",
474         )
475         candidate_clades = _expand_candidate_pool(
476             candidate_supports,
477             reduced_labels,
478             pseudo_members,
479             len(terminals),
480         )
481     fit_started = perf_counter()
482     result = fit_laminar_l1_sensitivity(
483         fit_training,
484         validation_scenarios=fit_validation,
485         initial_supports=supports,
486         candidate_supports=candidate_supports,
487         r_upper_bound=coefficient_bound,
488         x_upper_bound=coefficient_bound,
489         time_limit=time_limit,
490     )
491     elapsed = perf_counter() - fit_started
492     predicted = (
493         _nontrivial_result_clades(result, len(terminals))
494         if pseudo_members is None
495         else _expand_pseudo_result_clades(result, pseudo_members, len(terminals))
496     )
497     score = _family_score(predicted, truth_clades)
498     missing = truth_clades - predicted
499     extra = predicted - truth_clades
500     milp_rows.append(
501         {
502             "case": case_name,
503             "initializer": initializer,
504             "rnj_initial_supports": ";".join(
505                 _serialize(clade) for clade in selected
506             ),
507             "frozen_support_count": len(supports),
508             "frozen_leaf_singleton_count": len(result.terminal_labels),
509             "frozen_rnj_block_count": len(selected),
510             "candidate_pool_mode": (
511                 "unrestricted" if candidate_supports is None else candidate_pool_mode
512             ),
513             "candidate_pool_count": (
514                 "" if candidate_supports is None else len(candidate_supports)
515             ),
516             "candidate_pool_only_truth_recall": (
517                 ""
518                 if candidate_supports is None
519                 else _family_score(candidate_clades, truth_clades)[

```

```

520         "nontrivial_support_recall"
521     ]
522 ),
523 "candidate_pool_plus_frozen_truth_recall": (
524     ""
525     if candidate_supports is None
526     else _family_score(
527         candidate_clades | set(selected), truth_clades
528     )["nontrivial_support_recall"]
529 ),
530 "candidate_pool_clades": _serialize_family(candidate_clades),
531 "milp_terminal_count": len(result.terminal_labels),
532 "elapsed_seconds": elapsed,
533 "support_count": len(result.support_labels),
534 "nontrivial_support_count": len(predicted),
535 "clade_precision": score["nontrivial_support_precision"],
536 "clade_recall": score["nontrivial_support_recall"],
537 "clade_f1": score["nontrivial_support_f1"],
538 "train_mae": result.train_mae,
539 "validation_mae": result.validation_mae,
540 "stop_reason": result.stop_reason,
541 "attempt_count": len(result.attempted_extensions),
542 "selected_path_index": result.selected_path_index,
543 "path_length": len(result.path),
544 "path_support_counts": ";".join(
545     str(len(point.support_labels)) for point in result.path
546 ),
547 "path_validation_mae": ";".join(
548     "" if point.validation_mae is None else f"{point.validation_mae:.12g}"
549     for point in result.path
550 ),
551 "attempt_statuses": ";".join(
552     str(attempt.diagnostics.status)
553     for attempt in result.attempted_extensions
554 ),
555 "attempts_proven_optimal": sum(
556     solver_diagnostics_prove_optimality(attempt.diagnostics)
557     for attempt in result.attempted_extensions
558 ),
559 "attempt_runtimes": ";".join(
560     f"{attempt.diagnostics.runtime_seconds:.6g}"
561     for attempt in result.attempted_extensions
562 ),
563 "attempt_objectives": ";".join(
564     "" if attempt.diagnostics.objective is None
565     else f"{attempt.diagnostics.objective:.12g}"
566     for attempt in result.attempted_extensions
567 ),
568 "attempt_dual_bounds": ";".join(
569     "" if attempt.diagnostics.dual_bound is None
570     else f"{attempt.diagnostics.dual_bound:.12g}"
571     for attempt in result.attempted_extensions
572 ),
573 "attempt_mip_gaps": ";".join(
574     "" if attempt.diagnostics.mip_gap is None
575     else f"{attempt.diagnostics.mip_gap:.12g}"
576     for attempt in result.attempted_extensions
577 ),

```

```

578         "attempt_messages": " | ".join(
579             attempt.diagnostics.message.replace("\n", " ")
580             for attempt in result.attempted_extensions
581         ),
582         "predicted_clades": _serialize_family(predicted),
583         "true_clades": _serialize_family(truth_clades),
584         "missing_clades": _serialize_family(missing),
585         "extra_clades": _serialize_family(extra),
586     }
587 )
588 pd.DataFrame(milp_rows).to_csv(output / "milp_results.csv", index=False)
589
590 payload = {
591     "config": {
592         "cases": list(cases),
593         "scenario_count": scenario_count,
594         "samples_per_scenario": samples_per_scenario,
595         "training_replicate": training_replicate,
596         "validation_replicate": validation_replicate,
597         "pq_noise_rel": pq_noise_rel,
598         "voltage_noise_rel": voltage_noise_rel,
599         "preprocessing": RECIPE["name"],
600         "constraint_mode": "ordered",
601         "distance_mode": "RX_75R_25X",
602         "tolerance_factor": tolerance_factor,
603         "bootstrap_replicates": bootstrap_replicates,
604         "bootstrap_kind": "circular_moving_block",
605         "block_length": block_length,
606         "confidence_threshold": confidence_threshold,
607         "maximum_candidate_count": maximum_candidate_count,
608         "selection_only": selection_only,
609         "run_baseline": run_baseline,
610         "contract_blocks": contract_blocks,
611         "deembedding_weight": deembedding_weight,
612         "candidate_pool_mode": candidate_pool_mode,
613         "time_limit_seconds_per_extension": time_limit,
614         "truth_used_for_candidate_generation": False,
615     },
616     "rnj": summary_rows,
617     "milp": milp_rows,
618 }
619 (output / "metrics.json").write_text(
620     json.dumps(payload, indent=2, ensure_ascii=False, allow_nan=False),
621     encoding="utf-8",
622 )
623 return payload
624
625
626 def main() -> None:
627     parser = argparse.ArgumentParser()
628     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
629     parser.add_argument("--cases", nargs="+", choices=DEFAULT_CASES, default=list(DEFAULT_CASES))
630     parser.add_argument("--scenario-count", type=int, default=3)
631     parser.add_argument("--samples-per-scenario", type=int, default=96)
632     parser.add_argument("--bootstrap-replicates", type=int, default=100)
633     parser.add_argument("--block-length", type=int, default=4)
634     parser.add_argument("--confidence-threshold", type=float, default=0.75)
635     parser.add_argument("--maximum-candidate-count", type=int, default=2)

```

```

636 parser.add_argument(
637     "--time-limit",
638     type=float,
639     default=1800.0,
640     help="HiGHS wall-time limit for each one-block MILP extension",
641 )
642 parser.add_argument("--run-milp", action="store_true")
643 parser.add_argument("--hybrid-only", action="store_true")
644 parser.add_argument("--contract-blocks", action="store_true")
645 parser.add_argument("--deembedding-weight", type=float, default=0.5)
646 parser.add_argument(
647     "--candidate-pool-mode",
648     choices=("unrestricted", "rnj", "rnj_one_edit"),
649     default="rnj",
650 )
651 args = parser.parse_args()
652 run(
653     args.output,
654     cases=tuple(args.cases),
655     scenario_count=args.scenario_count,
656     samples_per_scenario=args.samples_per_scenario,
657     bootstrap_replicates=args.bootstrap_replicates,
658     block_length=args.block_length,
659     confidence_threshold=args.confidence_threshold,
660     maximum_candidate_count=args.maximum_candidate_count,
661     selection_only=not args.run_milp,
662     run_baseline=not args.hybrid_only,
663     contract_blocks=args.contract_blocks,
664     deembedding_weight=args.deembedding_weight,
665     candidate_pool_mode=args.candidate_pool_mode,
666     time_limit=args.time_limit,
667 )
668
669
670 if __name__ == "__main__":
671     main()

```

Listing 7: experiments/run_rx75_rnj_milp_hybrid_ac.py

G RX75-RNJ 联合回归测试完整源码

```

1 from types import SimpleNamespace
2
3 import pandas as pd
4 import pytest
5
6 import experiments.run_rx75_rnj_milp_hybrid_ac as hybrid
7
8
9 def test_leaf_singletons_cover_every_reduced_terminal() -> None:
10     assert hybrid._leaf_singletons(4) == [(0,), (1,), (2,), (3,)]
11     with pytest.raises(ValueError, match="positive"):
12         hybrid._leaf_singletons(0)
13
14
15 def test_pseudo_supports_expand_to_original_terminal_clades() -> None:
16     result = SimpleNamespace(
17         support_labels=((900000,), (900000, 3), (3,), (900000, 3, 4))
18     )
19     pseudo_members = {
20         900000: frozenset({1, 2}),
21         3: frozenset({3}),
22         4: frozenset({4}),
23     }
24
25     expanded = hybrid._expand_pseudo_result_clades(
26         result,
27         pseudo_members,
28         terminal_count=4,
29     )
30
31     assert expanded == {
32         frozenset({1, 2}),
33         frozenset({1, 2, 3}),
34     }
35
36
37 def test_contraction_freezes_all_reduced_leaf_singletons(monkeypatch) -> None:
38     columns = [900000, 3, 4]
39     training = [{"name": "train", "P_terminal": pd.DataFrame([[1.0, 2.0, 3.0]], columns=columns)}]
40     validation = [{"name": "validation", "P_terminal": pd.DataFrame([[2.0, 3.0, 4.0]], columns=columns)}]
41     members = {
42         900000: frozenset({1, 2}),
43         3: frozenset({3}),
44         4: frozenset({4}),
45     }
46
47     monkeypatch.setattr(
48         hybrid,
49         "fit_projected_sensitivity",
50         lambda *_args, **_kwargs: (None, None, 1.0, 1.0),
51     )
52
53     def fake_aggregate(scenarios, *_args, **_kwargs):
54         return scenarios, members
55

```

```

56 monkeypatch.setattr(hybrid, "aggregate_rooted_scenarios", fake_aggregate)
57 monkeypatch.setattr(hybrid, "preprocess_scenarios", lambda scenarios, _recipe: scenarios)
58
59 _, _, frozen, returned_members = hybrid._contracted_milp_inputs(
60     training,
61     validation,
62     training,
63     terminals=[1, 2, 3, 4],
64     selected=[frozenset({1, 2})],
65     confidence={frozenset({1, 2}): 0.9},
66     deembedding_weight=0.5,
67 )
68
69 assert frozen == [(0,), (1,), (2,)]
70 assert returned_members == members
71
72
73 def test_rnj_candidate_pool_maps_contracted_blocks_and_adds_one_edits() -> None:
74     reduced_labels = [900000, 3, 4, 5]
75     members = {
76         900000: frozenset({1, 2}),
77         3: frozenset({3}),
78         4: frozenset({4}),
79         5: frozenset({5}),
80     }
81     full_clades = {
82         frozenset({1, 2}),
83         frozenset({1, 2, 3}),
84         frozenset({4, 5}),
85     }
86
87     base = hybrid._rnj_reduced_candidate_pool(
88         full_clades,
89         reduced_labels,
90         members,
91         include_one_edit=False,
92     )
93     augmented = hybrid._rnj_reduced_candidate_pool(
94         full_clades,
95         reduced_labels,
96         members,
97         include_one_edit=True,
98     )
99
100     assert base == ((0, 1), (2, 3))
101     assert (0, 1, 2) in augmented
102     assert (1, 2, 3) in augmented
103     assert hybrid._map_clade_to_reduced_support(
104         frozenset({1, 3}), reduced_labels, members
105     ) is None
106
107     expanded = hybrid._expand_candidate_pool(
108         base,
109         reduced_labels,
110         members,
111         terminal_count=5,
112     )
113     assert expanded == {

```

```
114     frozenset({1, 2, 3}),  
115     frozenset({4, 5}),  
116 }
```

Listing 8: tests/test_rx75_rnj_milp_hybrid_ac.py